



UNIVERSIDAD DE GRANADA

TRABAJO FIN DE GRADO
INGENIERÍA INFORMÁTICA

StayIn

Portal de búsqueda de vivienda

Autor

Carlos Hoyo Liddle

Directores

Carmen Martínez Cruz



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Granada, a junio de 2026

StayIn: Portal de búsqueda de vivienda

Carlos Hoyo Liddle

Palabras clave: full-stack, portal inmobiliario, Scrum, sistema de recomendaciones, sistema de valoraciones, convivencia, NLP, microservicios, compatibilidad entre compañeros

Resumen

Este proyecto trata el proceso de desarrollo de software de una aplicación centrada en la búsqueda de vivienda y compañeros. El objetivo principal de esta aplicación es ofrecer una forma personalizada de realizar esta búsqueda, basándose en un sistema de recomendaciones y valoraciones entre usuarios, cuya finalidad es ofrecer las mayores probabilidades de compatibilidad en función de las preferencias de los usuarios y de las valoraciones recibidas por gente con la que han convivido realmente. De esta manera, se trata de ayudar a todas las partes; tanto a la gente que busca piso, como a aquellos caseros/inquilinos que buscan ocupar un hueco libre en la vivienda, sin alterar la buena convivencia en ella.

El sistema integra una capa visual de interfaz de usuario desarrollada en React Native y Expo, conectada a un servidor desarrollado mediante Node.js y Express, que se comunicará con la base de datos, desarrollada mediante PostgreSQL y alojada en un contenedor a través de Docker. El sistema de recomendaciones se ha implementado en un microservicio aparte, desarrollado en Python junto con FastAPI, con el objetivo de mantener toda la capa de recomendaciones centralizada y apartada del servidor principal. El sistema cuenta también con un panel de administración desarrollado para escritorio mediante React y Vite que, a través del servidor del *backend*, permite la interacción con la base de datos y la realización de las funcionalidades básicas de un administrador del sistema.

Durante el desarrollo de este trabajo se ha seguido una metodología de desarrollo incremental basada en Scrum, usando todos los conceptos e ideas principales de esta metodología y adaptándolos a un equipo de una persona.

Los resultados obtenidos han cumplido la base principal de la idea del proyecto, pero no ha sido posible implementar el sistema al completo en el tiempo disponible, dejando fuera la implementación de funcionalidades más específicas pero necesarias para poder presentar el proyecto como un producto finalizado y listo para producción.

Este proceso de investigación, planificación, implementación y desarrollo han servido para sentar las bases principales del sistema y permitir la mejora continua del mismo más allá de la entrega de este trabajo, además de dar lugar a distintas adaptaciones de la idea inicial que, una vez puesto en marcha el proceso de creación de la aplicación, han requerido modificaciones de cara a la calidad del producto final, la experiencia de usuario y la utilidad de la aplicación en entornos de producción reales.

Todo el código de este proyecto se puede encontrar alojado y disponible para consulta en el repositorio remoto de GitHub [1].

Keywords: full-stack, real estate portal, Scrum, recommendation system, rating system, coliving, NLP, microservices, roommate matching

Abstract

This project addresses the software development process of an application focused on the search for housing and roommates. Its main goal is to provide a personalized way of carrying out this search, based on a system of recommendations and user-to-user ratings whose purpose is to maximize the likelihood of compatibility according to users' preferences and to the ratings they have received from people they have actually lived with. In this way, the application aims to help all parties involved: both those looking for a place to live and the landlords or tenants seeking to fill a vacancy in a property without disrupting the good coexistence within it.

The system integrates a user interface layer built with React Native and Expo, connected to a server developed with Node.js and Express, which communicates with the database, built with PostgreSQL and hosted in a container through Docker. The recommendation system has been implemented as a separate microservice, developed in Python together with FastAPI, in order to keep the entire recommendation layer centralized and decoupled from the main server. The system also includes an administration panel developed for desktop with React and Vite which, through the *backend* server, allows interaction with the database and the performance of the basic functions of a system administrator.

Throughout the development of this work, an incremental development methodology based on Scrum has been followed, applying all the main concepts and principles of this methodology and adapting them to a one-person team.

The results obtained have fulfilled the core idea of the project, although it was not possible to implement the system in full within the available time, leaving out more specific functionalities that are nonetheless necessary to present the project as a finished, production-ready product.

This process of research, planning, implementation and development has served to lay the main foundations of the system and to enable its continuous improvement beyond the submission of this work, in addition to giving rise to various adaptations of the initial idea which, once the application's creation process was underway, required modifications with a view to the quality of the final product, the user experience and the usefulness of the application in real production environments.

All the code for this project can be found hosted and available for consultation in the remote GitHub repository [1].

Contenido

1.	ESPECIFICACIÓN DEL TRABAJO	12
1.1	Introducción.....	12
1.2	Contexto y justificación	12
1.3	Motivación	13
1.4	Objetivos.....	13
1.4.1	Objetivo general.....	13
1.4.2	Objetivos específicos	13
1.4.3	Objetivos personales	14
1.4.4	Estructura del documento	14
2.	ESTADO DEL ARTE.....	16
2.1	Sistemas de recomendación	16
2.1.1	Cómo funcionan los sistemas de recomendación.....	16
2.1.2	Tipos de sistemas de recomendación	17
2.2	Modelos de Procesamiento de Lenguaje Natural (NLP).....	18
2.3	Soluciones existentes al problema abordado.....	18
2.3.1	Portales generales	19
2.3.2	Aplicaciones	20
2.3.3	Inmobiliarias.....	20
2.3.4	Análisis comparativo	21
3.	GESTIÓN Y PLANIFICACIÓN DEL PROYECTO	23
3.1	Análisis de tecnologías	23
3.1.1	Desarrollo del <i>frontend</i>	24
3.1.2	Desarrollo del <i>backend</i>	25
3.1.3	Base de datos	26
3.1.4	Sistema de recomendaciones	27
3.1.5	Qué configuración utilizar para mi proyecto	27
3.1.6	Valoración final sobre las tecnologías escogidas	28
3.2	Metodología de desarrollo.....	28
3.2.1	Adaptación de la metodología al proyecto.....	30
3.2.2	Lista de Historias de Usuario.....	30
3.2.3	Product Backlog del proyecto.....	48
3.3	Planificación temporal	49
3.3.1	Planificación de <i>Sprints</i>	49
3.3.2	Diagrama de Gantt.....	50
3.4	Análisis de riesgos.....	53
3.5	Presupuesto	54

3.5.1	Estimación inicial del presupuesto	54
3.5.2	Estimación con recuento de horas aproximado	57
3.6	Aspectos legales	58
3.6.1	Protección de datos personales.....	58
3.6.2	Tratamiento de datos de terceros y derecho al honor.....	58
3.6.3	Condición de prestador de servicios de la sociedad de la información	59
3.6.4	Normativa de arrendamiento y vivienda.....	59
3.6.5	Propiedad intelectual y licencias de software.....	59
4.	ANÁLISIS DEL SISTEMA	60
4.1	Definición de actores.....	60
4.1.1	Jerarquía de actores.....	60
4.1.2	Actores principales.....	60
4.1.3	Actores secundarios.....	60
4.2	Descripción del problema.....	60
4.2.1	Roles de usuario	60
4.2.2	Estados	61
4.2.3	Conexiones.....	61
4.2.4	Valoraciones.....	62
4.2.5	Aspectos de la convivencia entre usuarios	63
4.2.6	Aspectos de la estancia en un inmueble	64
4.2.7	Anuncios de las viviendas	65
4.3	Requisitos funcionales.....	66
4.3.1	Gestión de usuarios y roles.....	66
4.3.2	Gestión de anuncios de viviendas.....	66
4.3.3	Gestión de valoraciones	67
4.3.4	Gestión de conexiones entre usuarios y viviendas	67
4.3.5	Gestión de recomendaciones.....	67
4.4	Requisitos no funcionales	67
4.5	Casos de uso.....	68
5.	DISEÑO DEL SISTEMA.....	73
5.1	Arquitectura del sistema	73
5.2	Diseño de datos	74
5.2.1	Tablas esenciales.....	76
5.2.2	Tablas hijas	76
5.2.3	Tablas específicas.....	77
5.2.4	Integración con el sistema	78
5.3	Navegabilidad entre pantallas	78
5.3.1	Flujo Sign Up	78

5.3.2	Flujo Home.....	79
5.3.3	Flujo buscar	79
5.3.4	Flujo chat.....	79
5.3.5	Flujo social.....	79
5.3.6	Flujo perfil.....	79
5.4	Diseño de interfaz UI/UX	80
5.4.1	Logo de la aplicación	80
5.4.2	Mockups y prototipos	80
6.	IMPLEMENTACIÓN Y DESARROLLO	84
6.1	Gestión integral de perfil y estados (Sprint 1).....	84
6.1.1	Lógica de base de datos.....	84
6.1.2	Implementación en el <i>backend</i>	84
6.1.3	Implementación en el <i>frontend</i>	85
6.2	Publicación y gestión de anuncios (Sprint 2).....	86
6.2.1	Lógica de base de datos.....	86
6.2.2	Implementación en el <i>backend</i>	86
6.2.3	Implementación en el <i>frontend</i>	87
6.2.4	Desviaciones e imprevistos	87
6.3	Vinculaciones y social (Sprint 3)	88
6.3.1	Lógica de base de datos.....	88
6.3.2	Implementación en el <i>backend</i>	88
6.3.3	Implementación en el <i>frontend</i>	89
6.4	Sistema de valoraciones (Sprint 4).....	89
6.4.1	Lógica de base de datos.....	89
6.4.2	Implementación en el <i>backend</i>	90
6.4.3	Implementación en el <i>frontend</i>	90
6.5	Sistema de recomendación (Sprint 5)	91
6.5.1	Arquitectura general del sistema de recomendación híbrido.....	92
6.5.2	Cálculo de la puntuación: capas comunes a ambos motores.....	93
6.5.3	Anuncios	98
6.5.4	Compañeros	106
6.5.5	Gestión de la memoria.....	110
6.5.6	Diferencias clave entre ambos motores	111
6.5.7	Reflejo en el <i>frontend</i> de las recomendaciones calculadas.....	112
6.5.8	Desviaciones y complicaciones.....	112
6.6	Sistema de búsqueda (Sprint 5).....	112
6.6.1	Contratiempos y modificaciones necesarias en el sistema	113
6.6.2	Conexiones externas y APIs necesarias para la implementación	113

6.6.3	Arquitectura e implementación del sistema de búsqueda	116
6.7	Panel de administración (Sprint 6).....	116
6.7.1	Seguridad y control de acceso.....	116
6.7.2	Componentes del panel de administración.....	117
6.7.3	Moderación del contenido	118
6.7.4	Motor de búsqueda dinámico.....	119
6.8	Cierre de la implementación.....	119
7.	CONCLUSIONES Y TRABAJO FUTURO	120
7.1	Conclusiones	120
7.1.1	Evaluación de objetivos planteados.....	120
7.2	Trabajo futuro	121
7.2.1	Integración de Machine Learning en el motor de recomendaciones.....	121
7.2.2	Optimización de parámetros arbitrarios en los cálculos del motor de recomendación	121
7.2.3	Implementación web dedicada.....	122
7.2.4	Finalización de una primera versión estable y despliegue de la aplicación	122
7.3	Valoración final.....	122
8.	ANEXO	123
8.1	Creación de tablas de la base de datos	123
8.2	Creación de instancias.....	126
8.3	Sistema de recomendaciones	128
8.4	Sistema de búsqueda	130
9.	USO DE IA EN ESTE TRABAJO.....	133
10.	BIBLIOGRAFÍA	134

Índice de tablas

Tabla 1. Comparativa de los portales de búsqueda de vivienda más relevantes del mercado 21

Tabla 2. Cobertura de las funcionalidades diferenciales de StayIn en las soluciones existentes.

.....	22
Tabla 3. HU-1. Login de usuarios	31
Tabla 4. HU-2. Edición del perfil.....	32
Tabla 5. HU-3. Recuperar contraseña	33
Tabla 6. HU-4. Elección de rol.....	33
Tabla 7. HU-5. Simultaneidad de roles	34
Tabla 8. HU-6. Estado de búsqueda	34
Tabla 9. HU-7. Estado de hospedaje	35
Tabla 10. HU-8. Publicación de anuncios	36
Tabla 11. HU-9. Edición y eliminación de anuncios.....	36
Tabla 12. HU-10. Envío de solicitudes de registro en viviendas	37
Tabla 13. HU-11. Recepción de solicitudes de registro en viviendas.....	37
Tabla 14. HU-12. Registro de inquilinos en viviendas	38

Tabla 15. HU-13. Control de anuncios	39
Tabla 16. HU-14. Valoraciones Inquilino-Inquilino	39
Tabla 17. HU-15. Valoraciones Inquilino-Casero	40
Tabla 18. HU-16. Valoraciones Inquilino- Vivienda	41
Tabla 19. HU-17. Valoraciones Casero-Inquilino	42
Tabla 20. HU-18. Sistema de búsqueda.....	43
Tabla 21. HU-19. Chat	43
Tabla 22. HU-20. Conexión entre usuarios	44
Tabla 23. HU-21. Multiplataforma.....	44
Tabla 24. HU-22. Privacidad de la cuenta.....	45
Tabla 25. HU-23. Verificación de tiempo de convivencia en valoraciones.....	46
Tabla 26. HU-24. Sistema de recomendaciones	46
Tabla 27. HU-25. Eliminación de anuncios por administrador	47
Tabla 28. HU-26. Control de valoraciones.....	47
Tabla 29. HU-27. Baneo y suspensión de cuentas	48
Tabla 30. Product Backlog	49
Tabla 31. Sprint 1	49
Tabla 32. Sprint 2	50
Tabla 33. Sprint 3	50
Tabla 34. Sprint 4	50
Tabla 35. Sprint 5	50
Tabla 36. Sprint 6	50
Tabla 37. Matriz de valoración de nivel de riesgo	53
Tabla 38. Identificación y valoración de riesgos del proyecto	53
Tabla 39. Plan de prevención y mitigación de los riesgos.....	54
Tabla 40. Porcentajes de contratación indefinida a cargo de la empresa en España	55
Tabla 41. Licencias de software usadas en el proyecto.....	56
Tabla 42. Logos de StayIn.....	80
Tabla 43. Mockups de StayIn	83
Tabla 44. Organización por capas del sistema de recomendaciones	92
Tabla 45. Interacciones tenidas en cuenta.....	96
Tabla 46. Peso conservado por una interacción según su antigüedad (h = 14 días)	97
Tabla 47. Representación módulos de preferencias en anuncios	100
Tabla 48. Pesos de módulos y campos internos en preferencias de anuncios.....	101
Tabla 49. Ejemplo de recomendación de anuncio.....	106
Tabla 50. Diferencias entre motores de recomendación.....	111
Tabla 51. Comparación de servicios externos para sistema de búsqueda	115

Índice de ilustraciones

Ilustración 1. Distribución de larga cola.....	16
Ilustración 2. Lenguajes de programación más usados en 2025.....	24
Ilustración 3. Lenguajes de programación con mayor crecimiento en 2022	26
Ilustración 4. Diferencia entre programa clásico y Machine Learning	27
Ilustración 5. Diagrama de Gantt	52
Ilustración 6. Diagrama de actores y casos de uso del sistema.....	72
Ilustración 7. Diagrama de la arquitectura del sistema	73
Ilustración 8. Diagrama de tablas de la base de datos Fuente: diagrama exportado de BD real de DataGrip [106]	75
Ilustración 9. Sitemap de la aplicación.....	78
Ilustración 10. Logo StayIn Negro	80
Ilustración 11. Logo StayIn Gris	80
Ilustración 12. Logo StayIn Blanco	80
Ilustración 13. Mockup Home de inquilino	81
Ilustración 14. Mockup pantalla de búsqueda	81
Ilustración 15. Mockup Home inquilino	81
Ilustración 16. Mockup Home casero	81
Ilustración 17. Mockup valoraciones pendientes	82
Ilustración 18. Mockup pantalla valoraciones	82
Ilustración 19. Mockup añadir multimedia a anuncio	83
Ilustración 20. Pantalla de login de StayIn (modo claro)	86
Ilustración 21. Home del usuario Casero	87
Ilustración 22. Tarjeta de anuncio editable por parte del casero	87
Ilustración 23. Pantalla social con valoración pendiente.....	91
Ilustración 24. Valoración a usuario.....	91
Ilustración 25. Barra de navegación con notificación pendiente	91
Ilustración 26. Home del inquilino con recomendaciones	112
Ilustración 27. Pop Up para test de personalidad	112

D^a. **Carmen J. Martínez Cruz**, Profesora del Departamento de Lenguajes y Sistemas Informáticos de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado **StayIn, Portal de búsqueda de vivienda**, ha sido realizado bajo su supervisión por **Carlos Hoyo Liddle**, y se autoriza la defensa de dicho trabajo ante el tribunal que corresponda.

Agradecimientos

En primer lugar, me gustaría agradecer a la tutora de este trabajo, Carmen Martínez Cruz, por haber accedido a guiarme en el proceso de elaboración de este proyecto y haber aceptado mis propuestas en todo momento.

A la Escuela Técnica Superior de Ingenierías Informática y de Telecomunicaciones, por los conocimientos transmitidos tras los 5 años que he pasado como alumno, sin lo cuales sería imposible poder realizar un proyecto como el presentado.

Por último, también me gustaría tomar el tiempo para agradecer a mis padres y a mi familia, ya que son la causa principal por la que he podido llegar hasta aquí, los que han escuchado y valorado mis propuestas acerca de este trabajo y los que han visto el empeño y dedicación con el que se han realizado cada una de sus fases, ayudándome a levantar el ánimo en los momentos más importantes. Sin vosotros nada de esto habría sido posible.

1. ESPECIFICACIÓN DEL TRABAJO

1.1 Introducción

La búsqueda de vivienda es un proceso al que casi todas las personas se enfrentan en algún momento, ya sea por la necesidad de trasladarse a otra ciudad para estudiar, por el deseo de independizarse o, simplemente, por un cambio de domicilio.

Este proceso conlleva, en muchos casos, la necesidad de compartir piso con otras personas. Esta circunstancia es especialmente habitual entre la población joven, que con frecuencia busca alojamiento de forma independiente y dispone de recursos económicos más limitados.

Compartir piso puede convertirse en una experiencia muy positiva, e incluso en una de las mejores etapas en la vida de una persona, sobre todo cuando se comparte con amigos o personas conocidas, o cuando surge una buena conexión con los compañeros de convivencia.

Sin embargo, también es frecuente que ocurra lo contrario: una convivencia incómoda en la que las tensiones terminan pesando más que las ventajas de haberse emancipado. Es en este escenario donde aparecen los conflictos de convivencia.

Precisamente este es uno de los motivos por los que surge la idea de desarrollar una aplicación cuyo objetivo principal sea mejorar la búsqueda de vivienda y de compañeros de piso, facilitando tanto ese proceso como la posterior convivencia entre los usuarios.

En definitiva, este proyecto persigue transformar un proceso habitualmente tedioso, y a menudo pospuesto, en una tarea más sencilla y fiable.

1.2 Contexto y justificación

El interés de este proyecto no responde únicamente a una percepción personal, sino que trata de ser una solución a uno de los problemas más comunes a la hora de convivir con gente desconocida. Uno de los grandes problemas que presenta la juventud actual es el acceso a la vivienda, especialmente en España, siendo la tasa de emancipación juvenil del 14,5% [2], el mínimo histórico registrado, con una edad media que ya supera los 30 años [2]. Además, el 70% de esas personas lo hace a través de alquiler, de los cuales, el 87% comparte vivienda con el objetivo de reducir gastos [3], ya que el precio de alquiler medio (1.176€) equivale al 98,7% del sueldo medio de una persona joven [2]. Es decir, en España, se puede independizar aproximadamente 1 de cada 7 jóvenes y, de cada 10 que lo hacen, 6 se ven en la necesidad de compartir piso.

Ante el obvio problema existente en el país, es evidente que compartir piso ya no es únicamente cosa de estudiantes, sino que la mayoría de la gente joven independizada se ve en la necesidad de hacerlo para poder permitírselo. Con esto se abre un nuevo problema que sumar al ya comentado: ahora la dificultad no solo es encontrar una vivienda que se ajuste a lo que la persona busca, sino que también es necesario encontrar gente compatible con la que compartir ese espacio.

El público objetivo de una aplicación centrada en la convivencia compartida y en la compatibilidad de esta no es, por tanto, un nicho reducido, sino más bien todo lo contrario. Esta convivencia conlleva compartir espacios a diario con gente desconocida en muchos casos, lo que puede derivar muy fácilmente en problemas por incompatibilidad de hábitos, falta de información fiable sobre la persona o vivienda en cuestión y ausencia de mecanismos de confianza. El problema principal, la búsqueda de la vivienda en sí, ya está extensamente abordado por diferentes portales de vivienda e inmobiliarias, como se detalla en la sección 2.3.4 del documento, pero es el segundo de esos problemas donde no suelen llegar las principales plataformas, al objetivo de maximizar la compatibilidad entre compañeros. No solo es necesario encontrar el inmueble, sino que la plataforma también podría ayudar a encontrar aquella gente con la que compartirlo no se convierta en una experiencia desastrosa, que obligue a la persona a comenzar de nuevo ese proceso de búsqueda de piso desde cero. Es por esto por lo que se ha planteado este proyecto.

1.3 Motivación

La motivación principal del proyecto surge precisamente de esa segunda carencia. Mientras que la búsqueda de vivienda está ampliamente cubierta por los portales actuales, la de personas compatibles con las que convivir apenas se aborda, y no por casualidad: la compatibilidad en la convivencia es un problema difícil de tratar de forma automática, ya que depende de hábitos, costumbres y preferencias subjetivas y poco estructuradas. Precisamente ese reto de transformar información dispersa y subjetiva en recomendaciones útiles y fiables, es el que justifica afrontarlo desde la ingeniería informática, aplicando técnicas de sistemas de recomendación y procesamiento de lenguaje natural capaces de modelar la afinidad entre usuarios, en lugar de limitarse a la búsqueda y filtrado de anuncios.

A este factor se suma el de la confianza, igual de determinante. Gran parte de los conflictos al compartir vivienda nacen de la falta de información fiable, tanto sobre las personas como sobre los inmuebles; de hecho, según el Consejo de la Juventud de España, la mitad de los jóvenes que viven de alquiler ha tenido problemas con su arrendador [3]. Una herramienta que ayude a anticipar la compatibilidad entre convivientes y que, además, aporte fiabilidad mediante un sistema de valoraciones y un control de los contenidos publicados puede reducir de forma tangible el riesgo de una mala experiencia, que no solo arruina la convivencia, sino que obliga a la persona a reiniciar todo el proceso de búsqueda desde cero. El impacto potencial, por tanto, no es menor, dado el tamaño del colectivo afectado.

Por último, en el plano personal, este proyecto nace de una idea propia, concebida y planteada por iniciativa propia a partir de la observación directa del problema. Esa naturaleza autoiniciada constituye en sí misma una motivación: la de llevar un producto desde su concepción hasta una implementación funcional completa, recorriendo todo el ciclo de desarrollo (análisis, diseño, desarrollo full-stack e integración de un microservicio de recomendación) con una verdadera visión de producto. StayIn no se plantea únicamente como un ejercicio académico, sino con vocación de viabilidad real y, eventualmente, de iniciativa emprendedora, una perspectiva que ha guiado muchas de las decisiones de alcance y diseño tomadas a lo largo del trabajo.

1.4 Objetivos

1.4.1 Objetivo general

El objetivo principal de este TFG es diseñar y desarrollar un producto de software para ayudar en la gestión de la convivencia y los alquileres de vivienda, conectando de forma segura tanto a inquilinos como a caseros, fomentando la transparencia en todo momento, mediante la implementación de mecanismos de valoraciones y recomendaciones personalizadas, combinadas con un control de los anuncios, logrando así crear un ecosistema de confianza entre los usuarios del sistema. Este objetivo general se ha articulado en torno a la consecución de los objetivos específicos mencionados a continuación.

1.4.2 Objetivos específicos

- OE1: Definición y estructura del sistema

Definir y estructurar los requisitos fundamentales para el correcto funcionamiento del sistema.

- OE2: Sistema de autenticación y gestión de identidades

Implementar un sistema de gestión de perfiles de manera que un usuario pueda autenticarse en el sistema con sus datos, siendo estos editables en cualquier momento y permitiendo la elección de un rol (inquilino o casero) que le permita hacer uso de la aplicación para sacar el mayor beneficio posible.

- OE3: Implementar la lógica de gestión y vinculación de estancias

Desarrollar la publicación y gestión de anuncios de viviendas, así como la lógica de vinculación inquilino-vivienda mediante el envío y la aceptación de solicitudes de registro en una vivienda.

- OE3: Sistema de valoraciones

Implementar un sistema de valoraciones que permita evaluar la convivencia entre compañeros y la experiencia en los inmuebles, condicionado a la existencia de una relación de convivencia previa y verificable.

- OE4: Sistema de recomendaciones

Desarrollar un sistema de recomendaciones híbrido, de forma que a todos los usuarios del sistema les sean recomendados ciertos perfiles o viviendas en función de sus preferencias, interacciones con la aplicación, búsquedas, etc. con el objetivo de maximizar la compatibilidad de convivencia.

- OE5: Permitir la administración eficiente y efectiva del sistema

Implementar las funcionalidades de un administrador del sistema que pueda garantizar el correcto funcionamiento de este, así como el cumplimiento de las normas establecidas sobre los diferentes aspectos de la aplicación.

- OE6: Aplicación multiplataforma fluida

Crear una aplicación móvil y web, de manera que la experiencia de usuario y fluidez conseguidas sean del más alto nivel posible, permitiendo al usuario conectarse desde cualquiera de sus dispositivos.

1.4.3 Objetivos personales

- OP1: Aprendizaje de los *frameworks* a utilizar

Aprender el framework React Native junto con Expo y aplicarlo a un proyecto real.

1.4.4 Estructura del documento

Esta memoria está dividida en capítulos, secciones y subsecciones, por lo que se va a presentar un breve resumen de cómo se ha estructurado el documento y qué se puede encontrar en cada capítulo de este a partir de ahora.

- Capítulo 2. Estado del arte

Este capítulo va a conllevar la sección de investigación principal del documento, tratando el dominio de problema, los diferentes conceptos relevantes al proyecto a realizar y los trabajos relacionados, lo que también conlleva un análisis de la competencia.

- Capítulo 3. Gestión y planificación del proyecto

Se centrará en abordar el análisis de las posibles tecnologías a usar en este proyecto, junto con la elección y justificación del *stack* elegido; así como en la organización y planificación del desarrollo.

- Capítulo 4. Análisis del sistema

En este capítulo se definirán los principales actores de este proyecto, la descripción detallada del problema a abordar y el planteamiento estructurado de la solución.

- Capítulo 5. Diseño del sistema

Esta sección se centrará en la presentación del diseño de la solución planteada, adjuntando diagramas y figuras relevantes durante la implementación, con el objetivo de dar un mayor

entendimiento al lector de qué es lo que se ha planteado y cómo, así como definir la estructuración de los datos y la estructura final de la aplicación.

- Capítulo 6. Implementación y desarrollo

Aquí se definirá toda la implementación técnica llevada a cabo para construir los diferentes módulos y servicios definidos en el capítulo anterior, tratando de plasmar las decisiones tomadas y las implementaciones finales propuestas, así como las desviaciones y problemas encontrados en el proceso.

- Capítulo 7. Conclusiones y trabajo futuro

El capítulo final en cuanto al desarrollo de la aplicación planteada, con todas las valoraciones obtenidas en el proceso de desarrollo de esta solución, así como la mención y explicación de aquellos detalles propuestos como trabajo a futuro.

- Capítulo 8. Anexo

Este capítulo se usará para añadir todos los fragmentos de código mencionados durante el documento cuya longitud dificulte la correcta lectura del documento.

- Capítulo 9. Bibliografía

El capítulo donde se recogerán todas las fuentes citadas a lo largo del documento.

2. ESTADO DEL ARTE

En este capítulo se va a llevar a cabo una revisión del estado del arte actual, explicando todos los conceptos teóricos y tecnologías relacionadas y evaluando propuestas que hayan abordado el mismo problema; estudiando las soluciones dadas, las necesidades que cubren y, sobre todo, aquellas que no.

2.1 Sistemas de recomendación

Los sistemas de recomendación son una forma (o la forma) moderna de filtrar, seleccionar y presentar información relevante a los usuarios; que se ven, cada vez más, bombardeados por información de todo tipo en internet y redes sociales.

Funcionan mediante el uso de IA y se encargan de sugerir elementos a un usuario basándose en el análisis de sus datos y patrones de comportamiento.

Estos motores ayudan a los usuarios a encontrar información, productos o servicios basados en sus gustos y preferencias que, quizá, no habrían encontrado por sí mismos; además de jugar un papel muy importante en el beneficio de las empresas modernas, mejorando la experiencia del usuario y la retención de clientes, lo que se traduce en un aumento de entre un 5% y un 15% en los ingresos [4].

Los motores de recomendación han supuesto un antes y un después en la industria en general, propiciando la aparición del fenómeno conocido como *long tail*, popularizado por Chris Anderson [5], que explica en su libro cómo un elemento que en una tienda física no era rentable; debido a que ocupaba el mismo espacio físico que otro que vendía mucho más; comienza a tener valor cuando ese espacio no cuesta nada. Esto es exactamente lo que ocurre en el comercio online (con empresas como Amazon o Netflix), dando lugar a una curva en la que nunca se llega al 0, ya que todos los elementos se ven representados en un nicho individual de mercado, habiendo siempre alguien que lo consuma. Este tipo de curvas en estadística se conocen como *distribuciones de larga cola*, término que da lugar al nombre del fenómeno [6], [7], [8].

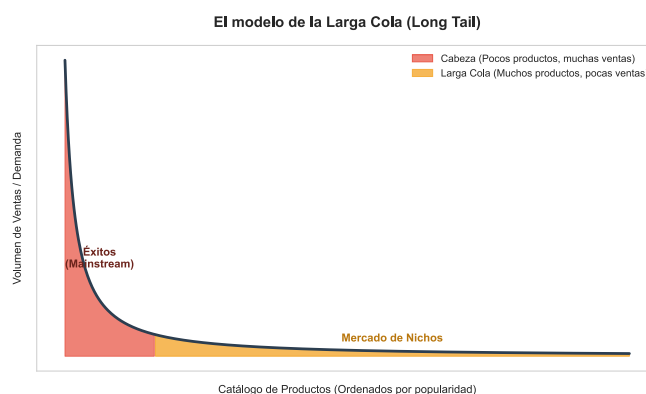


Ilustración 1. Distribución de larga cola.

Fuente: elaboración propia a partir de [5].

2.1.1 Cómo funcionan los sistemas de recomendación

Los sistemas de recomendación tratan de predecir qué productos o elementos pueden interesarle a cada usuario. Para esto, suelen funcionar en cinco fases [7].

- Recopilación de datos

Los datos son el pilar fundamental de un sistema de recomendación. Existen dos tipos de datos, los datos explícitos y los datos implícitos. Los primeros se centran en las acciones y actividades de los usuarios, como los comentarios o “me gustas” en redes sociales; mientras que los segundos

tratan de comprender los gustos del usuario en base a su historial de búsqueda y compras, cesta de compra en páginas, etc.

Los motores de recomendación también hacen uso de otros tipos de datos, como los datos demográficos y psicográficos, con el fin de encontrar usuarios similares.

- Almacenamiento

Una vez recopilados los datos, se hace uso de un sistema de almacenamiento, que variará en función de los datos obtenidos. Los sistemas de almacenamiento podrán ser de tres tipos:

- Almacén de datos

Puede agregar datos de diferentes fuentes.

- Data lake

Pueden almacenar datos estructurados y no estructurados.

- Lakehouse de datos

Combina los mejores aspectos de los almacenes de datos y de los data lakes.

- Análisis

En esta fase se hace uso de algoritmos de aprendizaje automático y técnicas de filtrado para generar las recomendaciones.

- Filtrado

En el punto final se lleva a cabo la filtración de los datos mediante el uso de reglas y fórmulas matemáticas que dependerán del tipo de sistema de recomendación utilizado. Todos los tipos de sistemas de recomendación se pueden ver en el punto 2.1.2.

- Refinamiento

También se puede añadir el paso de refinamiento de los datos en función de los datos obtenidos, de manera que se vayan obteniendo cada vez mejores resultados.

2.1.2 Tipos de sistemas de recomendación

La principal distinción entre un sistema de recomendación y otro es el método de filtrado que utiliza cada uno. Hay tres tipos principales de sistemas de recomendación, cada uno con unas aplicaciones distintas según las necesidades de cada caso [6], [7], [8].

- Filtrado colaborativo

El filtrado colaborativo agrupa usuarios en función de comportamientos similares, basándose en la idea de la sabiduría de la multitud para realizar sugerencias. Supone que un conjunto de usuarios que han tenido gustos comunes en el pasado, los seguirán teniendo en el futuro.

Este tipo de sistemas son ampliamente utilizados por empresas enormes, como pueden ser Spotify [9], Amazon [10] o Meta [11].

Son eficaces sin la necesidad de conocer grandes descripciones de los productos, pero pueden sufrir cuando los datos históricos de los que disponen son escasos o limitados.

Pueden ser, a su vez, de dos tipos distintos: basados en memoria (o vecindad) y basados en modelos [6], [7], [8].

- Basados en memoria

Se representa a los usuarios y elementos como una matriz y se busca a los vecinos cercanos que puedan ser similares, de la misma forma que funciona el algoritmo K-Nearest Neighbours (kNN) [12].

- Basados en modelos

Utilizan los datos (la matriz anterior, por ejemplo) para aprender a predecir los valores que faltan, de forma que sugiera a un usuario aquellos elementos que aún no ha encontrado. Se encarga

de, en base al entrenamiento obtenido con los datos, sugerir ítems al usuario que seguramente le interesen.

- **Filtrado basado en contenido**

Es un método de recuperación de información, de manera que pueda recomendar al usuario ítems relacionados con aquellos por los que ha mostrado preferencia en el pasado.

Representa al usuario y a los ítems como vectores en un espacio vectorial, de manera que la similitud entre ítems se determinará en función de la cercanía entre unos y otros, ofreciendo al usuario aquellos que mayor similitud tengan, o lo que es lo mismo, aquellos que más próximos entre sí estén.

En este tipo de filtrado se hacen dos distinciones de perfiles: por un lado, está el perfil de ítems; que describe el conjunto de características que definen al ítem; y, por otro lado, el perfil de usuario, que trata de representar los gustos del usuario en función de sus valoraciones pasadas.

Las recomendaciones se obtienen mediante una predicción de la valoración que el usuario dará a los ítems, basándose en la similitud entre el perfil del usuario y el de los ítems.[6], [7], [8], [13], [14].

- **Sistema de recomendación híbrido**

Un sistema de recomendación híbrido combina el filtrado colaborativo con el filtrado basado en contenido, llegando a dar grandes mejoras en el rendimiento de un motor de recomendación, aunque supone arquitecturas complejas y muy intensas computacionalmente [7].

Este tipo de sistemas son los que usa la plataforma de streaming Netflix [15].

En este proyecto se va a implementar un algoritmo de recomendación con el objetivo de que a cada usuario del sistema le sean mostrados una serie de perfiles y/o viviendas en función de sus interacciones y preferencias, generando una experiencia más personalizada y eficaz en la búsqueda de compañeros, inquilinos o viviendas.

2.2 Modelos de Procesamiento de Lenguaje Natural (NLP)

El procesamiento de lenguaje natural (NLP) es el campo de la inteligencia artificial que permite a las máquinas interpretar y representar el lenguaje humano [16]. Este tipo de modelos, aunque muy presentes en el mundo actual (NLP), remontan su comienzo a 1950, con el ampliamente conocido en el sector informático test de Turing [17], creado por Alan Turing [18], [19] para medir el comportamiento inteligente de las máquinas. Este experimento es llevado a cabo mediante la conversación a través de una serie de preguntas entre un humano y un ordenador, donde el primero tendrá que identificar, en función de las respuestas recibidas, si habla con un humano o con una máquina [20].

En este proyecto resulta especialmente relevante una de sus aplicaciones: la representación semántica del texto mediante *embeddings*, vectores numéricos que codifican el significado de una palabra o frase de modo que dos textos de sentido parecido quedan próximos en el espacio vectorial [21]. Estos vectores se obtienen mediante modelos preentrenados de tipo *transformer* [22] (en concreto, de la familia *sentence-transformers* [23]) y la afinidad entre dos textos se mide habitualmente mediante la similitud del coseno entre sus vectores [24]. Esta técnica es la que permite, en el sistema desarrollado, comparar automáticamente descripciones y preferencias expresadas en lenguaje libre, complementando a los criterios estructurados del motor de recomendación.

2.3 Soluciones existentes al problema abordado

Hoy en día existen muchas maneras distintas de buscar vivienda y compañeros, cada una de ellas con unos puntos fuertes y débiles, cada una centrada en un producto final y un determinado tipo de usuario. En este punto se pretenden enumerar y comparar todos los portales más populares

actualmente; con sus elementos destacables; y explicar por qué la idea planteada puede ser una mejora en este sector.

Para esta sección, se ha considerado la división de los portales en tres campos diferentes, cada uno de los cuales abarcando un determinado modelo de negocio y usabilidad.

Por un lado, están los portales generales, que definen plataformas web masivas, donde no hay un control exhaustivo de lo que se publica y donde se busca bombardear al usuario con variedad de opciones a elegir.

En segundo lugar, las aplicaciones; consideradas más de “nicho” y destinadas a la búsqueda de alquiler de habitaciones de corta duración (generalmente destinadas a estudiantes).

Por último, las inmobiliarias tradicionales; empresas destinadas a la gestión de la búsqueda de piso de sus clientes, brindando las mejores opciones en base a los requisitos y posibilidades de cada uno de ellos de forma tradicional. (Se considera tradicional a la gestión de la búsqueda mediante interacción real de agentes inmobiliarios con clientes a través de oficinas físicas).

2.3.1 Portales generales

- Idealista

Página web/aplicación donde se publican todo tipo de alquileres/ventas de viviendas. No hay control sobre los anuncios, se pueden publicar desde anuncios bien documentados y con amplia información de la vivienda, hasta anuncios con pocas o ninguna imagen útil.

Es, junto con algunas de las siguientes, uno de los portales más usados en España[25], con una amplia base de datos, posibilidad de dibujar una zona de búsqueda sobre el mapa, además de contar con filtros avanzados; como compra, alquiler o compartir; tipo de vivienda o zona geográfica, tamaño, precio, estado, número de habitaciones, estado del inmueble, etc.

También se pueden ordenar los inmuebles por precio, relevancia, fecha de publicación, rebaja, etc.

Ofrece, además, la posibilidad de marcar anuncios como favoritos, por los cuales se recibirán notificaciones cuando estén o dejen de estar disponibles, baje su precio o cualquier tipo de información relevante para el usuario interesado.

Cuenta también con presencia en Italia y Portugal [26], [27].

- Fotocasa

La alternativa a Idealista funciona de manera muy similar, aunque incluyendo herramientas visuales muy interesantes, como tours 3D por la vivienda. Cuenta, al igual que Idealista[26], con más de 1.5 millones de anuncios.

Dispone a su vez con una página web y aplicación móvil, que permiten el contacto con el arrendador de dos maneras posibles: o bien el arrendador permite la visibilidad de su número de teléfono en el propio anuncio, o mediante un formulario de contacto a través de la propia aplicación.

Una de las características más interesantes es la posibilidad de acceder a su mapa interactivo, donde se podrá acceder a la ubicación exacta del inmueble y los servicios próximo, permitiendo tener acceso a un calculador y comparador de las hipotecas de la zona, una valoración del barrio, búsqueda del inmueble por trayecto o ayuda para conseguir el certificado energético de la vivienda.

Ofrece un apartado (Fotocasalife) a modo de blog, donde se publican todo tipo de noticias relevantes para los usuarios de la aplicación [27], [28].

- Habitaclia

Especialmente popular en la zona de Cataluña, aunque con anuncios por toda España. Oferta de todo tipo de inmuebles y búsqueda visual por zonas y barrios dentro de las ciudades [25], [27], [29].

Presenta unas funcionalidades muy parecidas a las de Idealista[26] y Fotocasa[28].

- Pisos.com

Portal clásico, forma de búsqueda similar a la anterior [27], [30].

- YaEncontre

Alternativa generalista que replica el modelo de los grandes portales, sin aportar funcionalidades diferenciales relevantes frente a ellos [31].

2.3.2 Aplicaciones

- Badi

Aplicación para buscar habitación en pisos compartidos. Funciona a modo de “red social”, tratando de asegurar afinidad entre compañeros.

Servicio Badi plus, donde se muestran habitaciones preparadas para entrar en zonas seleccionadas en barrios concretos, con el fin de ofrecer un servicio especial y más exclusivo [25], [27], [32].

- Spotahome

Aplicación de búsqueda de piso por toda Europa, contando con más de 800 ciudades, su principal objetivo es facilitar la búsqueda de piso para larga duración para aquella gente que se mueva a otras ciudades, donde no conoce gente. Viviendas verificadas, pudiendo evitar las visitas físicas, teniendo la posibilidad de que acudan al inmueble los “Homecheckers” de la empresa, que opinarán y recomendarán acerca de alquilar o no la vivienda.

Tienen un gran servicio al cliente, ofreciendo cobertura si el propietario cancela a última hora, además de gestionar problemas o inconvenientes no esperados que se podrán comunicar durante las primeras 24 horas de estancia [27], [33].

- Housfy

Inmobiliaria digital que permite vender o alquilar viviendas contactando directamente con el propietario. Afirman la garantía de la venta del inmueble en un máximo de 60 días, tres veces más rápido que otros sitios web

Están centrados en la compraventa de inmuebles, aunque también ofrecen alquiler.

Tiene una amplia gama de servicios, entre los que figuran un simulador y comparador de hipotecas, calculadora de la rentabilidad del alquiler o un sistema de estimación del valor de tu inmueble [27], [34].

- Uniplaces

Plataforma de alquiler temporal popular entre estudiantes internacionales y jóvenes profesionales, con la posibilidad de buscar alojamiento por universidad [35].

2.3.3 Inmobiliarias

- Tecnocasa

Es la inmobiliaria más popular de toda España en lo que a gestión de vivienda de segunda mano y alquiler se refiere, contando con oficinas físicas en casi todos los barrios de las ciudades medianas y pequeñas de España [36].

- Redpiso

Inmobiliaria popular principalmente en las grandes ciudades y capitales. Tienen un sistema de financiación propio [37].

- RE/MAX España

Es una de las empresas líder en el sector, con un volumen de negocio superior a 45,3 millones de euros en 2024, 164 oficinas y más de 2300 agentes [38].

2.3.4 Análisis comparativo

En la Tabla 1 se puede ver una comparativa entre los portales de vivienda más relevantes del mercado actual, destacando su uso principal, punto fuerte, tipo de búsqueda implementada y nivel de uso en España [39], [40].

Herramienta	Uso principal	Punto fuerte	Búsqueda	Nivel de uso
Idealista	Compra/venta y alquiler	Muy popular, mucha oferta	Convencional y área 3D dibujada sobre el mapa	Muy alto
Fotocasa	Compra/venta y alquiler	Tecnologías 3D muy útiles y alta oferta	Convencional y área 3D dibujada sobre el mapa	Muy alto
Habitacalia	Compra/venta y alquiler	Popular en zona costera y Cataluña	Convencional	Medio
Pisos.com	Compra/venta y alquiler	Popular en toda España	Convencional	Medio
YaEncontre	Compra/venta y alquiler	Alternativa a las más conocidas	Varias opciones interesantes	Medio
Badi	Compartir piso	Contacto personal entre usuarios	Convencional y mediante Badi plus	Medio
Spotahome	Alquiler online	Alquiler internacional	Búsqueda convencional y mapa	Medio
Housfy	Compra/venta online	Combina herramientas digitales avanzadas con asesoramiento de inmobiliaria	Búsqueda convencional	Medio
Uniplaces	Alquiler estudiantil o juvenil	Dedicado a alquiler de estudiantes	Búsqueda convencional con la posibilidad de buscar por universidad	Bajo
Tecnocasa	Compra/venta y alquiler tradicional (en persona)	Gran red de oficinas físicas	Convencional o asistida por agentes inmobiliarios	Alto
Redpiso	Compra/venta y alquiler tradicional (en persona)	Popular en grandes ciudades	Convencional o asistida por agentes inmobiliarios	Medio
RE/MAX España	Compra/venta y alquiler tradicional (en persona)	De las más populares del país	Convencional o asistida por agentes inmobiliarios	Alto

Tabla 1. Comparativa de los portales de búsqueda de vivienda más relevantes del mercado

Fuente: elaboración propia a partir de datos referenciados en el punto anterior

Por otro lado, en la Tabla 2, podemos ver de forma más representativa la funcionalidad principal que el resto de las opciones del mercado no ofrecen, y donde StayIn les puede tomar ventaja.

Categoría	Búsqueda de compañeros compatibles	Valoraciones / reputación entre usuarios	Recomendación personalizada de convivencia	Enfoque en la convivencia
Portales generales	No	No	No (solo filtros)	No
Inmobiliarias	No	No	No (asesor humano)	No
Aplicaciones nicho	Parcial (Badi conecta usuarios, sin compatibilidad basada en datos)	No /muy limitado	No	Parcial (solo Badi)
StayIn	Sí	Sí	Sí	Sí

Tabla 2. Cobertura de las funcionalidades diferenciales de StayIn en las soluciones existentes.

Fuente: elaboración propia

Del análisis anterior se desprende que, pese a sus diferencias, las tres categorías comparten una misma limitación: todas resuelven el problema de encontrar una vivienda, pero ninguna aborda el de encontrar a las personas adecuadas con las que compartirla. Los portales generales y las inmobiliarias se centran por completo en el inmueble y no contemplan ni la compatibilidad entre convivientes, ni un sistema de valoraciones o reputación entre usuarios, ni una recomendación personalizada más allá de los filtros de búsqueda. Las aplicaciones de nicho se acercan algo más, destacando Badi, en particular, que conecta a posibles compañeros a modo de red social. No obstante, ninguna sustenta esa conexión en un sistema de valoraciones basado en convivencias reales ni en un motor de recomendación que estime la afinidad entre las personas. Es precisamente en ese hueco donde se sitúa StayIn.

En definitiva, la oferta actual cubre con solvencia la búsqueda y publicación de viviendas, con plataformas consolidadas y tecnologías muy maduras en la presentación de anuncios. La distinción de StayIn no está, por tanto, en ofrecer un portal de vivienda más, sino en incorporar a esas funcionalidades esperadas una experiencia personalizada basada en valoraciones y recomendaciones entre usuarios, orientada a maximizar la compatibilidad de convivencia y la confianza entre las partes. Esta es su principal ventaja competitiva y la razón por la que puede resultar especialmente útil para quienes, año tras año, se enfrentan a este problema. Identificada la competencia con sus puntos fuertes y débiles, se procede a la gestión y planificación del proyecto.

3. GESTIÓN Y PLANIFICACIÓN DEL PROYECTO

En este capítulo se tratan los aspectos relacionados con la gestión y la planificación del proyecto, tales como qué tecnologías existen como solución a este, cuáles se usarán y por qué, qué metodologías se seguirán y su correspondiente justificación. Además, también se realizará una planificación temporal del desarrollo del proyecto, el análisis de posibles riesgos y el presupuesto estimado (en caso de que este proyecto se llevara a cabo de manera profesional) que supondría la creación de esta plataforma. Para esta estimación se ha hecho uso de herramientas para la monitorización del tiempo dedicado a trabajar en distintos archivos, con el objetivo de reflejar la realidad lo más fielmente posible.

3.1 Análisis de tecnologías

Actualmente, existen todo tipo de tecnologías y *frameworks* enfocados al desarrollo web y de aplicaciones, todas con sus ventajas y sus inconvenientes. En este apartado enumero las tecnologías y configuraciones de desarrollo más populares, eligiendo la que considero más adecuada a mi proyecto.

En la Ilustración 2 podemos ver cuáles fueron los lenguajes de programación más usados en 2025, lo que nos puede dar un mejor contexto de cara a la valoración del *stack* tecnológico escogido para la implementación de este proyecto.

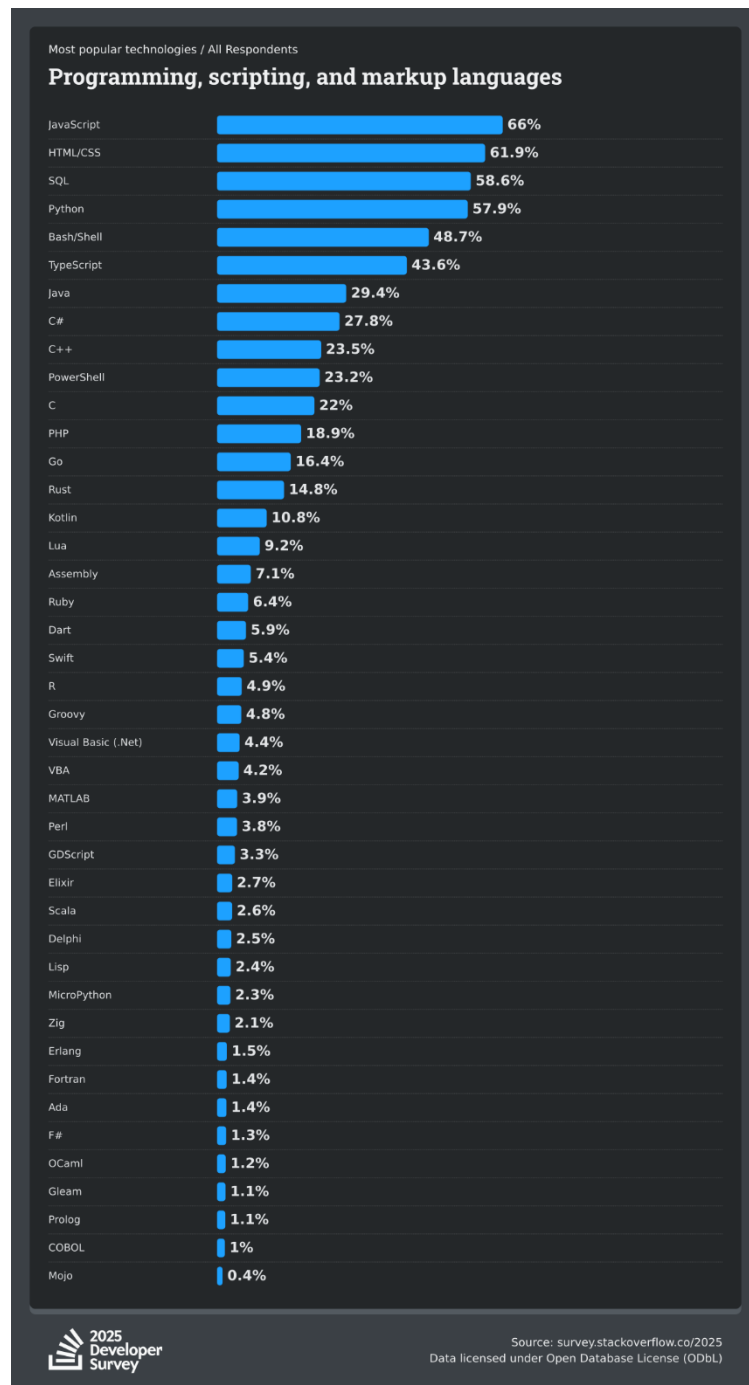


Ilustración 2. Lenguajes de programación más usados en 2025

Fuente: [41]

3.1.1 Desarrollo del *frontend*

La elección de las tecnologías a usar para el desarrollo del *frontend* es un aspecto muy importante en un trabajo de estas características, ya que acompañará a lo largo de todo el desarrollo de este, de manera que conviene tomar una decisión segura, con una investigación previa sobre todas las posibilidades, escogiendo la que mejor se adapte a nuestro proyecto y sus particularidades. Para esta decisión también conviene tener en cuenta el estándar de la industria, de manera que creemos un producto escalable y mantenible en el tiempo.

En este punto se van a evaluar las distintas configuraciones y tecnologías a usar para esta parte del proyecto.

- Desarrollo plano

Una forma de desarrollar este proyecto sería desarrollar la aplicación web mediante el uso de HTML, CSS y JavaScript plano y pasarlo a un diseño responsive mediante el uso de *media queries*. No obstante, este método está desfasado hoy en día y no se va a utilizar, ya que existen otros métodos para implementar este tipo de aplicaciones de forma mucho más ágil y moderna.

- Desarrollo mediante el uso de *frameworks*

- React Native + Expo

Es la forma de trabajar mediante JavaScript/TypeScript. React[42] es un framework de JavaScript desarrollado por Meta[11] utilizado para el desarrollo de aplicaciones web. Con el uso de React Native, se pueden crear aplicaciones móviles de forma nativa y React Native for Web[43] (integrado en Expo) para la parte web, mediante el uso de HTML.

Se puede compartir la mayoría de código y, al renderizar componentes de forma nativa, se logra un rendimiento fluido en todas las plataformas.

Se descarta directamente la opción de desarrollo mediante React Native para la parte móvil y React.js para la parte web ya que, con esta configuración, se compartiría la lógica de la aplicación, pero no el diseño de la interfaz de usuario, lo que conllevaría a la implementación del doble de código, algo poco óptimo para este caso, debido a que dificultaría el desarrollo del proyecto en el tiempo disponible, así como la escalabilidad y configuración de este.

- Flutter

Flutter[44] es un framework desarrollado por Google, permitiendo implementar interfaces de usuario para aplicaciones en diferentes plataformas mediante un único código base desarrollado en Dart[45]. Flutter pinta cada píxel de la pantalla por sí mismo, dando lugar a interfaces de usuario exactamente iguales en diferentes plataformas. Proporciona además un rendimiento casi nativo, por lo que es una gran opción para proyectos con características similares a este [44].

- Capacitor by Ionic (con React, Vue o Angular)

Con esta configuración, se podría crear la aplicación web en cualquier framework que se quisiera y, mediante Capacitor[46], envolverla en una app nativa, teniendo acceso a cámara, notificaciones, etc.

No obstante, al tener la aplicación en un navegador incrustado, el rendimiento móvil no es tan fluido como lo es con React o Flutter [47], [48].

3.1.2 Desarrollo del *backend*

Hay también muchas opciones distintas para desarrollar el *backend* de un proyecto. La decisión de cuál elegir dependerá también de la configuración elegida para el desarrollo del *frontend* de la aplicación.

- PHP

Es el lenguaje clásico en lo que al desarrollo *backend* se refiere. No obstante, pese a nunca llegar a desaparecer, en gran parte por su gran comunidad y su uso en plataformas populares como WordPress, hoy en día ya no es la primera opción para muchos casos, como lo podía ser antes, debido a la alta competencia de *frameworks* web que han ido apareciendo y que siguen haciéndolo a día de hoy [49], [50].

- Python

Uno de los lenguajes más populares y el más popular si se van a implementar algoritmos de IA o Machine Learning [49], [50].

En la Ilustración 3 podemos ver el crecimiento de Python en el año 2022, un fenómeno ocasionado principalmente por el increíble crecimiento de la ciencia de datos, IA y Machine Learning.



01 HCL	56.1%
02 Rust	50.5%
03 TypeScript	37.8%
04 Lua	34.2%
05 Go	28.3%
06 Shell	27.7%
07 Makefile	23.7%
08 C	23.5%
09 Kotlin	22.9%
10 Python	22.5%

Ilustración 3. Lenguajes de programación con mayor crecimiento en 2022

Fuente: [51]

- JavaScript (Node.js)

Es una de las mejores opciones si se va a realizar el desarrollo del *frontend* con React o React Native, ya que de esta forma se mantiene el mismo lenguaje en todo el proyecto, tanto *frontend* como *backend*, dando gran versatilidad a la hora de realizar proyectos full-stack. Se puede utilizar con el framework Nest.js [49].

- Go

Es el lenguaje (junto con Rust) perfecto para desarrollar sistemas de alto rendimiento, microservicios o sistemas cloud. Es muy rápido y también fácil de entender [49], [50].

- Java

Utilizado en grandes sistemas empresariales y requerido en sectores como la banca o las telecomunicaciones, presenta una gran robustez y rendimiento [49], [50].

- Ruby

Enfocado en la productividad del desarrollador, especialmente con el framework Ruby on Rails, es demandado en startups y proyectos que valoran la rapidez del desarrollo.[49]

- C#

Normalmente utilizado para aplicaciones empresariales y videojuegos por su integración con .NET [49].

3.1.3 Base de datos

Para la base de datos, el estándar en la industria ahora mismo se divide en 3 categorías: Relacional (SQL), NoSQL y bases de datos en el dispositivo; para tener acceso offline desde el móvil, por ejemplo [52].

- Relacional

El tipo de bases de datos más profundizada en la carrera. Los más populares son MySQL[53], PostgreSQL[54] y MariaDB[55], siendo el primero el más antiguo, el segundo el que más auge ha tenido recientemente y el tercero una derivación del primero [56], [57], [58].

- NoSQL

NoSQL es un enfoque utilizado en el diseño de bases de datos que permite el almacenamiento y consulta de datos fuera de las estructuras tradicionales que se encuentran en las bases de datos relacionales, donde la decisión entre usar un tipo u otro dependerá del contexto y el caso de uso.

Permiten la administración de altos volúmenes de datos no estructurados que cambian con rapidez de formas diferentes a una base de datos relacional (SQL) con filas y tablas.

Para este tipo de bases de datos se suele usar MongoDB. Ideal para aquellos datos que no requieren una estructura fija [59], [60], [61].

- Bases de datos en el dispositivo

Estas bases de datos pueden entrar en cualquiera de los dos grupos anteriores, aunque se diferencian precisamente en la manera de almacenar los datos, siendo estos guardados directamente en el dispositivo del usuario, de manera que sean accesibles incluso aunque el usuario no tenga conexión, evitando así que la aplicación se quede cargando indefinidamente cuando el usuario no tenga conexión a internet [62].

Ejemplos de este tipo de bases de datos pueden ser SQLite[63] o Realm[64].

3.1.4 Sistema de recomendaciones

Para este tipo de motores se suele usar, como ya se ha mencionado anteriormente, Python, aunque también pueden usarse Java, SQL o R [65].

En la Ilustración 4 se puede ver una diferencia entre los programas clásicos y los programas de Machine Learning a la hora de qué datos deben introducirse, cómo son estos tratados y qué resultado podemos esperar.

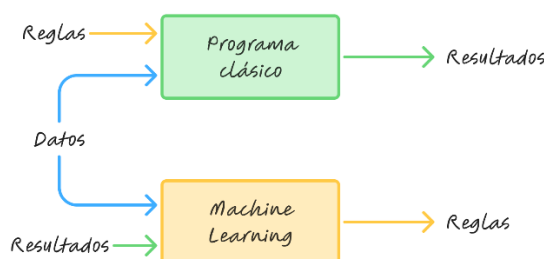


Ilustración 4. Diferencia entre programa clásico y Machine Learning

Fuente: elaboración propia adaptado de [65]

3.1.5 Qué configuración utilizar para mi proyecto

Tras barajar todas estas posibles opciones, la configuración elegida para este proyecto será la siguiente:

- *Frontend*

Para esta parte del desarrollo, se ha optado por la opción de usar React Native + Expo. Esto ha sido decidido por múltiples factores.

En primer lugar, la familiarización con el uso de JavaScript en otras asignaturas de la carrera, como SIBW (Sistemas de Información Basados en Web)[66], hacen que la curva de aprendizaje sea mucho menor a otras tecnologías abordadas en el punto anterior, lo que permitirá optimizar el tiempo de desarrollo del proyecto

Por otro lado, uno de los factores más importantes para elegir esta configuración ha sido la validación de la viabilidad y rendimiento de React Native en un proyecto de estas características, debido a su amplia presencia en el mundo laboral. Es, además, una forma de realizar un producto escalable que se podría seguir desarrollando finalizado este TFG sin ningún problema.

Finalmente, una de las claras desventajas que tiene el desarrollo de aplicaciones usando React respecto a Flutter, es no poder usar un solo código base que funcione para todas las

plataformas. Sin embargo, con el uso de Expo[67], obtenemos un resultado muy similar, pudiendo reciclar la gran mayoría de código en el paso de una plataforma a otra.

Es por esto por lo que se ha decidido usar este framework como tecnología para el desarrollo y la implementación del *frontend* de la aplicación.

- *Backend*

Para esta parte se ha decidido llevar a cabo la implementación de manera híbrida. Esto es debido a la decisión de la tecnología a utilizar para el *frontend* junto con la presencia de algoritmos de sistemas de recomendación e IA en este proyecto.

De esta forma, se ha decidido realizar la implementación del código del *backend* de este trabajo con Node.js, como complemento a la utilización de React Native en el *frontend*, sumado al uso de Python, mediante FastAPI, para la implementación de la parte de sistema de recomendaciones mediante un microservicio conectado con el *backend* de la aplicación.

- Base de datos

Debido a la idea del proyecto, donde se tienen datos con una estructura fija, se ha optado por una base de datos relacional implementada mediante PostgreSQL, una de las opciones más populares hoy en día en la industria del desarrollo de software (sino la más).

3.1.6 Valoración final sobre las tecnologías escogidas

Se han elegido un conjunto de tecnologías y configuraciones aplicables al estándar general del sector del desarrollo de software hoy en día, donde todas las tecnologías usadas son muy populares y las opciones escogidas por muchas grandes empresas, de manera que se pueda realizar un desarrollo e implementación del producto aplicables al mercado real.

3.2 Metodología de desarrollo

Para el desarrollo de este proyecto, se va a seguir una metodología de trabajo basada en Scrum, adaptada al desarrollo de un equipo de trabajo con un único integrante.

Scrum es una metodología ágil de trabajo ampliamente usada en el mundo del desarrollo para todo tipo de tareas, desde la creación de productos hasta la coordinación de proyectos, pasando por la gestión de solicitudes de TI. Esta metodología permite abordar proyectos complejos dividiendo el trabajo en ciclos iterativos más pequeños, los *sprints*, impulsando la transparencia y la mejora continua [68].

Un equipo de trabajo Scrum suele estar dividido en diferentes roles:

- Product Owner

Es el encargado de maximizar el valor del producto desarrollado y, por consecuencia, de maximizar el trabajo realizado por el equipo de desarrollo, aportando valor al negocio. Se encarga, además, de gestionar el *product backlog* y de representar a los *stakeholders* [68], [69], [70].

- Scrum Máster

Es el encargado de facilitar y promover la colaboración y comunicación dentro del equipo, garantizando que se sigan los procesos Scrum de manera eficiente. Se encarga de guiar al equipo en la práctica, tratando de respetar en todo momento los principios de la metodología y la realización de los eventos Scrum en los tiempos determinados [68], [70], [71].

- Equipo de desarrollo

Es la parte del equipo que se encarga de la implementación del producto en sí y del cumplimiento de los *sprints*. Se organiza de manera autónoma, de forma que no dependen ni del *Product Owner* ni del *Scrum Máster* para decirles cómo organizar los *sprints*. No obstante, deben trabajar codo con codo con el *Product Owner*, ya que, como se ha comentado, es el encargado de maximizar el trabajo de los desarrolladores y, por tanto, del producto final.

Un proyecto basado en la metodología Scrum está compuesto por una serie de eventos, como son el *sprint planning*, el *sprint*, la *daily meeting*, el *sprint review* y la retrospectiva, y una serie de artefactos, como son el *Product Backlog*, el *Sprint Backlog* y el incremento (o metas del *sprint*). Los eventos en Scrum son una serie de sucesos pensados para el equipo, aunque no todos los miembros acudirán a cada evento, mientras que los artefactos son información esencial que el equipo usa para definir y desarrollar el producto [68], [72].

- **Sprint Planning**

En este evento se resuelven dos cuestiones muy importantes para el desarrollo del *sprint*: **Qué** va a hacer el equipo y **Cómo** se van a desarrollar las tareas. En la primera se definirá el objetivo del *sprint*, mientras que en la segunda se definirá la manera de desarrollar los diferentes elementos que conformen el *sprint* [68], [73].

- **Sprint**

Un *sprint* es un evento realizado en un periodo de 2 a 3 semanas, donde se desarrollan las tareas escogidas en el *Sprint Planning*, con el objetivo de alcanzar la meta del *sprint*, definida previamente en el *Sprint Planning* [68], [74].

- **Daily Meeting**

Es una reunión diaria breve que se realizará al comienzo del día, donde cada miembro deberá explicar qué hizo el día anterior y qué tienen previsto para ese día. Se hace con el objetivo de mantener un control del desarrollo de las tareas y exponer los problemas encontrados, en caso de haberlos; no con el objetivo de resolverlos, sino más bien de hacerlos saber, para posteriormente resolverlos con los miembros correspondientes [68], [75].

- **Sprint Review**

Es una sesión formal realizada al finalizar el *sprint*, revisando las tareas que ahora están finalizadas y comentando lo realizado a lo largo de este [68].

- **Retrospectiva del Sprint**

Es una reunión donde el equipo se reunirá para comentar qué ha funcionado y qué no a lo largo del *sprint*, con el objetivo de centrarse en las cosas que han ido bien y en exponer qué se debería mejorar de cara al siguiente *sprint* [68].

- **Product Backlog**

Es la lista principal de tareas que se deben llevar a cabo para el correcto desarrollo del producto. Es una lista en constante cambio y evolución a lo largo del desarrollo y es la pieza principal para poder comenzar a trabajar y, por tanto, lo primero que se llevará a cabo antes de dar comienzo a la implementación de este proyecto.

Los elementos presentes en el *Product Backlog* son conocidos como Historias de Usuario y contarán con una prioridad asignada, de manera que se pueda maximizar el valor para el cliente; además de unos puntos de historia, con el objetivo de estimar el esfuerzo, complejidad y riesgo de una tarea [68], [76].

- **Sprint Backlog**

Es una lista seleccionada de historias de usuario para su desarrollo a lo largo del *sprint*. Estas historias son elegidas durante la planificación de dicho *sprint*.

- **Incremento**

Es la suma de los puntos de historia de todas las historias de usuario elegidas para ese *sprint*, de manera que se pueda, de forma cuantitativa, comprobar cuánto se ha mejorado el producto durante esa iteración [68], [77].

3.2.1 Adaptación de la metodología al proyecto

Para este proyecto, la metodología consistirá en una adaptación de la metodología Scrum al equipo conformado por la tutora de este Trabajo Fin de Grado, Carmen Martínez Cruz, y el alumno a desarrollarlo, Carlos Hoyo Liddle.

De esta manera, las labores de *Product Owner* y Equipo de Desarrollo recaerán sobre el alumno, Carlos Hoyo Liddle, mientras que la tutora será la encargada de guiarle en el proceso, cumpliendo con todas las pautas establecidas y las entregas, realizando así la labor de Scrum Máster adaptada al proyecto.

Se llevarán a cabo reuniones entre ambas partes cada cierto número de semanas, con el objetivo de valorar las tareas realizadas, a modo de *Sprint Review* y retrospectiva, así como de establecer las tareas a realizar durante las siguientes semanas hasta la próxima reunión, a modo de *Sprint Planning*. Se omiten eventos como las *daily meetings*, que quedarán sujetas a la labor del estudiante de organizarse para cumplir con los objetivos marcados a lo largo del *sprint*.

3.2.2 Lista de Historias de Usuario

A continuación, se muestra una explicación detallada de todas las Historias de Usuario que componen el funcionamiento de este sistema.

Identificador: HU-1	Login de usuarios	
Descripción: Como usuario, quiero poder iniciar sesión o registrarme en el sistema para poder acceder a mi cuenta y a las funcionalidades de la aplicación de forma segura.		
Estimación: 3	Prioridad: 1	Iteración: 1
Tareas relacionadas: • HU-2. Edición del perfil • HU-3. Recuperar contraseña • HU-4. Elección de rol • HU-5. Simultaneidad de roles • HU-6. Estado de búsqueda • HU-7. Estado de hospedaje • HU-22. Privacidad de la cuenta		
Pruebas de aceptación:		

• Inicio de sesión exitoso - Dado que el usuario está en la plantilla de login y tiene una cuenta previamente creada. - Cuando introduce su correo electrónico y su contraseña correcta y pulsa “entrar”. - Entonces el sistema le da acceso y lo redirige a la pantalla principal de la aplicación. • Contraseña incorrecta - Dado que el usuario intenta iniciar sesión. - Cuando introduce un correo registrado pero una contraseña incorrecta. - Entonces el sistema deniega el acceso. • Usuario no registrado - Dado que el usuario intenta iniciar sesión. - Cuando introduce un correo no registrado. - Entonces el sistema deniega el acceso. • Validación de formato de campos - Dado que el usuario intenta iniciar sesión. - Cuando introduce un texto en formato incorrecto. - Entonces el sistema no permite iniciar sesión, indicando que e formato de un campo es incorrecto. • Cierre de sesión - Dado que un usuario ha iniciado sesión. - Cuando pulsa el botón de “cerrar sesión”. - Entonces el sistema destruye su token de acceso y lo devuelve a la pantalla de login.
Observaciones:

Tabla 3. HU-1. Login de usuarios

Identificador: HU-2	Edición del perfil	
Descripción: Como usuario, quiero poder editar los datos de mi perfil para poder mostrar exactamente la imagen que quiero en todo momento.		
Estimación: 3	Prioridad: 1	Iteración: 1
Tareas relacionadas: • HU-1. Login de usuarios • HU-3. Recuperar contraseña • HU-4. Elección de rol • HU-5. Simultaneidad de roles • HU-6. Estado de búsqueda • HU-7. Estado de hospedaje • HU-22. Privacidad de la cuenta		
Pruebas de aceptación:		

- Actualización exitosa de los datos Dado que un usuario ha iniciado sesión y se encuentra en la pantalla de editar perfil. Cuando modifica los campos que quiere y pulsa el botón “Guardar cambios”. Entonces el sistema actualiza su información con los datos introducidos. - Campos obligatorios vacíos Dado que un usuario ha iniciado sesión y se encuentra en la pantalla de editar perfil. Cuando deja campos obligatorios sin rellenar. Entonces el sistema alerta de que debe rellenar todos los campos obligatorios antes de proceder con los cambios. - Foto de perfil con tamaño inválido Dado que un usuario ha iniciado sesión y se encuentra en la pantalla de editar perfil. Cuando modifica la foto de perfil de su cuenta, introduciendo una de un tamaño superior al permitido. Entonces el sistema alerta de que no se puede seleccionar esa foto, debido a que supera el tamaño permitido. - Subida de foto de perfil exitosa Dado que un usuario ha iniciado sesión y se encuentra en la pantalla de editar perfil. Cuando modifica su foto de perfil. Entonces el sistema actualiza su información con los datos introducidos. - Cancelación de la edición Dado que un usuario ha iniciado sesión y se encuentra en la pantalla de editar perfil. Cuando el usuario pulsa el botón “Salir” y después “Salir sin guardar”. Entonces el sistema desecha los datos introducidos y mantiene los que ya tenía.
Observaciones:

Tabla 4. HU-2. Edición del perfil

Identificador: HU-3	Recuperar contraseña	
Descripción: Como usuario, quiero poder recuperar y restablecer mi contraseña mediante mi correo electrónico en caso de haberla olvidado, para no perder el acceso a mi cuenta y a mis datos.		
Estimación: 3	Prioridad: 2	Iteración: 1
Tareas relacionadas: • HU-1. Login de usuarios • HU-2. Edición del perfil • HU-4. Elección de rol • HU-5. Simultaneidad de roles • HU-6. Estado de búsqueda • HU-7. Estado de hospedaje • HU-22. Privacidad de la cuenta		
Pruebas de aceptación:		

• Solicitud de restablecimiento exitosa - Dado que el usuario se encuentra en la pantalla de login y pulsa el botón “He olvidado mi contraseña”. - Cuando introduce un correo electrónico válido y pulsa “Enviar”. - Entonces el sistema envía un correo electrónico a la dirección facilitada. • Solicitud de restablecimiento no exitosa - Dado que el usuario se encuentra en la pantalla de login y pulsa el botón “He olvidado mi contraseña”. - Cuando introduce un correo electrónico no registrado y pulsa “Enviar”. - Entonces el sistema no envía un correo electrónico a la dirección facilitada. • Enlace de recuperación expirado - Dado que el usuario recibe el correo de recuperación. - Cuando tarda más del tiempo máximo en usarlo. - Entonces el sistema le notifica de que el enlace ha expirado. • Restablecimiento de contraseña exitoso - Dado que el usuario ha recibido un correo de recuperación de contraseña. - Cuando introduce una nueva contraseña válida. - Entonces el sistema actualiza la información de la cuenta con la nueva contraseña.
Observaciones:

Tabla 5. HU-3. Recuperar contraseña

Identificador: HU-4	Elección de rol	
Descripción: Como usuario, quiero poder elegir el rol de mi cuenta para poder hacer uso de la aplicación y sacarle le mayor beneficio.		
Estimación: 3	Prioridad: 1	Iteración: 1
Tareas relacionadas: • HU-1. Login de usuarios • HU-2. Edición del perfil • HU-3. Recuperar contraseña • HU-5. Simultaneidad de roles • HU-6. Estado de búsqueda • HU-7. Estado de hospedaje • HU-22. Privacidad de la cuenta		
Pruebas de aceptación: • Elección del rol - Dado que el usuario ha iniciado sesión y se encuentra en la pantalla de elección de rol. - Cuando selecciona el rol concreto que quiere tener asociado. - Entonces el sistema actualiza la información de la cuenta con la nueva elección.		
Observaciones:		

Tabla 6. HU-4. Elección de rol

Identificador: HU-5	Simultaneidad de roles	
Descripción: Como usuario, quiero tener la posibilidad de tener asociados tanto el rol de casero como el de inquilino para poder tanto alquilar mi vivienda como ser entrar en la vivienda de alguien como inquilino.		
Estimación: 3	Prioridad: 2	Iteración: 1
Tareas relacionadas: • HU-1. Login de usuarios • HU-2. Edición del perfil • HU-3. Recuperar contraseña • HU-4. Elección de rol • HU-6. Estado de búsqueda • HU-7. Estado de hospedaje • HU-22. Privacidad de la cuenta		
Pruebas de aceptación: • Elección de varios roles simultáneos - Dado que el usuario ha iniciado sesión y se encuentra en la pantalla de elección de roles. - Cuando selecciona que quiere tener ambos roles asociados a su cuenta. - Entonces el sistema actualiza la información de la cuenta con la elección.		
Observaciones:		

Tabla 7. HU-5. Simultaneidad de roles

Identificador: HU-6	Estado de búsqueda	
Descripción: Como usuario, quiero poder declarar si me encuentro en búsqueda de inquilinos, compañeros o vivienda para así mostrarme disponible a posibles usuarios interesados.		
Estimación: 3	Prioridad: 1	Iteración: 1
Tareas relacionadas: • HU-1. Login de usuarios • HU-2. Edición del perfil • HU-3. Recuperar contraseña • HU-4. Elección de rol • HU-5. Simultaneidad de roles • HU-7. Estado de hospedaje • HU-22. Privacidad de la cuenta		
Pruebas de aceptación: • Declaración de estado de búsqueda - Dado que el usuario ha iniciado sesión y, desde la pantalla de información del perfil, pulsa el botón “Establecer estado de búsqueda” o “Modificar estado de búsqueda”. - Cuando establece su estado de búsqueda a uno de los mostrados por la aplicación como disponibles. - Entonces el sistema actualiza la información de la cuenta con el nuevo estado de búsqueda.		
Observaciones:		

Tabla 8. HU-6. Estado de búsqueda

Identificador: HU-7	Estado de hospedaje	
Descripción: Como usuario con rol inquilino, quiero poder establecer mi estado de hospedaje para así poder ofrecer más información a posibles usuarios interesados.		
Estimación: 3	Prioridad: 1	Iteración: 1
Tareas relacionadas: • HU-1. Login de usuarios • HU-2. Edición del perfil • HU-3. Recuperar contraseña • HU-4. Elección de rol • HU-5. Simultaneidad de roles • HU-6. Estado de búsqueda • HU-22. Privacidad de la cuenta		
Pruebas de aceptación: • Declaración de estado de hospedaje - Dado que el usuario ha iniciado sesión y, desde la pantalla de información del perfil, pulsa el botón “Establecer estado de hospedaje” o “Modificar estado de hospedaje”. - Cuando establece su estado de hospedaje a uno de los mostrados por la aplicación como disponibles. - Entonces el sistema actualiza la información de la cuenta con el nuevo estado de hospedaje.		
Observaciones:		

Tabla 9. HU-7. Estado de hospedaje

Identificador: HU-8	Publicación de anuncios	
Descripción: Como casero, quiero poder publicar anuncios para mostrar mi vivienda a posibles usuarios interesados.		
Estimación: 3	Prioridad: 1	Iteración: 2
Tareas relacionadas: • HU-9. Edición y eliminación de anuncios		
Pruebas de aceptación: • Publicación de anuncio exitosa - Dado que el usuario con rol de casero ha iniciado sesión y, desde la pantalla de publicación de anuncios ha rellenado todos los campos solicitados en el proceso de publicación. - Cuando selecciona la opción “Publicar piso”. - Entonces el anuncio es valorado y aceptado, y pasa a ser publicado en el sistema. • Publicación de anuncio no exitosa - Dado que el usuario de rol casero ha iniciado sesión y, desde la pantalla de publicación de anuncios ha rellenado todos (o no) los campos solicitados en el proceso de publicación. - Cuando selecciona la opción “Publicar anuncio”. - Entonces el anuncio es valorado y denegado, proporcionando una explicación detallada al usuario de por qué su anuncio no cumple con las normas impuestas por el sistema.		

Observaciones:

Tabla 10. HU-8. Publicación de anuncios

Identificador: HU-9	Edición y eliminación de anuncios	
Descripción: Como casero, quiero poder editar y eliminar mis anuncios de vivienda para mostrar exactamente lo que quiero al resto de usuarios de la aplicación.		
Estimación: 3	Prioridad: 2	Iteración: 2
Tareas relacionadas: HU-8. Publicación de anuncios		
Pruebas de aceptación: - Edición de anuncio exitosa Dado que el usuario con rol de casero ha iniciado sesión y, desde la pantalla de edición de anuncios ha modificado y rellenado correctamente todos los campos solicitados en el proceso de publicación. Cuando selecciona la opción “Publicar anuncio”. Entonces el anuncio es valorado y aceptado, y pasa a ser actualizado en el sistema con la nueva información. - Edición de anuncio no exitosa Dado que el usuario con rol de casero ha iniciado sesión y, desde la pantalla de edición de anuncios ha modificado y rellenado todos (o no) los campos solicitados en el proceso de publicación. Cuando selecciona la opción “Publicar piso”. Entonces el anuncio es valorado y denegado, manteniendo la versión sin modificar en el sistema y proporcionando al usuario una explicación detallada de por qué el anuncio no cumplía con las normas impuestas por el sistema. - Eliminación de anuncio Dado que el usuario con rol de casero ha iniciado sesión y se encuentra en la pantalla de edición de anuncios. Cuando selecciona la opción “Eliminar anuncio” y confirma la selección. Entonces el anuncio es eliminado del sistema.		
Observaciones:		

Tabla 11. HU-9. Edición y eliminación de anuncios

Identificador: HU-10	Envío de solicitudes de registro en viviendas	
Descripción: Como usuario con rol de casero, quiero poder enviar solicitudes de registro en la vivienda a usuarios para que puedan entrar a vivir en mi vivienda.		
Estimación: 3	Prioridad: 2	Iteración: 3
Tareas relacionadas: • HU-11. Recepción de solicitudes de registro en viviendas • HU-12. Registro de inquilinos en viviendas • HU-19. Chat • HU-20. Conexión entre usuarios		
Pruebas de aceptación:		

- Envío de solicitud de registro exitosa Dado que el usuario con rol de casero ha iniciado sesión y ha encontrado un usuario con rol de inquilino interesado en su piso. Cuando selecciona la opción “Enviar solicitud de registro en vivienda”. Entonces el usuario con rol de inquilino acepta la solicitud y es registrado en el sistema como inquilino de la vivienda. - Envío de solicitud de registro no exitosa Dado que el usuario con rol de casero ha iniciado sesión y ha encontrado un usuario con rol de inquilino interesado en su piso. Cuando selecciona la opción “Enviar solicitud de registro en vivienda”. Entonces el usuario con rol de inquilino rechaza la solicitud y no es registrado en el sistema como inquilino de la vivienda.
Observaciones:

Tabla 12. HU-10. Envío de solicitudes de registro en viviendas

Identificador: HU-11	Recepción de solicitudes de registro en vivienda	
Descripción: Como usuario con rol de inquilino, quiero poder recibir, aceptar y rechazar solicitudes de registro en viviendas por parte de usuarios con rol de casero.		
Estimación: 3	Prioridad: 2	Iteración: 3
Tareas relacionadas: • HU-10. Envío de solicitudes de registro en viviendas • HU-12. Registro de inquilinos en viviendas • HU-19. Chat • HU-20. Conexión entre usuarios		
Pruebas de aceptación: • Recepción de solicitud de registro - Dado que el usuario con rol de inquilino tiene cuenta en la aplicación. - Cuando un usuario con rol de casero envía una solicitud de registro en una vivienda. - Entonces el usuario con rol de inquilino es notificado de que se ha realizado dicha solicitud. • Aceptación de solicitud de registro - Dado que el usuario con rol de inquilino tiene cuenta en la aplicación. - Cuando recibe una notificación de una solicitud de registro en una vivienda por parte de un usuario con rol de casero. - Entonces el usuario con rol de inquilino acepta la solicitud y es registrado en el sistema como inquilino de la vivienda. • Rechazo de solicitud de registro - Dado que el usuario con rol de inquilino tiene cuenta en la aplicación. - Cuando recibe una notificación de una solicitud de registro en una vivienda por parte de un usuario con rol de casero. - Entonces el usuario con rol de inquilino rechaza la solicitud y no es registrado en el sistema como inquilino de la vivienda.		
Observaciones:		

Tabla 13. HU-11. Recepción de solicitudes de registro en viviendas

Identificador: HU-12	Registro de inquilinos en viviendas	
Descripción: Como usuario, quiero poder registrar o ser registrado en una vivienda cuando se dé el caso, para así poder actualizar el estado e información de mi cuenta o vivienda.		
Estimación: 3	Prioridad: 1	Iteración: 3
Tareas relacionadas: - HU-10. Envío de solicitudes de registro en viviendas - HU-11. Recepción de solicitudes de registro en viviendas - HU-19. Chat - HU-20. Conexión entre usuarios		
Pruebas de aceptación: - Registro de inquilino en vivienda exitoso Dado que el usuario con rol de inquilino ha recibido y aceptado una solicitud de registro en una vivienda. Cuando se completan correctamente todos los datos necesarios para el registro. Entonces el usuario con rol de inquilino es registrado en el sistema como inquilino de la vivienda. - Registro de inquilino en vivienda no exitoso Dado que el usuario con rol de inquilino ha recibido y aceptado una solicitud de registro en una vivienda. Cuando se completan incorrectamente todos (o no) los datos necesarios para el registro. Entonces el usuario con rol de inquilino no es registrado en el sistema como inquilino de la vivienda y se le proporciona una explicación de los datos incorrectos, dándole la opción de volver a rellenarlos.		
Observaciones:		

Tabla 14. HU-12. Registro de inquilinos en viviendas

Identificador: HU-13	Control de anuncios	
Descripción: Como administrador, quiero poder revisar y aceptar o rechazar los anuncios que se intentan publicar en la aplicación para así garantizar el cumplimiento de las normas.		
Estimación: 3	Prioridad: 2	Iteración: 6
Tareas relacionadas: • HU-25. Eliminación de anuncios por administrador • HU-26. Control de valoraciones • HU-27. Baneo y suspensión de cuentas		
Pruebas de aceptación:		

• Aceptación de anuncio - Dado que un usuario con rol de casero ha completado el proceso de publicación de anuncio. - Cuando el administrador considera que el anuncio cumple las normas establecidas en la aplicación. - Entonces el administrador acepta la publicación del anuncio y este es entonces publicado. • Denegación de anuncio - Dado que un usuario con rol de casero ha completado el proceso de publicación de anuncio. - Cuando el administrador considera que el anuncio incumple las normas establecidas en la aplicación. - Entonces el administrador deniega la publicación del anuncio y este es entonces ignorado por el sistema, que mandará una explicación detallada al usuario que ha intentado publicar el anuncio, explicándole por qué ha sido rechazado.
Observaciones:

Tabla 15. HU-13. Control de anuncios

Identificador: HU-14	Valoraciones Inquilino-Inquilino	
Descripción: Como usuario con rol de inquilinos, quiero poder valorar a otros inquilinos con los que haya convivido para poder dar más información al resto de usuarios de la aplicación.		
Estimación: 3	Prioridad: 1	Iteración: 4
Tareas relacionadas: • HU-15. Valoraciones Inquilino-Casero • HU-16. Valoraciones Inquilino- Vivienda • HU-17. Valoraciones Casero-Inquilino • HU-23. Verificación de tiempo de convivencia en valoraciones		
Pruebas de aceptación: • Valoración exitosa - Dado que dos usuarios con rol de inquilinos han mantenido una relación de convivencia. - Cuando el sistema envía una notificación de “Valoración a compañero disponible”, alguno de los inquilinos, los dos, acceden a realizar la valoración y rellenan todos los campos correctamente. - Entonces el sistema almacena esa valoración, que queda disponible para el resto de los usuarios de la aplicación. • Valoración no exitosa - Dado que dos usuarios con rol de inquilinos han mantenido una relación de convivencia. - Cuando el sistema envía una notificación de “Valoración a compañero disponible”, alguno de los inquilinos, o los dos, acceden a realizar la valoración y rellenan todos (o no) los campos. - Entonces el sistema detecta alguna violación de las normas de valoraciones establecidas en la aplicación (como lenguaje ofensivo) y rechaza la valoración, notificando al usuario por qué se ha rechazado y permitiéndole volver a rellenar el campo conflictivo.		
Observaciones:		

Tabla 16. HU-14. Valoraciones Inquilino-Inquilino

Identificador: HU-15	Valoraciones Inquilino-Casero	
Descripción: Como usuario inquilino, quiero poder valorar a los usuarios que hayan sido mis caseros para así dar más información al resto de usuarios de la aplicación.		
Estimación: 3	Prioridad: 1	Iteración: 4
Tareas relacionadas: • HU-14. Valoraciones Inquilino-Inquilino • HU-16. Valoraciones Inquilino- Vivienda • HU-17. Valoraciones Casero-Inquilino • HU-23. Verificación de tiempo de convivencia en valoraciones		
Pruebas de aceptación: • Valoración exitosa - Dado que dos usuarios con roles de inquilino y casero han mantenido una relación de convivencia. - Cuando el sistema envía una notificación de “Valoración a casero disponible” al usuario inquilino, que accede a realizar la valoración y rellena todos los campos correctamente. - Entonces el sistema almacena esa valoración, que queda disponible para el resto de los usuarios de la aplicación. • Valoración no exitosa - Dado que dos usuarios con roles de inquilino y casero han mantenido una relación de convivencia. - Cuando el sistema envía una notificación de “Valoración a casero disponible” al usuario inquilino, que accede a realizar la valoración y rellena todos (o no) los campos. - Entonces el sistema detecta alguna violación de las normas de valoraciones establecidas en la aplicación (como lenguaje ofensivo) y rechaza la valoración, notificando al usuario por qué se ha rechazado y permitiéndole volver a rellenar el campo conflictivo.		
Observaciones:		

Tabla 17. HU-15. Valoraciones Inquilino-Casero

Identificador: HU-16	Valoraciones Inquilino-Vivienda	
Descripción: Como usuario con rol de inquilino, quiero poder valorar las viviendas en las que me he hospedado para así poder dar más información al resto de usuarios de la aplicación.		
Estimación: 3	Prioridad: 1	Iteración: 4
Tareas relacionadas: • HU-14. Valoraciones Inquilino-Inquilino • HU-15. Valoraciones Inquilino-Casero • HU-17. Valoraciones Casero-Inquilino • HU-23. Verificación de tiempo de convivencia en valoraciones		
Pruebas de aceptación:		

• Valoración exitosa - Dado que un usuario con rol de inquilino se ha alojado en una vivienda. - Cuando el sistema envía una notificación de “Valoración a vivienda disponible”, el usuario accede a realizar la valoración y rellena todos los campos correctamente. - Entonces el sistema almacena esa valoración, que queda disponible para el resto de los usuarios de la aplicación. • Valoración no exitosa - Dado que un usuario con rol de inquilino se ha alojado en una vivienda. - Cuando el sistema envía una notificación de “Valoración a vivienda disponible”, el usuario accede a realizar la valoración y rellena todos (o no) los campos. - Entonces el sistema detecta alguna violación de las normas de valoraciones establecidas en la aplicación (como lenguaje ofensivo) y rechaza la valoración, notificando al usuario por qué se ha rechazado y permitiéndole volver a rellenar el campo conflictivo.
Observaciones:

Tabla 18. HU-16. Valoraciones Inquilino- Vivienda

Identificador: HU-17	Valoraciones Casero-Inquilino	
Descripción: Como usuario con rol de casero, quiero poder realizar valoraciones de aquellos inquilinos que se hayan alojado en alguna de mis viviendas para poder dar más información al resto de los usuarios de la aplicación.		
Estimación: 3	Prioridad: 1	Iteración: 4
Tareas relacionadas: • HU-14. Valoraciones Inquilino-Inquilino • HU-15. Valoraciones Inquilino-Casero • HU-16. Valoraciones Inquilino- Vivienda • HU-23. Verificación de tiempo de convivencia en valoraciones		
Pruebas de aceptación:		

• Valoración exitosa - Dado que dos usuarios con roles de inquilino y casero han mantenido una relación de convivencia. - Cuando el sistema envía una notificación de “Valoración a inquilino disponible” al usuario casero, que accede a realizar la valoración y rellena todos los campos correctamente. - Entonces el sistema almacena esa valoración, que queda disponible para el resto de los usuarios de la aplicación. • Valoración no exitosa - Dado que dos usuarios con roles de inquilino y casero han mantenido una relación de convivencia. - Cuando el sistema envía una notificación de “Valoración a inquilino disponible” al usuario casero, que accede a realizar la valoración y rellena todos (o no) los campos. - Entonces el sistema detecta alguna violación de las normas de valoraciones establecidas en la aplicación (como lenguaje ofensivo) y rechaza la valoración, notificando al usuario por qué se ha rechazado y permitiéndole volver a rellenar el campo conflictivo.
Observaciones:

Tabla 19. HU-17. Valoraciones Casero-Inquilino

Identificador: HU-18	Sistema de búsqueda	
Descripción: Como usuario, quiero poder buscar tanto viviendas como a otros usuarios para poder encontrar aquellos que más se adecúen a lo que busco.		
Estimación: 3	Prioridad: 2	Iteración: 5
Tareas relacionadas: • HU-24. Sistema de recomendaciones		
Pruebas de aceptación: • Búsqueda general - Dado que un usuario ha iniciado sesión en la aplicación. - Cuando pulsa en la barra de búsqueda en la parte superior de la pantalla e introduce una cadena de texto. - Entonces el sistema muestra todas las viviendas y usuarios relacionadas con la cadena introducida. • Búsqueda concreta - Dado que un usuario ha iniciado sesión en la aplicación. - Cuando pulsa en la barra de búsqueda en la parte superior de la pantalla, introduce una cadena de texto y pulsa el botón de “Usuarios” o “Viviendas”. - Entonces el sistema muestra todas las viviendas o usuarios, según cuál haya seleccionado, relacionadas con la cadena introducida. • Búsqueda no exitosa - Dado que un usuario ha iniciado sesión en la aplicación. - Cuando pulsa en la barra de búsqueda en la parte superior de la pantalla e introduce una cadena de texto (pulsando o no alguno de los botones de “Usuarios” o “Viviendas”). - Entonces el sistema no encuentra nada relacionado con la cadena introducida y, por tanto, muestra el mensaje “No se han encontrado resultados”.		

Observaciones:

Tabla 20. HU-18. Sistema de búsqueda

Identificador: HU-19	Chat	
Descripción: Como usuario, quiero poder hablar con otros usuarios mediante un chat dentro de la aplicación para así poder comunicarme de forma sencilla con ellos.		
Estimación: 3	Prioridad: 2	Iteración: 3
Tareas relacionadas: • HU-10. Envío de solicitudes de registro en viviendas • HU-11. Recepción de solicitudes de registro en viviendas • HU-12. Registro de inquilinos en viviendas • HU-20. Conexión entre usuarios		
Pruebas de aceptación: • Acceso a chats - Dado que un usuario ha iniciado sesión en la aplicación. - Cuando pulsa en el icono de mensajes de la barra de navegación de la aplicación. - Entonces el sistema muestra la pantalla con todos los chats. • Acceso a una conversación - Dado que un usuario ha iniciado sesión en la aplicación y se encuentra en la pantalla de chats o en el perfil de un usuario de la aplicación. - Cuando pulsa en la conversación, en caso de estar en la pantalla de chats, o en el botón de enviar mensaje, en caso de estar en el perfil de un usuario. - Entonces el sistema muestra la pantalla de la conversación con el usuario seleccionado. • Envío de mensaje - Dado que un usuario se encuentra en la conversación con otro usuario. - Cuando escribe un mensaje y pulsa en el icono de enviar. - Entonces el mensaje se envía al otro usuario, que es notificado de dicho mensaje. • Edición de mensaje - Dado que un usuario se encuentra en la conversación con otro usuario y ha mandado un mensaje hace menos de 5 minutos. - Cuando mantiene el mensaje pulsado (en móvil) o presiona click derecho (en ordenador) y pulsa en “Editar mensaje” e introduce el contenido que desee. - Entonces el mensaje es modificado con el nuevo texto escrito. • Eliminación de mensaje - Dado que un usuario se encuentra en la conversación con otro usuario y ha mandado un mensaje hace menos de 5 minutos. - Cuando mantiene el mensaje pulsado (en móvil) o presiona click derecho (en ordenador) y pulsa en “Eliminar mensaje” y, posteriormente en “Confirmar”. - Entonces el mensaje es eliminado de la conversación para ambos usuarios.		
Observaciones:		

Tabla 21. HU-19. Chat

Identificador: HU-20	Conexión entre usuarios	
Descripción: Como usuario, quiero poder establecer una conexión con otros usuarios con los que haya convivido para que el resto de los usuarios comunes puedan ver que hemos convivido.		
Estimación: 3	Prioridad: 3	Iteración: 3
Tareas relacionadas: • HU-10. Envío de solicitudes de registro en viviendas • HU-11. Recepción de solicitudes de registro en viviendas • HU-12. Registro de inquilinos en viviendas • HU-19. Chat		
Pruebas de aceptación: • Conexión - Dado que dos usuarios han mantenido o mantienen una relación de convivencia. - Cuando esta comienza a tener efecto. - Entonces el sistema establece una conexión entre ambos usuarios, visible para aquellos usuarios en común de uno o ambos usuarios al entrar en el perfil de alguno de los dos.		
Observaciones:		

Tabla 22. HU-20. Conexión entre usuarios

Identificador: HU-21	Multiplataforma	
Descripción: Como usuario, quiero poder disfrutar de la aplicación en todos mis dispositivos para así poder hacer uso de ella en todo momento.		
Estimación: 3	Prioridad: 2	Iteración:
Tareas relacionadas:		
Pruebas de aceptación: • Multiplataforma - Dado que un usuario tiene una cuenta activa en la aplicación. - Cuando trata de entrar a ella desde su dispositivo móvil u ordenador. - Entonces el sistema muestra la versión correspondiente al dispositivo utilizado para entrar.		
Observaciones: no se han vinculado otras historias de usuario debido a que esta historia surge directamente de la elección de tecnologías para el <i>frontend</i> (React Native + Expo) y no está directamente asociado a otra historia de usuario. En esta primera versión del proyecto, se satisface la versión web mediante la reutilización de la aplicación móvil generada automáticamente por Expo, aunque una correcta implementación web dedicada queda planteada como trabajo futuro.		

Tabla 23. HU-21. Multiplataforma

Identificador: HU-22	Privacidad de la cuenta	
Descripción: Como usuario, quiero que la información de mi perfil solo sea visible para el resto de los usuarios cuando yo así lo indique, para poder así disfrutar de mi privacidad.		
Estimación: 3	Prioridad: 3	Iteración: 1
Tareas relacionadas:		

• HU-1. Login de usuarios • HU-2. Edición del perfil • HU-3. Recuperar contraseña • HU-4. Elección de rol • HU-5. Simultaneidad de roles • HU-6. Estado de búsqueda • HU-7. Estado de hospedaje
Pruebas de aceptación: • Cuenta pública - Dado que un usuario ha iniciado sesión en la aplicación. - Cuando selecciona, bien a través del mensaje al entrar por primera vez a la aplicación desde esa cuenta, o bien desde el panel de edición del perfil, que quiere que su cuenta sea pública. - Entonces el sistema hace visible la información (únicamente aquella visualizable, lo que excluye información sensible o realmente privada, como pueden ser el correo y la contraseña) del perfil a los usuarios de la aplicación. • Cuenta privada - Dado que un usuario ha iniciado sesión en la aplicación. - Cuando selecciona, bien a través del mensaje al entrar por primera vez a la aplicación desde esa cuenta, o bien desde el panel de edición del perfil, que quiere que su cuenta sea privada. - Entonces el sistema hace visible únicamente la información relacionada con el nombre de usuario y de la persona, las cuentas comunes que tengan y las valoraciones recibidas.
Observaciones:

Tabla 24. HU-22. Privacidad de la cuenta

Identificador: HU-23	Verificación de tiempo de convivencia en valoraciones	
Descripción: Como usuario, quiero que el sistema tenga en cuenta el tiempo de convivencia con otro usuario en la valoración que ponga de mi perfil para evitar que me intenten hundir la reputación.		
Estimación: 3	Prioridad: 3	Iteración: 3
Tareas relacionadas: • HU-14. Valoraciones Inquilino-Inquilino • HU-15. Valoraciones Inquilino-Casero • HU-16. Valoraciones Inquilino- Vivienda • HU-17. Valoraciones Casero-Inquilino • HU-26. Control de valoraciones		
Pruebas de aceptación:		

• Aviso de tiempo de convivencia - Dado que dos usuarios han convivido menos de 4 meses y se han valorado. - Cuando otro usuario accede a la pestaña de valoraciones del perfil de otro usuario. - Entonces el sistema mostrará una alerta al lado de la valoración, indicando que la convivencia que ha dado lugar a esa valoración ha sido breve. • Revisión de valoración por tiempo de convivencia - Dado que un usuario ha sido puntuado por otro con el que ha convivido durante un periodo inferior a 4 meses. - Cuando el usuario valorado considere que esa valoración no es fiel a la realidad. - Entonces el sistema mostrará un mensaje de alerta indicando que el usuario valorado apela que esa valoración es falsa o no es del todo cierta.
Observaciones:

Tabla 25. HU-23. Verificación de tiempo de convivencia en valoraciones

Identificador: HU-24	Sistema de recomendaciones	
Descripción: Como usuario, quiero que el sistema me recomiende usuarios o viviendas que me podrían interesar en función de mi estado de búsqueda y hospedaje y de mis preferencias, interacciones y usuarios y viviendas valoradas.		
Estimación: 3	Prioridad: 1	Iteración: 5
Tareas relacionadas: • HU-18. Sistema de búsqueda		
Pruebas de aceptación: • Recomendación en función de información del perfil - Dado que un usuario haya realizado acciones en la aplicación como buscar, interactuar, establecer unas preferencias de búsqueda, establecer un estado de búsqueda y hospedaje o valorado usuarios o viviendas. - Cuando el usuario entre en la aplicación. - Entonces el sistema mostrará sugerencias de perfiles y/o viviendas en función del estado de la cuenta y preferencias del usuario.		
Observaciones:		

Tabla 26. HU-24. Sistema de recomendaciones

Identificador: HU-25	Eliminación de anuncios por administrador	
Descripción: Como administrador del sistema, quiero poder eliminar anuncios de viviendas que incumplan las normas para poder mantener un correcto funcionamiento y fiabilidad de la aplicación.		
Estimación: 3	Prioridad: 2	Iteración: 6
Tareas relacionadas: • HU-13. Control de anuncios • HU-26. Control de valoraciones • HU-27. Baneo y suspensión de cuentas		
Pruebas de aceptación:		

• Incumplimiento de las normas en anuncio - Dado que un usuario con rol de casero haya publicado un anuncio que incumpla las normas de la aplicación. - Cuando el administrador vea ese mensaje. - Entonces el administrador podrá eliminar el mensaje, indicando siempre las causas de la eliminación de este, que serán notificadas al usuario propietario del anuncio.
Observaciones:

Tabla 27. HU-25. Eliminación de anuncios por administrador

Identificador: HU-26	Control de valoraciones	
Descripción: Como administrador del sistema, quiero poder realizar un control y eliminación de las valoraciones pertinentes para garantizar el cumplimiento de las normas de la aplicación.		
Estimación: 3	Prioridad: 2	Iteración: 6
Tareas relacionadas: • HU-13. Control de anuncios • HU-14. Valoraciones Inquilino-Inquilino • HU-15. Valoraciones Inquilino-Casero • HU-16. Valoraciones Inquilino- Vivienda • HU-17. Valoraciones Casero-Inquilino • HU-25. Eliminación de anuncios por administrador • HU-27. Baneo y suspensión de cuentas		
Pruebas de aceptación: • Incumplimiento de las normas en anuncio - Dado que un usuario haya realizado una valoración que incumpla las normas de la aplicación. - Cuando el administrador vea esa valoración. - Entonces el administrador podrá eliminar la valoración, indicando siempre las causas de la eliminación de este, que serán notificadas al usuario propietario del anuncio.		
Observaciones:		

Tabla 28. HU-26. Control de valoraciones

Identificador: HU-27	Baneo y suspensión de cuentas	
Descripción: Como administrador del sistema, quiero poder banear y suspender cuentas de usuarios que incumplan las normas de la aplicación en proporción a la gravedad del incumplimiento, para así poder mantener el orden en la aplicación.		
Estimación: 3	Prioridad: 2	Iteración: 6
Tareas relacionadas: • HU-13. Control de anuncios • HU-25. Eliminación de anuncios por administrador • HU-26. Control de valoraciones		
Pruebas de aceptación:		

• Suspensión temporal de cuenta de usuario - Dado que un usuario haya incumplido ciertas normas de la aplicación. - Cuando el administrador sea notificado. - Entonces el administrador podrá suspender temporalmente la cuenta del usuario en cuestión en proporción a la violación de las normas que haya cometido. • Suspensión permanente (baneo) de cuenta de usuario - Dado que un usuario haya gravemente incumplido normas de la aplicación o se hayan producido reiterados incumplimientos de estas. - Cuando el administrador sea notificado. - Entonces el administrador podrá banear permanentemente el sistema al usuario en cuestión, que perderá su cuenta y todo lo asociado a ella.
Observaciones:

Tabla 29. HU-27. Baneo y suspensión de cuentas

3.2.3 Product Backlog del proyecto

Para el Product Backlog y la planificación y ejecución de los *sprints* se va a utilizar la herramienta Jira [78], de manera que se pueda realizar todo el proceso de forma ordenada y estructurada. No obstante, para su representación en esta memoria, se va a hacer uso de la Tabla 30, de manera que su visualización sea más clara y sencilla.

Para el cálculo de la estimación de cada Historia de Usuario, se han tomado como referencia los números de la sucesión de Fibonacci [79], de forma que se asigna la dificultad en relación a la siguiente matriz [80]:

1 = Tareas que no requieren apenas esfuerzo.

2 = Tareas que consisten en añadir una funcionalidad concreta o detalle breve a algún componente.

3 = Tareas que requieren crear alguna funcionalidad necesaria de la aplicación, aunque no conllevan una excesiva complejidad.

5 = Tareas que requieren añadir o crear funcionalidades complejas, pero pueden compaginarse con otras al mismo tiempo.

8 = Tareas que requieren crear funcionalidades muy complejas que son difícilmente compaginables con otras.

13 = Tareas que requieren crear funcionalidades especialmente complejas, con trabajo de investigación y documentación previo y cuyo *sprint* irá prácticamente dedicado de manera íntegra a su realización.

Respecto a la prioridad, se ha seguido la técnica MoSCoW, donde las tareas se priorizan en función de las categorías **MUST** have (debe tenerlo, en mi caso, valor 1), **SHOULD** have (debería tenerlo, en mi caso, valor 2), **COULD** have (podría tenerlo, en mi caso, valor 3) y **WON'T** have (no lo tendrá, en mi caso particular no se ha utilizado) [81].

Ident.	Título	Estim.	Prio.
HU-1	Login de usuarios	3	1
HU-2	Edición del perfil	3	1
HU-3	Recuperar contraseña	2	2
HU-4	Elección de rol	2	1
HU-5	Simultaneidad de roles	1	2
HU-6	Estado de búsqueda	2	1
HU-7	Estado de hospedaje	2	1
HU-8	Publicación de anuncios	5	1
HU-9	Edición y eliminación de anuncios	3	2
HU-10	Envío de solicitud de registro de vivienda	3	2
HU-11	Recepción de solicitudes de registro de vivienda	3	2
HU-12	Registro de inquilino en vivienda	2	1
HU-13	Control de anuncios	3	2
HU-14	Valoraciones Inquilino-Inquilino	5	1
HU-15	Valoraciones Inquilino-Casero	5	1
HU-16	Valoraciones Inquilino-Vivienda	5	1
HU-17	Valoraciones Casero-Inquilino	5	1
HU-18	Sistema de Búsqueda	8	2
HU-19	Chat	8	2
HU-20	Conexión entre usuarios	3	3
HU-21	Multiplataforma	5	2
HU-22	Privacidad de la cuenta	3	3
HU-23	Verificación de tiempo de convivencia en valoraciones	2	3
HU-24	Sistema de Recomendaciones	13	1
HU-25	Eliminación de anuncios por administrador	3	2
HU-26	Control de valoraciones	3	2
HU-27	Baneo y suspensión de cuentas	3	2

Tabla 30. Product Backlog

3.3 Planificación temporal

Para la planificación temporal del proyecto se ha realizado una agrupación por fases concretas de las Historias de Usuario del *Backlog* y las diferentes tareas definidas, en relación con la finalidad de cada una. También se han organizado los distintos *sprints* en los que se dividirá el desarrollo del proyecto, siendo estos plasmados en un Diagrama de Gantt, facilitando la visualización del proceso con sus respectivos tiempos asociados.

3.3.1 Planificación de *Sprints*

Para la planificación de los *sprints*, como ya se ha comentado en el punto 3.2.3, se ha utilizado la herramienta Jira. No obstante, para su mejor representación y visualización en este documento, se ha representado mediante tablas, de la Tabla 31 a la Tabla 36.

Para este proyecto, los *sprints* se han planificado con una duración de 1 a 3 semanas, dependiendo de la complejidad de las tareas a realizar, siendo la mayoría de los *sprints* de 2 semanas.

Sprint 1. Creación y gestión de usuarios		Inicio: 19/03/2026 Fin: 01/04/2026
Ident.	Título	Estim.
HU-1	Login de usuarios	3
HU-2	Edición del perfil	3
HU-3	Recuperar contraseña	2
HU-4	Elección de rol	2
HU-5	Simultaneidad de roles	1
HU-6	Estado de búsqueda	2
HU-7	Estado de hospedaje	2
HU-22	Privacidad de la cuenta	3

Tabla 31. Sprint 1

Sprint 2. Publicación y gestión de anuncios		Inicio: 02/04/2026 Fin: 15/04/2026
Ident.	Título	Estim.
HU-8	Publicación de anuncios	5
HU-9	Edición y eliminación de anuncios	3

Tabla 32. Sprint 2

Sprint 3. Vinculaciones y social		Inicio: 16/04/2026 Fin: 29/04/2026
Ident.	Título	Estim.
HU-10	Envío de solicitud de registro de vivienda	3
HU-11	Recepción de solicitudes de registro de vivienda	3
HU-12	Registro de inquilino en vivienda	2
HU-19	Chat	8
HU-20	Conexión entre usuarios	3
HU-23	Verificación de tiempo de convivencia en valoraciones	2

Tabla 33. Sprint 3

Sprint 4. Sistema de valoraciones		Inicio: 30/04/2026 Fin: 06/05/2026
Ident.	Título	Estim.
HU-14	Valoraciones Inquilino-Inquilino	5
HU-15	Valoraciones Inquilino-Casero	5
HU-16	Valoraciones Inquilino-Vivienda	5
HU-17	Valoraciones Casero-Inquilino	5

Tabla 34. Sprint 4

Sprint 5. Sistemas de búsqueda y recomendaciones		Inicio: 07/05/2026 Fin: 27/05/2026
Ident.	Título	Estim.
HU-18	Sistema de Búsqueda	8
HU-24	Sistema de Recomendaciones	13

Tabla 35. Sprint 5

Sprint 6. Implementación del administrador		Inicio: 28/05/2026 Fin: 10/06/2026
Ident.	Título	Estim.
HU-13	Control de anuncios	3
HU-25	Eliminación de anuncios por administrador	3
HU-26	Control de valoraciones	3
HU-27	Baneo y suspensión de cuentas	3

Tabla 36. Sprint 6

Estos seis *sprints* planteados son los que definen el proceso de implementación explicado por módulos en el capítulo 6 de este documento.

3.3.2 Diagrama de Gantt

Un diagrama de Gantt es una herramienta de gestión de proyectos utilizada para la planificación de tareas y actividades necesarias durante el proceso de desarrollo de un producto. Hace de cronograma visual, mostrando la lista de tareas (o agrupaciones de tareas, en este caso), fechas de inicio y fin, hitos, dependencias y personas asignadas [82].

En el caso de este proyecto, cuyo diagrama de Gantt puede verse en la Ilustración 5, las tareas se han dividido en doce grupos, seis de los cuales son los *sprints* definidos en el apartado 3.3.1, mientras que los otros seis están conformados por las agrupaciones: inicio y planificación, donde se define la idea principal y la propuesta a la tutora del proyecto; análisis y diseño, la fase en la que se comienza a redactar el cuerpo y esquema general de la memoria y del proyecto en sí; la fase de pruebas y validación, llevada a cabo tras finalizar la implementación de todo el código, de forma que se puedan probar y terminar de refinar todas las funcionalidades de la aplicación al completo; la fase de documentación y redacción, que representará el proceso de escritura de esta memoria, por lo que acompañará a lo largo de toda la temporización del trabajo; y, por último, la preparación de la defensa,

un proceso que comenzará poco antes de entregar el trabajo y que se extenderá hasta la defensa del mismo.

Por otro lado, el diagrama cuenta a su vez con tres hitos durante el proceso de desarrollo; en primer lugar, tendremos la aprobación del análisis y diseño del proyecto, que tiene lugar a mediados de marzo, dando lugar al comienzo de la implementación del código de la aplicación; en segundo lugar, tenemos la finalización de la programación de la aplicación, que permitirá el comienzo de la fase de pruebas y validación, donde se podrán finalmente realizar pruebas integrales del software completo; por último, tenemos el hito de la entrega del trabajo, el día 22 de junio, como así se marca en el calendario de Trabajos de Fin de Grado de la ETSIT [83].

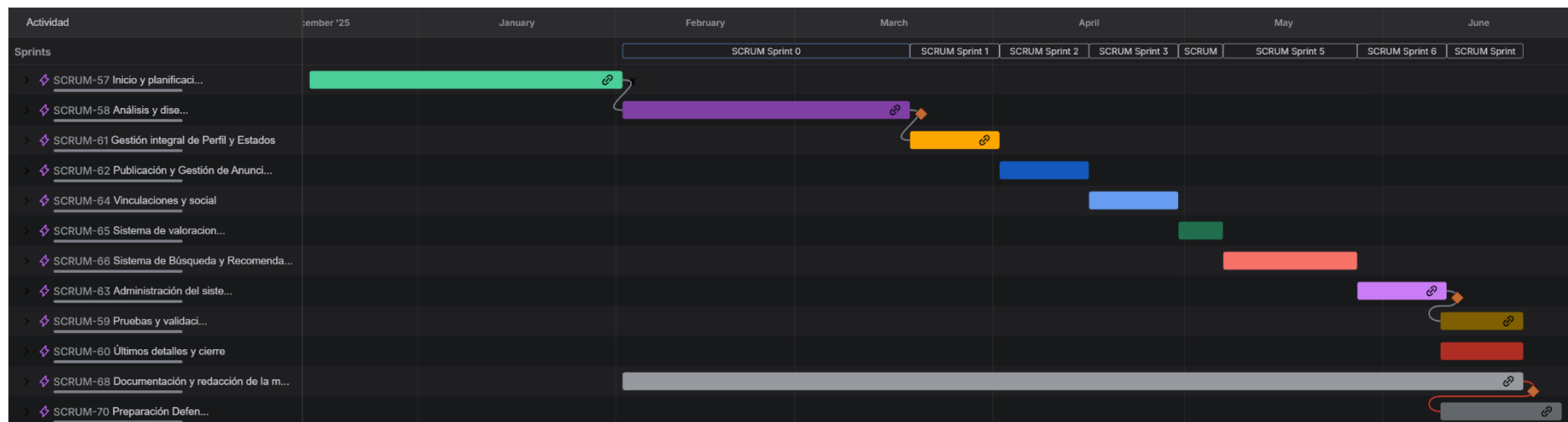


Ilustración 5. Diagrama de Gantt

3.4 Análisis de riesgos

Este proyecto, como suele pasar con los proyectos de desarrollo de software, está expuesto a una serie de riesgos y contingencias que pueden comprometer su alcance, planificación temporal o calidad. En este apartado se trata de identificar esos riesgos, clasificarlos según relevancia e idear un plan de prevención y mitigación para, en primer lugar, evitar que se den y, en segundo, reducir los daños lo máximo posible dado el caso de que no se puedan evitar.

Para priorizar los riesgos se ha valorado cada uno según su probabilidad de ocurrir (Baja / Media / Alta) y su impacto sobre el proyecto, siguiendo la misma escala. En la Tabla 37 se puede observar cómo la combinación de ambos da lugar al riesgo presentado para el sistema.

	Impacto		
Probabilidad	Impacto bajo	Impacto medio	Impacto alto
Probabilidad baja	Bajo	Bajo	Medio
Probabilidad media	Bajo	Medio	Alto
Probabilidad alta	Medio	Alto	Crítico

Tabla 37. Matriz de valoración de nivel de riesgo

A partir de esta escala, la Tabla 38 recoge los riesgos identificados junto con su valoración, y la Tabla 39 detalla las medidas de prevención y mitigación planteadas para cada uno.

ID	Riesgo	Categoría	Prob.	Impacto	Nivel
R1	Complejidad del sistema de recomendación (NLP y emparejamiento bidireccional) mayor de la prevista	Técnico	Alta	Alto	Crítico
R2	Curva de aprendizaje de tecnologías no cursadas en el grado (FastAPI, sentence-transformers, React Native/Expo)	Técnico	Media	Medio	Medio
R3	Subestimación del tiempo y alcance excesivamente ambicioso	Planificación	Alta	Alto	Crítico
R4	Imposibilidad de validar el recomendador con métricas estándar por falta de datos reales de uso	Técnico	Alta	Medio	Alto
R5	Problemas de integración o consistencia entre el microservicio FastAPI y el <i>backend</i> Node/PostgreSQL	Técnico	Media	Medio	Medio
R6	Pérdida de datos o fallo del equipo de desarrollo	Externo	Baja	Alto	Medio
R7	Dependencia de servicios de terceros y posibles cambios en sus precios o condiciones (APIs de Google)	Externo	Media	Medio	Medio
R8	Tratamiento de datos personales sensibles de convivencia y cumplimiento del RGPD	Legal	Media	Alto	Alto
R9	Solapamiento con otras asignaturas y carga académica	Planificación	Media	Medio	Medio

Tabla 38. Identificación y valoración de riesgos del proyecto

ID	Prevención	Mitigación
R1	Prototipado temprano y validación incremental del motor	Implementar una primera versión funcional y posponer el filtrado colaborativo a trabajo futuro
R2	Pruebas de concepto y estudio previo de las tecnologías	Apoyo en la documentación oficial y en herramientas de asistencia al desarrollo
R3	Planificación por <i>sprints</i> y definición de un producto mínimo viable	Priorización de funcionalidades y descarte de las no esenciales (p. ej. el chat, la versión web o las pruebas funcionales integrales, que se dejan para cuando la aplicación alcance su versión final)
R4	Uso del test de personalidad como aproximación inicial ante la ausencia de histórico	Reconocimiento explícito de la limitación y planteamiento de la validación empírica como línea futura

R5	Definición de contratos de API claros entre servicios	Pruebas de integración y registro de errores
R6	Control de versiones con Git y repositorio remoto en GitHub	Restauración del proyecto desde el repositorio remoto
R7	Diseño con una capa de abstracción sobre el proveedor externo	Sustitución por una alternativa equivalente, conforme a la comparativa de proveedores realizada
R8	Minimización de datos y obtención del consentimiento del usuario	Anonimización y revisión de los filtros sobre atributos sensibles (p. ej. el género)
R9	Planificación temporal realista con holguras entre <i>sprints</i>	Redistribución de tareas y reajuste del cronograma

Tabla 39. Plan de prevención y mitigación de los riesgos

Una vez finalizada la fase de implementación, se pueden observar cuáles de estos riesgos han presentado mayores problemas, cómo se han tratado de evitar y cómo se han tratado en caso de ocurrir. De esta forma, podemos ver cómo los dos riesgos identificados como críticos (R1 y R3) han llegado a producirse, dando lugar a la acotación del alcance del sistema de recomendaciones, dejando parte de la implementación como trabajo a futuro; así como la adaptación temporal del desarrollo del sistema, priorizando aquellos aspectos esenciales y realmente relevantes en esta primera versión de implementación del sistema, dejando como trabajo a futuro la implementación de módulos como, por ejemplo, el chat entre usuarios de la aplicación.

Por otro lado, el riesgo R4 se ha asumido como consecuencia del riesgo crítico R1, implementando el test de personalidad y dejando la validación del recomendador para implementar más adelante, con datos de usuarios reales.

Si bien el resto de los riesgos no han llegado a materializarse, las técnicas de prevención planteadas para ellos también se han mantenido a lo largo de todo el desarrollo del proyecto, en especial el control de versiones a través de GitHub [1].

Esta gestión deja clara la capacidad de anticipación y adaptación ante diversas complicaciones encontradas a lo largo del desarrollo.

3.5 Presupuesto

En este punto se va a realizar el cálculo del coste estimado del proyecto a realizar. Se dividirá en dos secciones, una para la estimación inicial antes de realizar el proyecto en sí, y la segunda para reflejar un recuento aproximado de horas reales dedicadas contando con las diferentes desviaciones y problemas encontrados, proponiendo un presupuesto más realista en función de los datos recogidos.

3.5.1 Estimación inicial del presupuesto

Para la estimación inicial antes de la implementación del proyecto, se va a partir de la premisa indicada por la Universidad de Granada sobre el número de horas estimadas por crédito reconocido, que es de 25 [84]. Teniendo en cuenta que la asignatura del Trabajo de Fin de Grado en Ingeniería Informática por la Universidad de Granada es de 12 ECTS, la estimación inicial de horas es de 300 para la realización de este proyecto, contando todas sus fases de desarrollo.

Para el cálculo total del presupuesto, se deben tener en cuenta varios aspectos que se detallan a continuación.

- Coste de personal

Para el cálculo del coste de personal se va a tener en cuenta el sueldo medio de un desarrollador de software junior en España, ya que sería (o podría ser) el cargo ocupado para la realización de un proyecto de este tipo, sumado a los valores de IRPF (que ya va incluido en el sueldo medio, al ser bruto) y Seguridad Social que una empresa tendría que asumir para la contratación de un perfil como el definido.

Para este puesto, los datos indican que el sueldo medio actual en España ronda los 25.000€ brutos anuales [85], [86], que, sumando el coste de la Seguridad Social asumido por la empresa, el cual se puede ver reflejado en la Tabla 40, habría que añadirle un ~32,15% adicional, quedando un total de:

$$25000€ \cdot (1 + 0,3215) = 25000€ \cdot 1,3215 \approx 33037,50€$$

Teniendo el sueldo anual, para calcular el sueldo por hora, partimos de la premisa de una jornada completa (~40 horas semanales) y unos 36 días no trabajados como máximo, que salen de la suma de vacaciones (22 días laborables) y festivos (14 días como máximo), que hacen un total de 7 semanas no trabajadas que, restadas a las 52 que tiene un año, se queda en un total de 45 semanas. Por tanto, tendríamos:

$$45 \text{ semanas} \times 40 \text{ horas/semana} = 1800 \text{ horas anuales}$$

$$\text{Coste_empresa/hora} = 33037,50€ \div 1800 \text{ horas} \approx 18,35€/hora$$

Sabiendo el coste total asumido por la empresa por cada hora trabajada, y que se han estimado 300 horas de trabajo real, tendríamos un total de:

$$18,35€/hora \times 300 \text{ horas} = 5505€ \rightarrow \text{Coste de personal}$$

Concepto	Porcentaje a cargo de la empresa
Contingencias comunes	23,60%
Desempleo (indefinido)	5,5%
Fondo de Garantía Salarial (FOGASA)	0,20%
Formación profesional	0,60%
MEI (Mecanismo Equidad Intergeneracional)	0,75%
Accidentes trabajo (tarifa mínima oficinas)	1,50%
TOTAL	32,15%

Tabla 40. Porcentajes de contratación indefinida a cargo de la empresa en España

Fuente: [87]

- Coste de material

Estos son todos los gastos relacionados con el equipo informático usado para el desarrollo e implementación de este proyecto, así como toda licencia de software o suscripción requerida o utilizada.

- Equipo

Este proyecto se ha realizado con un portátil de la marca ASUS (modelo ZenBook 14) del año 2021, con un procesador Intel Core i7 de 11ª Generación y 16 GB de RAM, cuyo precio de mercado fue 1099€, al quitar el 21% IVA (dividiendo entre 1,21), el precio es de 908,26€.

Según la tabla de amortizaciones de la Agencia Tributaria Española [88], regida por la ley 27/2014, de 27 de noviembre, del Impuesto sobre Sociedades [89], la amortización de “equipos para procesos de información” representa un coeficiente lineal anual del 25%, es decir, 4 años de vida útil. De esta forma, podemos calcular el porcentaje de la vida útil empleada del portátil utilizado para la realización de este proyecto:

Cálculo del 25% del precio (amortización anual):

$$908,26 \times 0,25 = 227,06€$$

Cálculo de la amortización mensual:

$$227,06 \div 12 = 18,92€$$

Contando que el desarrollo del proyecto va de febrero a junio de 2026:

$$18,92\text{€} \times 5 \text{ meses} = \mathbf{94,6\text{€}}$$

- Licencias de software

Además del equipo informático, también se requiere de ciertas licencias de software para la realización del proyecto, definidas en la Tabla 41.

Licencia	Uso	Coste	Tipo	Precio comercial
JetBrains	DataGrip como SGBD	0€	Licencia estudiante	10,90€/mes (particulares) × 5 meses = 54,5€ [90]
Google Cloud API	Para servicios de mapas, búsqueda por ciudades, etc.	Capa gratuita de ~250€/mes, no excedida durante el desarrollo	Licencia comercial (capa gratuita)	Coste mensual en función de la carga de usuarios en el sistema [91]
Google Gemini Pro	Apoyo y ayuda en el desarrollo de las diferentes partes del proyecto	0€ (plan de estudiante)	Licencia estudiante	21,99€/mes [92]
VS Code, PostgreSQL, Docker, Node, Expo Go, FastAPI	Para el desarrollo e implementación de código	0€	Open source	0€
Figma	Diseño y desarrollo de logos y mockups de la aplicación	0€	Licencia estudiante	20€/mes × 5 meses = 100€ [93]

Tabla 41. Licencias de software usadas en el proyecto

Como se puede observar en la tabla; todas las licencias de software y programas usados son libres, se han usado con licencia estudiante (siendo su coste 0), o están dentro de la capa gratuita; por lo que no representan un coste adicional en el presupuesto del proyecto.

- Costes de despliegue y mantenimiento

Estos son todos los costes que conllevaría la puesta en producción del proyecto, requiriendo una infraestructura (servidores, contenedores, etc.) además de las licencias de publicación y despliegue en las tiendas de aplicaciones reconocidas para dispositivos Android y Apple.

- Infraestructura

Una vez puesto en producción, el producto requerirá de una infraestructura que lo aloje y permita su correcto funcionamiento. Para estimar los costes de despliegue y mantenimiento se propone, a modo de referencia, un despliegue en Amazon Web Services (región eu-west-1, Irlanda), por tratarse de uno de los proveedores de referencia del mercado, ofrecer precios públicos y verificables, y disponer de una configuración ajustada a los requisitos del sistema. La configuración se dimensiona según el componente más exigente (el microservicio de recomendación), que requiere mantener el modelo de NLP cargado en memoria: por ello se opta por una instancia EC2 t3.medium (4 GiB de RAM) que aloja tanto el *backend* como dicho microservicio, una base de datos gestionada RDS PostgreSQL, almacenamiento de objetos en S3 para las imágenes y la gestión de dominio mediante Route 53. El coste estimado asciende a ~50 \$/mes (~524 €/año) [94], ~218€ durante el tiempo de desarrollo (5 meses).

En una primera fase de despliegue de StayIn, el mantenimiento del sistema podría permanecer limitado al coste recurrente de la infraestructura, por lo que no se va a añadir coste adicional por este trabajo aún.

- Licencias y dominios

Al tratarse de una aplicación móvil, hace falta tener en cuenta los costes de despliegue en las diferentes tiendas de aplicaciones, Google Play Store y Apple App Store, cuyos precios son de 25\$ en pago único [95] y 99\$ anuales [96], respectivamente, lo que aumentaría el presupuesto en ~108.17€.

Además de la publicación, habría que hacer uso de un dominio web donde alojar, al menos, una *landing page*, de forma que la aplicación se pudiera presentar de manera atractiva para el usuario, ofreciendo los enlaces de descarga a las tiendas de aplicaciones. El coste de un dominio con extensión .com sería de ~15€ [97], mientras que el despliegue de la página se podría hacer mediante servicios gratuitos sin problema.

De esta manera, obtenemos un total de:

$$218€ + 108,17€ + 15€ = \mathbf{341,17€}$$

- Presupuesto total

Para el cálculo del presupuesto total, sumamos los diferentes costes estimados, obteniendo un total de:

$$5505€ + 94,6€ + 341,17€ = \mathbf{5940,77€}$$

3.5.2 Estimación con recuento de horas aproximado

Una vez terminada gran parte de la implementación del proyecto y finalizando los aspectos de este documento, se puede hacer una estimación de presupuesto ligeramente más ajustada al contar con un recuento más realista del número de horas dedicadas al proceso de desarrollo de StayIn.

Para este recuento de horas se ha hecho uso de varios mecanismos de monitorización. Por un lado, se ha utilizado la extensión WakaTime [98] de VS Code, el IDE usado para la implementación del código del proyecto; que muestra el tiempo real dedicado a la creación y edición de código dentro de un proyecto concreto. La gran ventaja de esta extensión es que, a diferencia de otros programas, como veremos a continuación, WakaTime solo hace registro del tiempo empleado realmente editando archivos, de manera que muestra el tiempo dedicado a la programación del código del proyecto en su totalidad. En el caso de este proyecto, el tiempo registrado es de 106 horas. Contando con que este tiempo es exclusivamente dedicado a la implementación del código en sí, podría hacerse una estimación de que, para las tareas de diseño y estructuración de las ideas que posteriormente han sido registradas por la extensión del IDE, se ha empleado adicionalmente, al menos, la mitad de ese tiempo, lo que sería un total de 53 horas.

Por otro lado, tenemos la redacción de la documentación del trabajo realizado recogida en este documento. El programa Word también lleva un recuento del tiempo de edición dedicado a un archivo, la diferencia es que, como se mencionaba antes, este tiempo no funciona de la misma manera que lo hace la extensión WakaTime. En este caso, Word registra todo el tiempo que el archivo permanece abierto en primer plano (como pestaña principal del dispositivo) esté siendo editado o no, por lo que el tiempo registrado puede ser considerablemente más elevado de lo que la realidad refleja. No obstante, el uso empleado del documento en primer plano, en la mayoría del tiempo ha sido bien para editarlo y redactar la memoria, o bien para leer y buscar información sobre diversos temas referenciados a lo largo del documento. Por este motivo, se ha decidido contar de manera aproximada con el 70% del tiempo registrado por Word, ya que se considera que representa fielmente la cantidad de trabajo dedicado a la realización de esta memoria. El tiempo de edición

registrado para este documento es de 19090 minutos, es decir, ~318 horas. Si calculamos el 70% de ese tiempo, tenemos un total de:

$$318 \text{ horas} \times 0,7 \approx 222 \text{ horas}$$

Si sumamos todas las horas empleadas estimadas, tenemos un total de:

$$106 + 53 + 222 = 381 \text{ horas estimadas}$$

Aplicando este nuevo recuento al cálculo del coste de personal detallado anteriormente, el resultado obtenido sería de:

$$18,35\text{€/hora} \times 381 \text{ horas} = \mathbf{6991,35\text{€}} \rightarrow \text{Coste de personal real estimado}$$

Como el resto de los costes calculados en el punto anterior no varía (material y mantenimiento y despliegue), el coste real total estimado sería de:

$$6991,35\text{€} + 94,6\text{€} + 341,17\text{€} = \mathbf{7427,12\text{€}}$$

Como se puede comprobar, a pesar de no tener un recuento exacto de todas las fases del desarrollo del trabajo realizado, este nuevo cálculo sirve para mostrar una estimación más cercana a la realidad basada en datos empíricos recogidos por el software utilizado para las diferentes partes que componen la realización de un Trabajo de Fin de Grado con estas características. También permite reflejar las desviaciones temporales acontecidas respecto a la estimación inicial.

Una vez calculadas las estimaciones realizadas sobre el presupuesto del proyecto, a continuación, se detallan los aspectos legales relacionados al despliegue y puesta en producción de esta aplicación.

3.6 Aspectos legales

El desarrollo y, especialmente, una eventual puesta en producción de StayIn implican el tratamiento de datos personales y la intermediación entre usuarios, lo que obliga a tener presentes varios marcos normativos. A continuación, se recogen los principales aspectos legales considerados en relación con el proyecto.

3.6.1 Protección de datos personales

Al registrar y gestionar usuarios, la aplicación trata datos personales (nombre, correo, DNI, teléfono, fecha de nacimiento, sexo y fotografía de perfil), por lo que queda sujeta al Reglamento (UE) 2016/679 (RGPD) [99] y a la Ley Orgánica 3/2018 (LOPDGDD) [100]. Esto implica respetar los principios de licitud, limitación de la finalidad y, sobre todo, minimización de datos, debiendo justificarse la necesidad de campos sensibles como el DNI; así como garantizar los derechos de los interesados (acceso, rectificación, supresión, oposición, limitación y portabilidad). El proyecto incorpora medidas alineadas con estos principios, como el almacenamiento de las contraseñas mediante funciones de hash y el borrado en cascada de la información asociada a un usuario al eliminar su cuenta, en línea con el derecho de supresión y con el enfoque de privacidad desde el diseño y por defecto. Una versión de producción requeriría, además, una política de privacidad completa, la recogida explícita del consentimiento y el establecimiento de una edad mínima de registro conforme a la normativa (con carácter general, 18 años; 14 con las salvedades previstas).

3.6.2 Tratamiento de datos de terceros y derecho al honor

Una particularidad de StayIn es que las valoraciones contienen información sobre otras personas y sobre las viviendas, lo que supone tratar datos de terceros y afecta a derechos

fundamentales como el honor, la intimidad y la propia imagen, protegidos por el artículo 18 de la Constitución y por la Ley Orgánica 1/1982 [101]. Por ello, la plataforma incorpora un control y moderación de los contenidos publicados que permite rechazar valoraciones con lenguaje ofensivo o que vulneren las normas, reduciendo el riesgo de injurias o difamación y ofreciendo al usuario la posibilidad de corregir el contenido conflictivo.

3.6.3 Condición de prestador de servicios de la sociedad de la información

Como plataforma que aloja contenidos generados por los usuarios (anuncios y valoraciones), StayIn queda sujeta a la Ley 34/2002 (LSSI-CE) [102] y al Reglamento (UE) 2022/2065 [103] (Reglamento de Servicios Digitales, DSA). De ello derivan obligaciones de información (aviso legal, condiciones de uso y, en su caso, política de cookies) y un régimen de responsabilidad limitada sobre los contenidos de terceros, condicionado a que el prestador actúe con diligencia retirando los contenidos ilícitos de los que tenga conocimiento. El módulo de administración, que permite controlar y eliminar anuncios y valoraciones, da soporte precisamente a ese deber de diligencia.

3.6.4 Normativa de arrendamiento y vivienda

Aunque el dominio de la aplicación es el alquiler de vivienda, regulado por la Ley 29/1994 de Arrendamientos Urbanos [104] y por la Ley 12/2023 [105] por el derecho a la vivienda, StayIn actúa exclusivamente como plataforma de contacto y compatibilidad entre las partes, sin formalizar ni ser parte de los contratos de arrendamiento. La relación contractual se establece directamente entre casero e inquilino y queda fuera del alcance de la aplicación, lo que limita su exposición a esta normativa, sin perjuicio del deber de diligencia frente a anuncios fraudulentos.

3.6.5 Propiedad intelectual y licencias de software

Por último, el proyecto se apoya en componentes de terceros de código abierto (entre otros, React Native y Expo, FastAPI, PostgreSQL y los modelos de sentence-transformers), cuyas licencias respectivas (MIT, Apache 2.0 y la licencia PostgreSQL) permiten su uso, pero imponen condiciones de atribución y conservación de los avisos de licencia que deben respetarse. Asimismo, las imágenes y contenidos que los usuarios suben a sus anuncios deben contar con los derechos correspondientes, extremo que debería recogerse en las condiciones de uso de la plataforma.

En conjunto, el proyecto ha tenido presentes estos marcos durante su diseño e implementación, si bien una puesta en producción real requeriría completar la documentación legal (política de privacidad y condiciones de uso) y, recomendablemente, una revisión jurídica específica.

4. ANÁLISIS DEL SISTEMA

En este capítulo se va a mostrar qué vamos a hacer, qué funcionalidades van a cumplirse y se van a tratar los casos de uso más relevantes del sistema.

4.1 Definición de actores

Como en todo producto de software basado en UML, el sistema cuenta con una serie de actores principales, que serán aquellos que inician los casos de uso; y secundarios, que serán los actores que reaccionan o asisten. Juntos conformarán el funcionamiento del sistema y los casos de uso de este. En este punto se van a definir todos los actores que intervengan en el funcionamiento de la aplicación.

4.1.1 Jerarquía de actores

Con el fin de evitar la redundancia en la definición del sistema, se ha establecido una relación jerárquica entre algunos usuarios del sistema, partiendo de un actor base, denominado Usuario Registrado, que podrá ser tanto Inquilino como Casero.

4.1.2 Actores principales

- Usuario registrado

Es todo usuario que consta de una cuenta en el sistema, independientemente del rol que desempeñe.

- Usuario con rol de Inquilino

Será un Usuario registrado con el rol de inquilino, de manera que tendrá unos estados de búsqueda y hospedaje asociados.

- Usuario con rol de Casero

Será un Usuario registrado con el rol de casero, por lo que tendrá, al menos, una vivienda asociada, unos posibles inquilinos asociados y un estado de búsqueda asociado.

4.1.3 Actores secundarios

- Administrador

Será el actor encargado de la gestión y administración del sistema, garantizando el funcionamiento y cumplimiento de sus normas.

- Sistema

Representará a la aplicación en sí y su lógica interna, actuando como actor secundario en ciertos casos como la ejecución de los algoritmos de recomendación.

4.2 Descripción del problema

En este apartado se van a definir las particularidades principales del proyecto, explicando los tipos de usuarios del sistema, los posibles estados, conexiones, valoraciones, aspectos de la convivencia a tener en cuenta y la gestión de los anuncios, de forma que se entienda mejor el funcionamiento esperado de la aplicación.

4.2.1 Roles de usuario

Los usuarios de la aplicación contarán con dos posibles roles, los cuales se describen a continuación:

- Casero

Será un tipo de usuario con una vivienda asociada, de la cual será propietario. Este tipo de usuario busca inquilinos para su vivienda.

- Inquilino

Este usuario será una persona con intención de compartir piso con otra gente, ya sea buscando personas para ocupar algún hueco/s disponible/s en el piso donde está alquilado, o buscando gente que ya se encuentre en un piso y tenga intención de entrar a vivir con ellos.

- Administrador

Será el usuario (o usuarios) encargado del funcionamiento del sistema y del cumplimiento de las normas de este. Podrá realizar acciones sobre el resto de los elementos de la aplicación, como usuarios, anuncios o valoraciones, regulándolos e incluso pudiendo llegar a eliminarlos.

Los roles de casero e inquilino podrían, en algunos casos, donde un usuario tenga algún anuncio de vivienda activo y además sea inquilino de alquiler en otro inmueble, estar activos al mismo tiempo.

4.2.2 Estados

Los usuarios con rol de inquilino tendrán los siguientes posibles estados, dentro de varios niveles. Por un lado, tendremos el nivel de hospedaje, en el cual se indicará si el usuario se encuentra actualmente hospedado en alguna vivienda y, por tanto, podemos decir que “tiene piso”, o si no está hospedado, es decir, no se encuentra residiendo actualmente en ninguna vivienda. El otro nivel que tendríamos sería el de búsqueda, ya sea de compañeros o de vivienda. Por tanto, tendríamos lo siguiente:

- Hospedaje
 - Hospedado.
 - No hospedado.
- Búsqueda
 - Búsqueda de compañeros.
 - Búsqueda de piso.
 - No busca nada.

Teniendo estos detalles en cuenta, podríamos tener varios ejemplos de usuarios:

- Caso 1
Estudiante *hospedado* en un piso *sin necesidad de buscar* uno nuevo o compañeros.
- Caso 2
Estudiante *hospedado en búsqueda* de un *nuevo piso*.
- Caso 3
Estudiante *hospedado* en un piso *en búsqueda de compañeros*.
- Caso 4
Estudiante *no hospedado en búsqueda de piso*.

4.2.3 Conexiones

El ecosistema de la aplicación se basa principalmente en la confianza de las relaciones entre usuarios. A diferencia de la conexión habitual que hay en las redes sociales tradicionales, donde las conexiones se realizan de manera arbitraria, mandando una petición a otro usuario y esperando su respuesta, aquí las conexiones están estrictamente ligadas a un evento físico y temporal: la relación de convivencia entre dos usuarios.

- Tipos de conexiones

Una relación de convivencia tiene lugar cuando dos usuarios comienzan a ser compañeros, o cuando un usuario pasa a ser casero o inquilino de otro. Este tipo de relaciones definen el sistema fundamental de la aplicación y se dividen en dos tipos:

- Conexiones horizontales

Se trata de un vínculo bidireccional entre dos usuarios con rol de inquilino que actúan como compañeros, es decir, que conviven o han convivido en el mismo inmueble.

- Conexiones verticales

Es el vínculo formado entre un usuario con rol de casero y uno o varios usuarios con roles de inquilinos.

- Ciclo de vida y estados de conexión

Estas conexiones poseen, a su vez, un ciclo de vida asociado, habiendo dos tipos distintos:

- Conexión activa

Es una conexión donde los dos usuarios son, o bien compañeros, o bien inquilino y casero, en ese preciso instante.

- Conexión histórica

Reservado para esas conexiones que han tenido lugar, pero han terminado, es decir, los usuarios ya no son ni compañeros, ni casero e inquilino. En el caso de que un usuario tenga una conexión histórica con otro y, en otro momento, vuelva a tener relación de convivencia, se creará una nueva conexión, manteniendo la histórica intacta.

- Gestión de la privacidad de las conexiones

El sistema implementa un control de la privacidad a los usuarios, de manera que puedan decidir si quieren que sus conexiones actuales sean visibles para el resto de los usuarios, o no. La visibilidad de la conexión para otros usuarios dependerá de que ambas partes de la conexión así lo configuren. Para el caso de las conexiones históricas, serán siempre visibles todos los usuarios de la aplicación, de manera que un usuario podrá ver las conexiones históricas comunes que tenga con otro en todos los casos.

Esto se implementa así con el fin de favorecer la transparencia y seguridad del ecosistema de la aplicación, evitando la ocultación de historiales conflictivos entre usuarios.

- Impacto en la lógica de negocio y la aplicación

- Registro de valoraciones

El sistema usa las conexiones como una regla de validación estricta, de manera que un usuario solo pueda valorar a otro si, y solo si, existe un registro de conexión entre ambos.

- Motor de recomendaciones

Las conexiones conforman un grafo social, de manera que el sistema sea capaz de inferir afinidad entre perfiles, recomendando así usuarios o viviendas que presenten características similares a las conexiones pasadas que hayan sido valoradas positivamente.

4.2.4 Valoraciones

La idea principal de por qué esta aplicación podría llegar a ser útil, es la posibilidad de poder valorar y puntuar diferentes aspectos de la convivencia de aquellos usuarios con los que se haya convivido, así como de valorar, por un lado, los inmuebles donde un usuario se ha alojado y, por otro, la figura del inquilino durante su estancia. De esta forma, se definen los distintos aspectos que conforman cada una de estas valoraciones.

- **Tiempo de convivencia o estancia**

Aquí se especificará el periodo de tiempo en el que llevó a cabo la convivencia entre ambas entidades del sistema. No es un aspecto de la valoración como tal, simplemente sirve para saber cómo de fiable puede ser esa valoración. Siempre puede haber casos en los que la convivencia haya sido corta o haya acabado de mala manera, por lo que las valoraciones de esa conexión podrían estar sesgadas o buscando “destrozar” la valoración del usuario o anuncio.

- **Puntuación de los aspectos de la convivencia**

Puntuación sobre 5 estrellas de cada aspecto de la convivencia (ver Aspectos de la convivencia entre usuarios y Aspectos de la estancia en un inmueble).

- **Comentario general**

Una breve información adicional acerca de la convivencia general con esa persona y experiencia personal.

El objetivo de la valoración es representar, en lo que se considerarían los aspectos más importantes de la convivencia o estancia, cómo es realmente una persona a la hora de convivir con ella, así como de reflejar la realidad de un inmueble. No se trata de buscar la perfección, sino la compatibilidad. No tiene sentido que una persona que se conforma con que la vivienda esté relativamente ordenada (pongamos un 2-3 sobre 5), busque convivir con alguien obsesionado con la limpieza (un 5), ya que esto puede dar como resultado diversos problemas de convivencia por ambas partes.

4.2.5 Aspectos de la convivencia entre usuarios

Estos aspectos reflejan los puntos a evaluar en las valoraciones realizadas entre usuarios que han convivido como compañeros (Inquilino ↔ Inquilino.).

- **Higiene y orden**

- **Frecuencia y calidad de limpieza**

¿Cumple el turno / horario de limpieza?

- **Gestión de la vajilla**

¿Friega los platos cuando termina de comer o los deja apilados hasta el día o días después?

- **Zonas comunes**

¿Es ordenado en las zonas comunes de la vivienda?

- **Economía y responsabilidad**

- **Puntualidad de los pagos**

¿Paga los gastos/alquiler a tiempo?

- **Gastos compartidos**

¿Es proactivo a la hora de reponer los elementos en común?

- **Consumo responsable**

¿Realiza un consumo responsable de la luz/agua/calefacción de la vivienda?

- **Ruido y horarios**

- **Respeto de las horas de descanso**

¿Respeto las horas de silencio de la vivienda (noche/madrugada/horas impuestas en la vivienda)?

- Visitas
 - Frecuencia de visitas

¿Trae visitas a casa a menudo?

- Uso de espacio con invitados

¿Cuándo hay visita, se respeta el uso de zonas comunes, permitiendo al resto de inquilinos usarlas cómodamente?

- Comunicación
 - Reacción ante el conflicto

¿Es resolutivo a la hora de resolver problemas en la convivencia?

- Empatía

¿Es comprensible con los posibles problemas que hayan podido surgir?

- Forma de comunicar las cosas

¿Es expresivo con aquellas cosas que considera problemas?

- Respeto de la propiedad
 - Cuidado del inmobiliario de la vivienda

¿Trata bien los electrodomésticos y muebles?

4.2.6 Aspectos de la estancia en un inmueble

Estos aspectos muestran los que se han considerado como los más importantes de cara a la estancia en un inmueble.

- Veracidad respecto al anuncio
 - Correspondencia con el anuncio

¿La vivienda se correspondía con las fotos y la descripción publicadas?

- Condiciones anunciadas

¿El precio, los gastos y las normas coincidían con lo anunciado?

- Estado y mantenimiento
 - Estado general del inmueble

¿La vivienda se encontraba en buen estado de conservación al entrar?

- Limpieza inicial

¿Estaba la estancia limpia y lista para habitar?

- Ausencia de desperfectos

¿Estaba libre de humedades, plagas o desperfectos relevantes?

- Equipamiento
 - Electrodomésticos

¿Los electrodomésticos funcionaban correctamente?

- Mobiliario

¿El mobiliario era suficiente y estaba en buen estado?

- Calefacción

¿Disponía de calefacción o refrigeración adecuada y funcional?

- Relación calidad-precio

- Precio adecuado

¿El precio era razonable para lo que ofrecía la vivienda?

- Proporción de gastos

¿Los gastos asociados eran proporcionados?

- Desviaciones encontradas respecto a las valoraciones

Como se ha comentado en distintas ocasiones anteriormente y como reflejan las Historias de Usuarios definidas en el capítulo 3, la intención principal era incluir 4 tipos distintos de valoraciones: Inquilino ↔ Inquilino, Inquilino ↔ Vivienda, Inquilino ↔ Casero y Casero ↔ Inquilino. No obstante, como se detalla en este apartado, solo se han registrado los aspectos de la convivencia para los dos primeros tipos de valoraciones. Esto es debido a una decisión de producto, ya que a la hora de comenzar a definir los diferentes aspectos de las valoraciones Inquilino ↔ Casero y Casero ↔ Inquilino, se plantean dos problemas principales.

- Carga de valoraciones a los usuarios

Las valoraciones son uno de los pilares fundamentales de este sistema y el calor diferenciador respecto a otros portales de búsqueda ya mencionados anteriormente en este documento. Debido a esto, uno de los principales problemas que podría plantear esta aplicación sería la falta de apoyo en ese sentido por parte de los usuarios, por lo que es fundamental hacer el proceso de valoración lo más ameno y llamativo posible, evitando así que los usuarios se aburran o cansen en el proceso.

Si se añadieran las 4 valoraciones planteadas inicialmente, un usuario de la aplicación que haya convivido durante un año con otros dos compañeros tendría que dejar 4 valoraciones por separado, una para cada uno de los compañeros, casero y vivienda. Como se puede ver, esto no cumple con la necesidad de hacer el proceso de valoración ameno y llamativo, ya que presentar tantas valoraciones a la vez puede resultar abrumador y puede motivar al usuario a dejar la aplicación.

- Ausencia de valor real significativo

Además de lo comentado anteriormente, pese a que las valoraciones dedicadas a cada sujeto presentarían información relevante en cuanto a ese aspecto y por eso se diseñaron en un inicio, el problema anterior da lugar al planteamiento de una solución en la que esos aspectos podrían ser suplidos perfectamente mediante las dos valoraciones diseñadas.

Por un lado, la valoración de un casero a un inquilino, aunque daría información al resto de caseros de la aplicación, no representa un factor de relevancia grande, puesto que el casero no ha convivido con esa persona, y podría suplirse con las mismas valoraciones del resto de compañeros que haya tenido.

Por otro lado, en caso de querer especificar algo sobre el casero, un inquilino siempre podría reflejar su opinión en el comentario de la valoración realizada sobre el inmueble.

Por estos motivos, se ha decidido mantener exclusivamente las valoraciones entre inquilinos y las de inquilinos a inmuebles, implementando acordemente el Sistema de valoraciones (Sprint 4) y Sistema de recomendación (Sprint 5) de la aplicación.

4.2.7 Anuncios de las viviendas

Como se ha comentado a lo largo de este documento, uno de los principales objetivos de este proyecto es lanzar una aplicación que sea lo más fiable y transparente posible. Es por esto por lo que se aplicarán una serie de normas y exigencias a todos los anuncios de viviendas, de manera que se eviten los anuncios de viviendas engañosos, o aquellos en los que no se vea bien cómo es realmente la vivienda.

- Multimedia

Todos los anuncios deberán contar con elementos multimedia (imágenes, vídeos, etc.) que representen fielmente la realidad del inmueble anunciado. La presencia y obligatoriedad de estos elementos detalla a continuación.

- Imagen clara de cada habitación

Será obligatoria la adición de una o varias imágenes visibles para cada una de las habitaciones de las que el usuario interesado vaya a hacer uso (dormitorio, zonas comunes y, al menos, uno de los baños, en caso de haber más de uno).

Además de estas imágenes; será indispensable la presencia de, al menos, uno de los siguientes puntos en todos los anuncios.

- Plano de la vivienda

Un plano fiel del inmueble, donde se muestre claramente la distribución de este, dando al usuario interesado una idea clara de lo que se está anunciando.

- Vídeo recorrido de la vivienda

Un vídeo donde se recorran todas las zonas del inmueble de las que el usuario interesado vaya a hacer uso.

Serán valorados positivamente los anuncios que contengan alguno de los siguientes puntos:

- Ruta 3D de la vivienda

Ruta 3D donde se pueda recorrer la vivienda de manera interactiva (cuando esté implementado).

4.3 Requisitos funcionales

4.3.1 Gestión de usuarios y roles

- **RF-01 Login:** El usuario debe de poder registrarse con su correo electrónico en el sistema.
- **RF-02 Restablecimiento de contraseñas:** el usuario registrado debe poder recuperar o restablecer su contraseña mediante correo electrónico en caso de haberla olvidado.
- **RF-03 Edición de cuentas:** El usuario registrado debe poder editar los datos de su cuenta, así como su nombre de usuario, foto de perfil, descripción, etc.
- **RF-04 Roles:** El usuario debe poder elegir el rol que desempeñará (casero, inquilino).
- **RF 05 Convivencia de roles:** El usuario registrado podrá tener asociados los roles de inquilino y de casero al mismo tiempo, permitiéndole tanto hospedar, como ser hospedado.
- **RF-06 Estado de búsqueda:** El usuario debe poder declarar y editar si está en búsqueda y, en caso de estarlo, de qué (compañeros, piso, inquilinos).
- **RF-07 Estado de hospedaje:** El usuario con rol de inquilino debe poder establecer y modificar su estado de hospedaje a “hospedado” o “no hospedado”.

4.3.2 Gestión de anuncios de viviendas

- **RF-08 Registro de anuncios:** El usuario con rol de casero debe poder registrar un anuncio de piso o habitación en la aplicación.
- **RF-09 Edición de anuncios:** El usuario con rol de casero debe poder editar, pausar o eliminar los datos de sus anuncios publicados.
- **RF-10 Solicitud de registro:** El usuario con rol de casero debe poder mandar una solicitud de registro de inquilino en la vivienda al usuario que quiera registrar en esta.
- **RF-11 Aceptación de solicitudes:** El usuario con rol de inquilino debe poder aceptar y rechazar las solicitudes de registro enviadas por los usuarios con rol de caseros.
- **RF-12 Registro de inquilinos:** El usuario con rol de casero debe poder registrar inquilinos en su propiedad.

- **RF-13 Control de anuncios:** El sistema debe verificar que los anuncios cumplan con las normas de la aplicación (especificadas en el resumen general del proyecto).
- **RF-14 Denegación de anuncios:** el sistema debe poder denegar anuncios que incumplan con las normas establecidas.

4.3.3 Gestión de valoraciones

- **RF-15 Valoraciones de usuarios:** El usuario debe poder valorar tanto viviendas como a usuarios con los que haya convivido.
- **RF-16 Sistema de valoraciones:** El sistema de valoraciones deberá obligatoriamente incluir los siguientes campos:

En caso de usuarios:

- Tiempo de convivencia.
- Puntuación sobre 5 estrellas de cada aspecto de la convivencia:
 - Higiene y orden (limpieza, vajilla, zonas comunes)
 - Economía y responsabilidad (pagos, gastos compartidos, consumo).
 - Ruido y horarios (respeto de las horas de descanso).
 - Visitas (frecuencia, uso del espacio).
 - Comunicación (reacción ante el conflicto, empatía, comunicación)
 - Respeto de la propiedad
- Comentario general.

En caso de viviendas:

- Tiempo de estancia.
- Puntuación sobre 5 estrellas de cada aspecto de la convivencia:
 - Veracidad respecto al anuncio (correspondencia con el anuncio, condiciones anunciadas)
 - Estado y mantenimiento (estado general, limpieza inicial, ausencia de desperfectos).
 - Equipamiento (electrodomésticos, mobiliario, calefacción).
 - Relación calidad-precio (precio adecuado, proporción de gastos).
- Comentario general

4.3.4 Gestión de conexiones entre usuarios y viviendas

- **RF-17 Búsqueda:** El usuario debe poder buscar tanto viviendas como a otros usuarios a través de un buscador de la aplicación.
- **RF-18 Chat entre usuarios:** El usuario debe poder escribir por mensaje a otros usuarios.
- **RF-19 Conexión entre usuarios que hayan convivido:** El usuario debe poder establecer conexión con otros usuarios con los que haya mantenido una relación de convivencia.

4.3.5 Gestión de recomendaciones

- **RF-20 Recopilación de datos:** el sistema podrá, siempre que se lo permita el usuario, recopilar información de este en función a sus visitas recientes y comportamientos dentro de la aplicación, con el objetivo de recomendar aquellos anuncios o usuarios que mejor se ajusten a sus preferencias.
- **RF-21 Sistema de recomendaciones:** El sistema debe recomendar inquilinos, compañeros o viviendas ajustadas a las preferencias del usuario y a las visitas y búsquedas realizadas recientemente.

4.4 Requisitos no funcionales

- **RNF-01 Multiplataforma:** El sistema debe funcionar tanto en ordenador como en dispositivos móviles y tablet.

- **RNF-02 Control de anuncios:** El sistema debe controlar todos los anuncios de la aplicación y asegurar que se cumplan las normas impuestas, de manera que se asegure la fiabilidad y transparencia de los mismos.
- **RNF-03 Privacidad de datos:** La aplicación deberá tratar la información sensible de sus usuarios adecuadamente, asegurando la privacidad de la misma mediante diferentes técnicas de programación seguras.
- **RNF-04 Privacidad de la información de la cuenta:** La visualización de información privada del usuario, como su estancia en un piso como inquilino, estará sujeta al permiso de este.
- **RNF-05 Integridad de las valoraciones:** El sistema debe usar el tiempo de estancia como contexto para la fiabilidad de la valoración.

4.5 Casos de uso

En este apartado se van a definir los casos de uso más relevantes del sistema desarrollado, siendo estos los que mejor definan su funcionamiento general.

Para este punto se han omitido funcionalidades básicas como la gestión de acceso debido a su estandarización, aunque sí han sido tenidas en consideración en el Backlog de desarrollo.

Caso de Uso	Gestión de Perfil y Estado				CU1
Actores	Usuario				
Tipo	Esencial				
Referencias	RF-03, RF-04, RF-05, RF-06, RF-07		HU-2, HU-4, HU-5, HU-6, HU-7, HU-22		
Precondición	El usuario cuyos datos van a ser gestionados debe estar registrado en el sistema				
Postcondición	Los datos del usuario quedarán registrados con los valores introducidos.				
Autor	Carlos Hoyo	Fecha	10/03/2026	Versión	1

Propósito
El usuario en cuestión quiere gestionar su cuenta, cambiando cualquiera de sus datos de perfil, estado de búsqueda u hospedaje.

Resumen
El usuario registrado hará uso de la posibilidad de editar su perfil y modificará los datos que quiera, cambiando así sus datos de perfil, estado de búsqueda o estado de hospedaje.

Curso normal (básico)			
1	El usuario entrará al área de personalización de la cuenta		
2	El usuario modificará los datos que considere		
3	El usuario guardará los datos una vez están a su gusto, los cuales se almacenarán de esa manera en el sistema.	4	El sistema modificará los datos del usuario con los nuevos datos introducidos.

Cursos alternos	
3a	El usuario trata de guardar unos datos de la cuenta no válidos (no siguen los formatos, campos obligatorios en blanco, etc.), por lo que no le dejará avanzar y volverá al paso 2.
4b	El sistema alertará al usuario de que los datos introducidos no son válidos obligará al usuario a modificarlos o abortar el proceso de edición de los datos, volviendo al paso 2.

Otros datos			
Frecuencia esperada	Bastante frecuente	Rendimiento	Alto
Importancia	Alta	Urgencia	Media
Estado	Pendiente	Estabilidad	Alta

Comentarios

Este caso de uso será llevado a cabo por de manera frecuente por usuarios que quieran modificar los datos de su cuenta. Será especialmente utilizado en períodos en los que los usuarios concretos se encuentren en búsqueda de alguno de los servicios ofrecidos por el sistema, con el fin de actualizar sus datos.

Caso de Uso	Publicación y verificación de anuncios				CU2
Actores	Usuario Casero, Usuario Administrador, Sistema				
Tipo	Esencial				
Referencias	RF-08, RF-09, RF-13, RF-14, RNF-02		HU-8, HU-9, HU-13, HU-25		
Precondición	El usuario casero debe estar registrado en el sistema con el rol de casero.				
Postcondición	El anuncio quedará registrado en el sistema y disponible para la vista de todos los usuarios interesados.				
Autor	Carlos Hoyo	Fecha	12/03/2026	Versión	1

Propósito

La publicación y edición de anuncios de viviendas por parte de un usuario con rol de casero en búsqueda de inquilinos.

Resumen

El casero realizará el proceso de publicación de un anuncio de vivienda, que será comprobado por un administrador, el cual será encargado de dar el visto bueno, o no, al anuncio.

Curso normal (básico)

1	El usuario casero entra al panel de publicación de anuncio.		
2	El usuario casero rellena toda la información requerida sobre el inmueble		
3	El usuario casero publica el anuncio	4	El sistema deja el anuncio en estado de pendiente, sin publicar aún, a la espera de la revisión y visto bueno del usuario administrador
5	El administrador da el visto bueno al anuncio y este es publicado finalmente	6	El sistema almacena el anuncio y lo muestra al resto de usuarios.

Cursos alternos

2a	Los datos introducidos por el casero no son válidos, por lo que el sistema alerta de ello y no lo publica, obligándole a rellenar todos los datos correctamente, volviendo al paso 2.
5b	El administrador interpreta que el anuncio no cumple las normas del sistema, por lo que deniega la publicación del anuncio, notificando al casero y volviendo al paso 1.

Otros datos

Frecuencia esperada	Frecuente	Rendimiento	Alto
Importancia	Muy alta	Urgencia	Muy alta
Estado	En curso	Estabilidad	Alta

Comentarios

Para que el anuncio del usuario con rol de casero sea publicado, este tendrá que cumplir las normas establecidas por el sistema, de lo contrario, se procederá a la denegación y la no publicación del anuncio de la vivienda en el sistema.

Caso de Uso	Sistema de valoraciones		CU3
Actores	Usuario del sistema (cualquier rol), objetivo a valorar (otro usuario o inmueble)		
Tipo	Esencial		
Referencias	RF-15, RF-16	HU-14, HU-16, HU-23, HU-26	
Precondición	El usuario debe de haber tenido una relación de convivencia con la entidad a valorar (convivido con/convivido en u hospedado en su inmueble)		

Postcondición	Se registrará una valoración del usuario a la entidad en cuestión, la cual será visible para el resto de los usuarios del sistema.				
Autor	Carlos Hoyo	Fecha	10/03/2026	Versión	1

Propósito

Un usuario que haya tenido una relación de convivencia con otra entidad (casero, inquilino o inmueble) podrá valorar sobre 5, en cada uno de los apartados especificados para cada una de las entidades, los aspectos más relevantes de la convivencia.

Resumen

Un usuario que ha mantenido una relación de convivencia con otra entidad del sistema realiza una valoración de los aspectos puntuables por la aplicación, con el objetivo de dar información relevante a futuros usuarios del sistema.

Curso normal (básico)

1	Un usuario mantiene una relación de convivencia con otra entidad del sistema		
2	Al finalizarse la misma, se abre la posibilidad de valorar a la otra parte	3	El sistema ofrece una serie de aspectos puntuables sobre 5 sobre la convivencia completada
4	El usuario guarda la valoración una vez completada	5	El sistema realiza una comprobación rápida antes de publicar la valoración del usuario

Cursos alternos

5a	El sistema detecta que la valoración incumple las normas de la aplicación (generalmente por el uso de vocabulario indebido en la parte del comentario) y deniega su publicación, obligando al usuario a volver al paso 3.
-----------	---

Otros datos

Frecuencia esperada	Media	Rendimiento	Alto
Importancia	Muy alta	Urgencia	Muy alta
Estado	Pendiente	Estabilidad	Alta

Comentarios

Las valoraciones realizadas por usuarios deberán cumplir las normas establecidas en la aplicación.

Caso de Uso	Vinculación Inquilino-Vivienda				CU4
Actores	Inquilino, Casero				
Tipo	Esencial				
Referencias	RF-10, RF-11, RF-12		HU-10, HU-11, HU-12		
Precondición	Tanto el inquilino, como el casero y la vivienda deben estar correctamente registrados en el sistema y con posibilidades de convivencia entre ellos (inquilino sin vivienda asociada y vivienda con huecos libres).				
Postcondición	El inquilino quedará vinculado a la vivienda mediante una relación de convivencia				
Autor	Carlos Hoyo	Fecha	10/03/2026	Versión	1

Propósito

La vinculación oficial entre un inquilino y una vivienda al entrar el usuario a vivir en ella.

Resumen

Cuando se llegue a un acuerdo por parte del inquilino y el casero, este último mandará una solicitud de registro en la vivienda al inquilino, de manera que, cuando sea aceptada, el inquilino y la vivienda pasarán a estar vinculados mediante una relación de convivencia inquilino-inmueble.

Curso normal (básico)			
1	El inquilino en búsqueda de vivienda y el casero entrarán en contacto por medio de la aplicación.		
2	Una vez alcanzado un acuerdo, el casero mandará una solicitud de registro en la vivienda al inquilino.		
3	El inquilino aceptará dicha solicitud, quedando así registrada la relación en el sistema	4	El sistema almacenará la relación entre el inquilino y la vivienda

Cursos alternos	
3a	El inquilino rechaza la solicitud de registro en la vivienda, abortando así el proceso de registro.

Otros datos			
Frecuencia esperada	Muy frecuente	Rendimiento	Alto
Importancia	Muy alta	Urgencia	Muy alta
Estado	Pendiente	Estabilidad	Alta

Comentarios	
El registro del inquilino en la vivienda no se oficializará hasta que el casero le mande una solicitud de registro y este la acepte.	

Caso de Uso	Motor de Búsqueda y Recomendaciones				CU5
Actores	Usuario, Sistema				
Tipo	Esencial				
Referencias	RF-17, RF-20, RF-21		HU-18, HU-24		
Precondición	El usuario debe de haber habilitado el uso de sus preferencias para la recomendación de otros usuarios o viviendas				
Postcondición	El sistema usará los datos del usuario para sugerirle ofertas o usuarios que le puedan interesar basado en sus referencias e interacciones recientes en la aplicación				
Autor	Carlos Hoyo	Fecha	10/03/2026	Versión	1

Propósito	
La recomendación personalizada para todos los usuarios que hayan habilitado el uso de sus datos para ese fin.	

Resumen	
Cuando el usuario interactúe con la aplicación, esta lo tendrá en cuenta y recopilará datos importantes para futuras recomendaciones que le puedan servir de ayuda.	

Curso normal (básico)			
1	El usuario interactúa con la aplicación (buscando usuarios o viviendas, estableciendo relaciones de convivencia, etc.)	2	El sistema recopilará estas interacciones, ofreciendo usuarios o viviendas de posible interés para este.

Cursos alternos	
1a	El usuario no permite que la aplicación haga uso de sus datos e información reciente, impidiendo así la realización de este caso de uso.

Otros datos			
Frecuencia esperada	Muy frecuente	Rendimiento	Alto

Importancia	Alta	Urgencia	Alta
Estado	Pendiente	Estabilidad	Alta

Comentarios
El sistema deberá de predecir la información más relevante para el usuario en todo momento, siempre que este lo haya permitido.

Para visualizar mejor la composición de actores del sistema y la interacción que realizan con los diferentes casos de uso planteados, se puede ver el diagrama de la Ilustración 6.

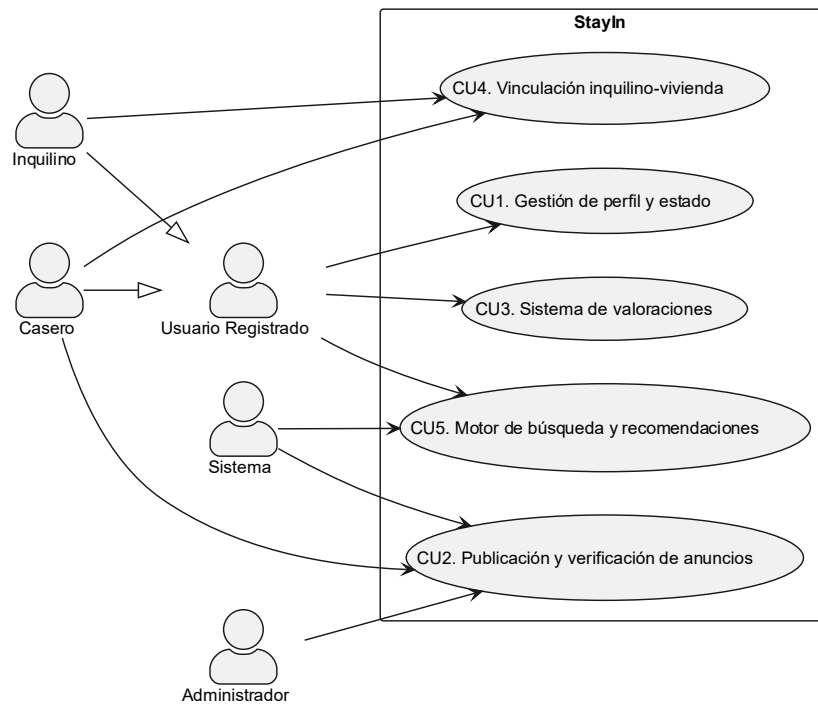


Ilustración 6. Diagrama de actores y casos de uso del sistema

5. DISEÑO DEL SISTEMA

En este capítulo se va a abordar cómo se van a llevar a cabo cada uno de los conceptos de este proyecto, definiendo una estructura sólida; lo que facilitará la posterior implementación, haciendo el proceso mucho más estructurado y tolerante a fallos.

5.1 Arquitectura del sistema

En este apartado se va a describir, a partir del diagrama de la Ilustración 7, los principales módulos y microservicios que representan el funcionamiento de la aplicación, reflejando las tecnologías usadas y la comunicación entre ellos.

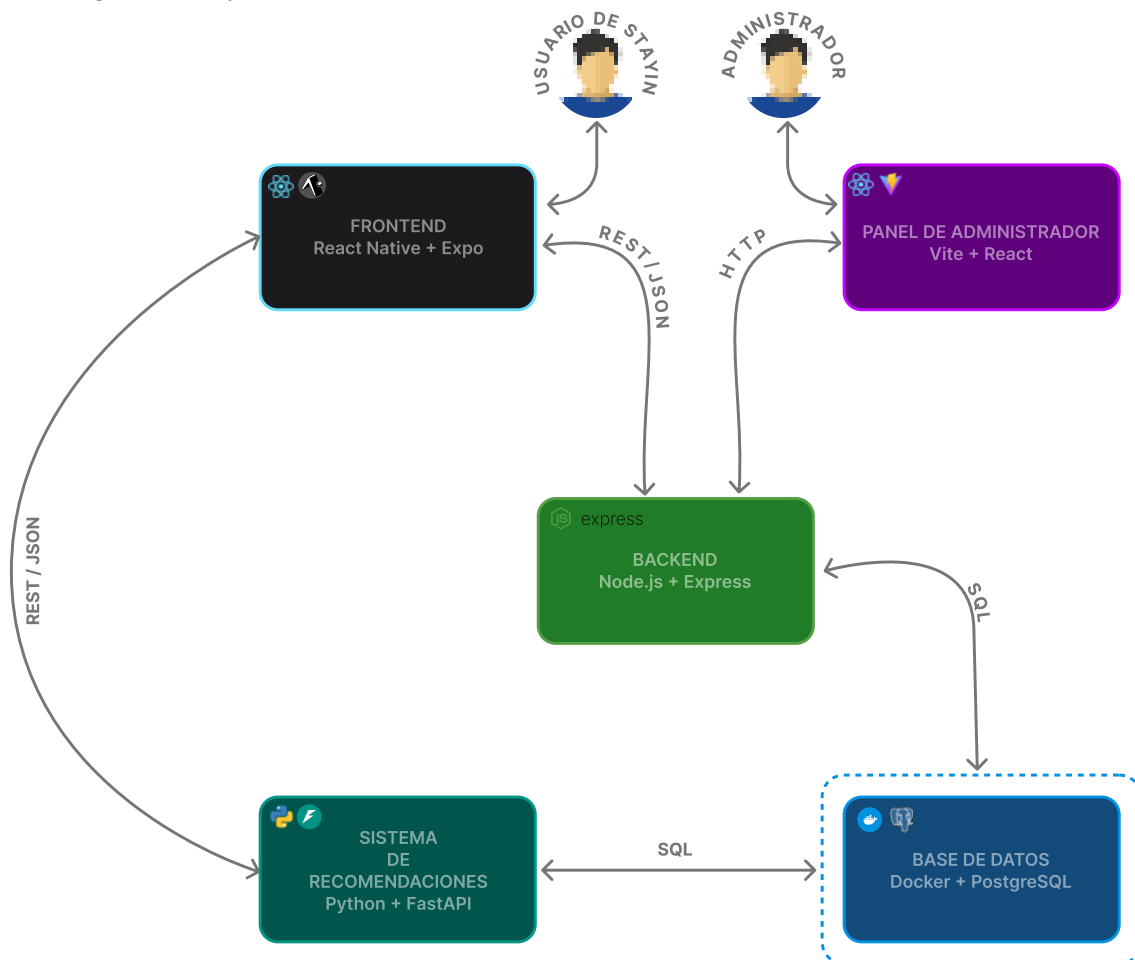


Ilustración 7. Diagrama de la arquitectura del sistema

Como se puede observar en el diagrama, la arquitectura del sistema está compuesta por un total de 5 módulos diferentes que se comunican entre sí mediante distintos protocolos.

En primer lugar, como módulo central, encontramos el servidor *backend* de la aplicación, desarrollado en Node.js y Express, actuando como puente entre la interfaz de usuario en el *frontend* y la base de datos, permitiendo al usuario interactuar realmente con el sistema y haciendo uso de los datos reales proporcionados por este.

Seguidamente, como ya se ha mencionado, está el *frontend*, desarrollado mediante React Native y Expo, ofreciendo una interfaz con unas funcionalidades modernas y coherentes a la exigencia necesaria para su funcionamiento. Como se ve en el diagrama, se comunica con el *backend* y con el sistema de recomendaciones implementado en FastAPI, mediante una APIRest a través de mensajes en formato JSON. A través de estos mensajes el usuario puede hacer indirectamente (ya que están definidas en el código) consultas a la base de datos, obteniendo, añadiendo o modificando datos de esta, a través de uno de los dos módulos con los que se comunica, dependiendo del caso.

A continuación, encontramos el panel de administrador, un servicio web aparte del *frontend* de la aplicación, cuya finalidad es facilitar la tarea de administración y gestión del sistema, permitiendo a los usuarios autorizados realizar operaciones directamente sobre la base de datos, tales como añadir o eliminar usuarios y anuncios u ocultar valoraciones. Se comunica, de nuevo, con el *backend* de la aplicación para poder llevar a cabo de manera correcta todas las tareas mencionadas, aunque esta vez el protocolo de comunicación es mediante HTTP. Las rutas están protegidas mediante un doble middleware para evitar el acceso no autorizado a una sección tan crítica del sistema de StayIn. Este aspecto se explica con mayor lujo de detalle en el capítulo 6.

Llegando a los dos últimos módulos, tenemos en primer lugar la base de datos en sí, implementada mediante PostgreSQL y contenerizada a través de Docker, permitiendo la comunicación con el *backend* y el microservicio de sistema de recomendaciones y el tratamiento en tiempo real de los datos almacenados en ella.

Con esto, llegamos al microservicio donde se implementa el sistema de recomendaciones, desarrollado en Python a través de FastAPI. Este microservicio se comunica, de nuevo, con la base de datos del sistema, a través de sentencias SQL, de manera que se puedan obtener los datos almacenados necesarios para realizar el proceso del cálculo de recomendaciones de usuarios y anuncios registrados. Se comunica, además, con el *frontend* de la aplicación, precisamente para pasarle esas recomendaciones en tiempo real a través de los datos conseguidos de la base.

Debido al carácter de consulta que tienen las comunicaciones del microservicio de recomendaciones con la base de datos, no hay ningún problema de desincronización entre el *backend* y este módulo, ya que todos los datos que el servidor principal modifique aparecerán reflejados en las consultas realizadas desde FastAPI.

5.2 Diseño de datos

Una vez definida la arquitectura del sistema, pasamos al diseño de los datos que conformarán la base de datos alojada en el contenedor de Docker. Para ello, se ha realizado un trabajo de estudio sobre qué clases principales deben conformar el sistema, alojando todos los datos necesarios y garantizando la eficiencia, la consistencia y la seguridad de los datos de StayIn.

En el diagrama de tablas mostrado en la Ilustración 8 se pueden observar las diferentes tablas que conforman la base de datos de la aplicación, junto con sus respectivas relaciones.

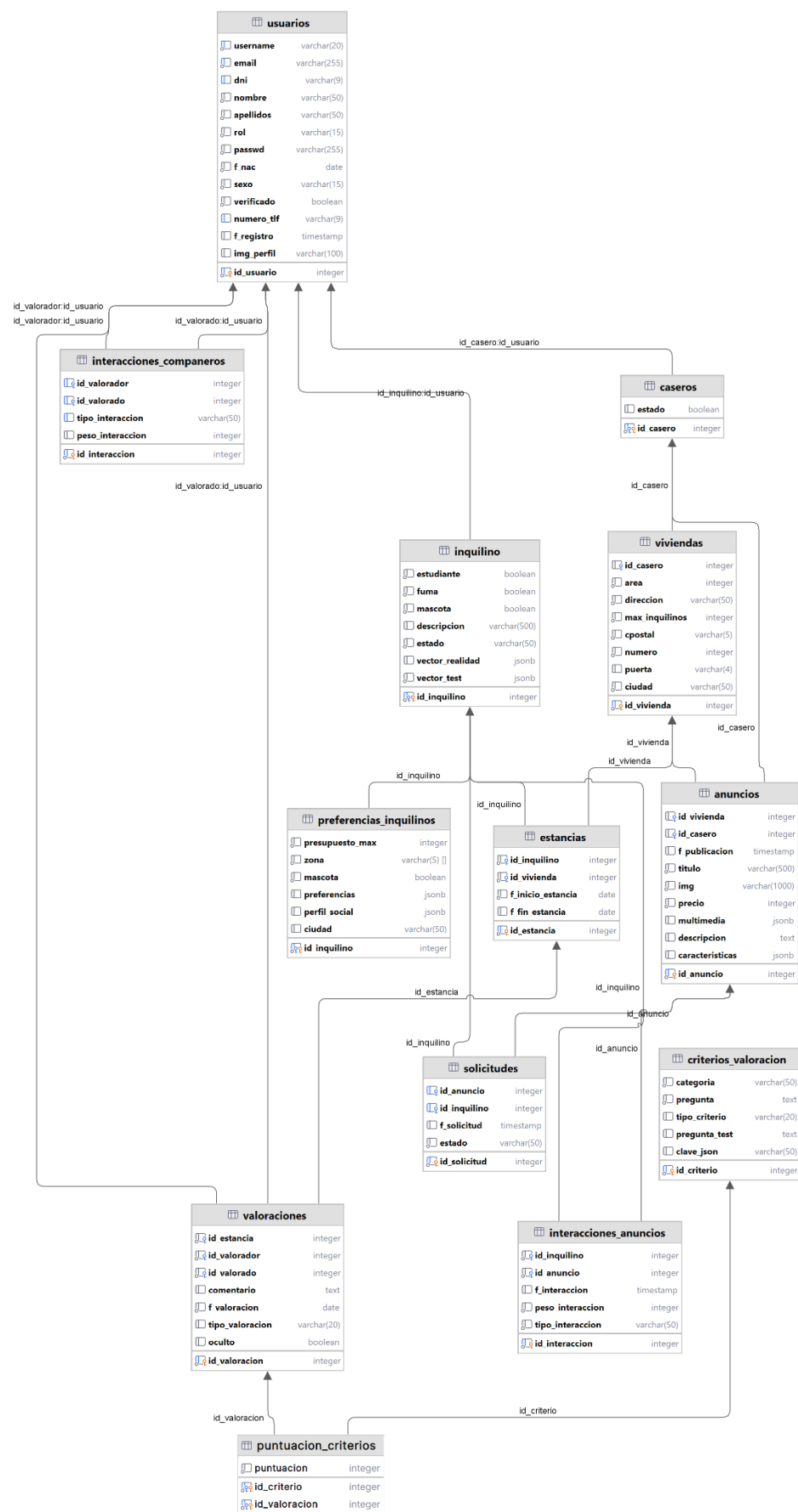


Ilustración 8. Diagrama de tablas de la base de datos
Fuente: diagrama exportado de BD real de DataGrip [106]

En este diagrama se pueden ver un total de 13 tablas que conforman la base de datos de la aplicación. Todas estas clases se han dividido por grupos de la forma que se detalla a continuación.

5.2.1 Tablas esenciales

Se han incluido en este grupo las tablas fundamentales que sustentan el resto del funcionamiento del sistema, que son las primeras en las que se piensa al definir la idea del proyecto.

Dentro de este grupo tenemos las siguientes tablas.

- Usuarios

La tabla que engloba a todos los usuarios del sistema, cuya declaración se puede encontrar en el Código 13. Esta tabla es la encargada de definir a todos los usuarios del sistema y almacena, por tanto, todos los datos comunes a ellos, independientemente del tipo que sean, como se puede observar en el diagrama.

Esta es una de las tablas consideradas como esenciales, puesto que es de la que parten la mayoría de las tablas del sistema y la necesaria en prácticamente todas las consultas a la base de datos.

- Viviendas

Esta tabla (Código 16), junto con la anterior y en combinación con la siguiente, definen el núcleo central de interacciones del sistema, almacenando todo lo relativo a los inmuebles ofrecidos en la aplicación. Se encarga de almacenar los datos relativos exclusivamente al inmueble en sí, tales como la dirección, el área, etc.; y no a los datos ofrecidos por un usuario propietario, ya que esos no tienen que ver con los datos reales del inmueble. Por estos motivos, se ha decidido realizar una separación en dos tablas, como se mostrará en la siguiente.

- Anuncios

El complemento de la tabla anterior. Debido a los datos necesarios para el almacenamiento de un anuncio, carecía de sentido implementar una única tabla con todos los datos almacenados en ella, por lo que se ha considerado hacer una división entre lo que serían los datos físicos de un anuncio (dirección, área, número máximo de inquilinos, etc.) en la tabla anterior, y lo que serían los datos lógicos, aquellos que no tienen presencia en el mundo físico, como pueden ser el precio, la descripción, etc. Estas dos tablas (Viviendas y Anuncios) en combinación, representan todos los inmuebles ofertados en StayIn, con sus datos y metadatos asociados; definiendo, junto con la tabla Usuarios, las bases esenciales para comenzar a definir el desarrollo del sistema. Su implementación se puede encontrar en el Código 17.

5.2.2 Tablas hijas

Se ha definido en este grupo a las dos tablas que bifurcan el funcionamiento de la aplicación para cada usuario, dependiendo del rol escogido durante su registro.

- Inquilinos

Es la tabla donde se alojarán todos los usuarios cuyo objetivo sea el de encontrar compañeros para convivir, vivienda, o ambas cosas a la vez. Esta tabla hereda todos los datos de la tabla usuarios y añade los específicos de aquellos usuarios con rol de inquilino. Su implementación se puede encontrar en el Código 14.

- Caseros

Al igual que la anterior, la tabla Caseros (Código 15) hereda todos los datos de Usuarios, especializando el caso para aquellos usuarios que quieran publicar anuncios de sus inmuebles con el objetivo de encontrar inquilinos. La separación de roles da lugar a una personalización de las pantallas de la aplicación, mostrando la información relevante para cada usuario.

- Aclaración sobre administrador

Debido a la ausencia de datos relevantes para el administrador, a priori, más allá de los definidos en la tabla Usuarios, se ha hecho uso de la columna definida en esta tabla (tipo) para identificar al usuario con rol de administrador, implementando una consigna de tipo *check* para comprobar que el tipo de usuario no puede ser otro que Administrador, Inquilino o Casero. De esta forma, se realiza una correcta división de usuarios y, por consiguiente, de sus permisos de acceso y funcionalidades habilitadas.

5.2.3 Tablas específicas

En este último grupo de tablas se han incluido todas aquellas que no representan una entidad básica o esencial del sistema, sino que se focalizan más en permitir la funcionalidad de distintos aspectos fundamentales de la aplicación. A continuación, se detallan cada uno de estos casos.

- Interacciones

Se han incluido en este apartado dos clases cuya funcionalidad es, en esencia, la misma, con la diferencia en la orientación de cada una. Por un lado, tenemos la tabla Interacciones_Companeros (Código 23) y, por otro, la tabla Interacciones_Anuncios (Código 24), cuyas funcionalidades, como se ha mencionado, son muy similares. Estas dos clases son las encargadas de almacenar las interacciones de los usuarios con la aplicación, en concreto, con otros usuarios y con anuncios, como sus propios nombres indican. La finalidad de ambas es la monitorización de la interacción del usuario con la aplicación con el objetivo de alimentar el modelo de Machine Learning preparado para mejorar el sistema de recomendaciones de StayIn, como se detallará, más adelante, a lo largo del apartado 6.5 de este documento.

- Valoraciones

De la misma manera que en el anterior punto descrito, se han incluido varias tablas del diagrama para explicar el funcionamiento de las valoraciones implementadas en este proyecto. Las tablas en cuestión que componen este funcionamiento son: Valoraciones (Código 20), Criterios_Valoracion (Código 21) y Puntuacion_Criterios (Código 22), cada una con una funcionalidad clara y definida para dar sustento al sistema de valoraciones de la aplicación, detallado en la sección 6.4 de esta memoria.

La primera de las tres tablas mencionadas se encarga de mostrar los datos principales de una valoración: quién la realiza, a quién valora, cuándo lo hace, el comentario opcional, el tipo (si es a otro usuario o a un inmueble) y el estado, si se muestra al resto de usuarios o si ha sido ocultada por algún administrador del sistema por haber incumplido alguna de las normas.

En el caso de la segunda, se encarga de almacenar los criterios de la valoración a realizar. Esto es, dependiendo del tipo, mostrará las categorías de preguntas y las preguntas de los criterios de valoraciones a usuarios o a viviendas, definidos en el apartado 4.2.4, de este documento.

Por último, la tabla restante se encarga de almacenar las puntuaciones de cada pregunta, almacenando el id del criterio valorado (heredado de Puntuacion_Criterios) y el id de la valoración (heredado de Valoraciones), de manera que se puedan acceder a las puntuaciones realizadas sobre las valoraciones de un usuario concreto.

- Preferencias de inquilinos

En este caso, la tabla Preferencias_Inquilinos (Código 25), se encarga de almacenar todos los datos que un usuario marca en sus preferencias de búsqueda a través de la aplicación. Su funcionalidad es, esencialmente, permitir al sistema de recomendaciones funcionar correctamente mediante el cruce entre las preferencias del usuario y las características ofrecidas por los anuncios publicados en el sistema, como se puede ver con todo lujo de detalle en el apartado 6.5.

- Estancias

Debido al funcionamiento de la aplicación y a sus características, es necesario almacenar un registro de las estancias realizadas por los diferentes usuarios registrados en el sistema. Esto es, básicamente, llevar un registro de qué usuarios se han convivido en una vivienda y durante qué período de tiempo, permitiendo así presentar la posibilidad de valorar tanto a los usuarios con los que se ha mantenido la convivencia, como al inmueble donde esta ha tenido lugar.

Sin esta tabla, el sistema de valoraciones no funcionaría, pues los usuarios nunca tendrían la posibilidad de poder realizar la valoración. Su implementación se puede encontrar en el Código 18.

- Solicitudes

Por último, la tabla Solicitudes (Código 19) almacena las solicitudes de registro en vivienda realizadas por los usuarios interesados en un anuncio, las cuales serán aceptadas o rechazadas por el usuario registrado como casero de este. Como se puede ver, esta tabla es clave para poder realizar el registro de usuarios en viviendas a través de la aplicación.

5.2.4 Integración con el sistema

Una vez definidas todas las tablas componentes de la base de datos de StayIn, es trabajo de los diferentes módulos y servicios definidos anteriormente en la Arquitectura del sistema (5.1) usarlas para conseguir el funcionamiento esperado de la aplicación.

5.3 Navegabilidad entre pantallas

Con las tablas de la base de datos definidas, en este apartado, mediante la Ilustración 9 se muestra un diagrama del flujo de movimiento entre las distintas pantallas al usar la aplicación, reflejando su estructura y navegabilidad.

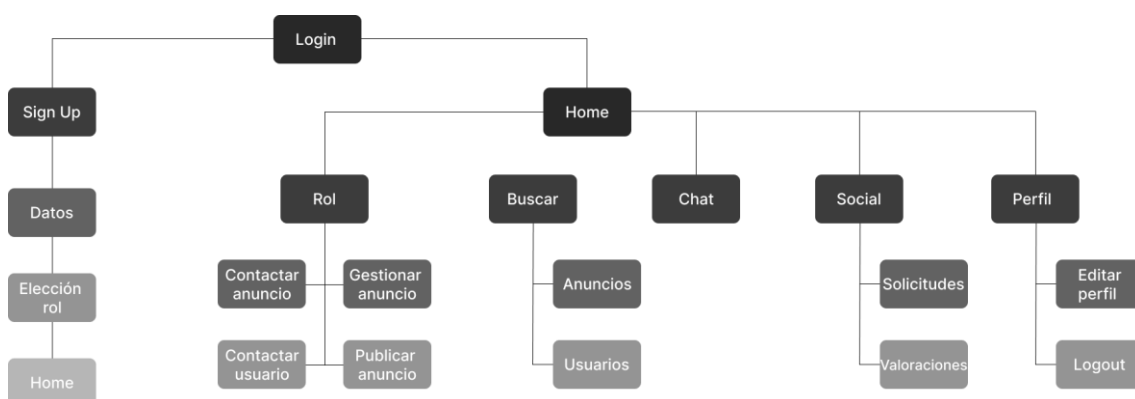


Ilustración 9. Sitemap de la aplicación

En este diagrama se muestra, de manera resumida, las diferentes opciones de navegabilidad desde la pantalla de inicio de sesión o registro en la aplicación. A continuación, se describe el funcionamiento descrito en el diagrama.

5.3.1 Flujo Sign Up

Este sería el flujo de registrarse como usuario nuevo en la aplicación. En este proceso, se pedirían los datos esenciales de la cuenta, tratando de exigir lo mínimo posible con el objetivo de conseguir que el usuario se registre.

Los datos para introducir en este momento son: nombre, apellidos, correo electrónico, nombre de usuario, ciudad, fecha de nacimiento y género.

Tras añadir todos los datos, se pasa a la elección del rol y, posteriormente, al *home* del usuario según el rol seleccionado.

5.3.2 Flujo Home

Una vez que un usuario ya tiene una cuenta registrada, puede iniciar sesión en el sistema, pasando a la pantalla principal del usuario. En este caso, hay varias posibilidades en función del rol de la cuenta.

- Rol inquilino

El *home* del inquilino, como puede observar en los mockups anteriores y se verá en el siguiente capítulo, está formado por dos componentes principales.

En primer lugar, se muestra un único anuncio, con la posibilidad de ampliar la lista para ver todos los recomendados. Debajo del anuncio sugerido, se muestra un carrusel de tres usuarios, de manera que, deslizando los perfiles hacia la izquierda, podemos ver los demás usuarios.

Todos estos anuncios y perfiles son sugerencias del motor de recomendaciones de la aplicación, en caso de tener datos para ofrecer sugerencias. Si no, simplemente se muestran anuncios de la ciudad indicada en el perfil a la hora de registrarse.

- Rol casero

El *home* del casero es ligeramente distinto al del inquilino, aunque de apariencia no lo es tanto, debido a que se muestran los anuncios registrados que tenga el casero en la aplicación, mediante los mismos componentes para las tarjetas de anuncios, aunque adaptados a caseros, permitiendo la gestión y edición de estos.

5.3.3 Flujo buscar

En caso de requerir una búsqueda más personalizada, especialmente para aquellos casos particulares, en los que el usuario quiere buscar algo fuera de lo que suele (otra ciudad, otros precios, etc.), podrá acceder a la pantalla de búsqueda, donde tendrá la opción de buscar por anuncios o por usuarios.

De esta forma, se presentan dos pantallas y dos formas distintas de encontrar información: mediante las recomendaciones de la pantalla *Home* del usuario, dependiendo de su rol, o a través de un buscador mediante filtros y parámetros para poder buscar en la aplicación al completo, fuera de recomendaciones personalizadas por el sistema, en caso de requerir algo puntual o concreto.

5.3.4 Flujo chat

Esta pantalla permite, lógicamente, el uso de un chat dentro de la aplicación, de manera que los usuarios puedan comunicarse fácilmente sin necesidad de compartir ningún otro tipo de dato con la persona en cuestión, facilitando así los acuerdos de estancias entre usuarios registrados.

El objetivo de esta pantalla es proveer al usuario de todas las necesidades requeridas a la hora de encontrar vivienda o compañeros, complementando el resto aspectos, como son las recomendaciones y las publicaciones de anuncios.

5.3.5 Flujo social

Esta será la pantalla responsable de dos funcionalidades primordiales en la aplicación: la aceptación de solicitudes de registro, por parte de los usuarios con rol de casero y la notificación de las valoraciones pendientes a usuarios y viviendas, tras haber completado una estancia.

A través de ella se recibirán las notificaciones cuando se produzca alguno de los dos eventos, dando la opción de aceptar o rechazar las solicitudes de registro y el acceso a la pantalla de valoraciones a usuario o vivienda, dependiendo del caso.

5.3.6 Flujo perfil

En esta última pantalla, se podrán realizar todas las modificaciones del perfil del usuario, desde su foto de perfil hasta las configuraciones de su cuenta, pasando por la alteración de todas las

preferencias de búsqueda para personalizar al máximo su perfil, de forma que se le puedan realizar recomendaciones adecuadas según sus datos.

Además, también se podrá realizar el cierre de sesión a través de esta pantalla, redirigiendo al usuario a la pantalla de *login*.

5.4 Diseño de interfaz UI/UX

La aplicación cuenta con un diseño previo a la implementación, de manera que esta sea más estructurada y definida, fomentando además la experiencia de usuario.

5.4.1 Logo de la aplicación

El logo de la aplicación surge junto con el nombre de esta, StayIn, donde se realiza un juego de palabras entre los términos ingleses “stay” (estancia) y “staying” (quedarse o quedándose), usando la letra “i” del nombre como bloque de pisos, con relación al objetivo principal del proyecto. Hay varias versiones del logo con distintas configuraciones de colores.

A continuación, se pueden ver los tres estilos de logo planteados, quedando el logo mostrado en la Ilustración 10 como el escogido para el logo oficial de la aplicación, mientras que el gris y el blanco, Ilustración 11 e Ilustración 12, respectivamente, quedan para otros posibles usos en combinación con diferentes colores.



Ilustración 10. Logo StayIn Negro



Ilustración 11. Logo StayIn Gris



Ilustración 12. Logo StayIn Blanco

Tabla 42. Logos de StayIn

5.4.2 Mockups y prototipos

Para la interfaz de la aplicación, se cuenta con los diseños las páginas más importantes, contando todas con una versión en modo claro y modo oscuro, que se ajustará al tema del dispositivo en cada momento. No obstante, en este documento solo se van a usar los diseños realizados para el modo claro, con el fin de no repetir pantallas y mantener el documento lo más limpio posible. Esta es una versión inicial de los mockups realizados antes de la implementación real de las pantallas. Para el resto de estas, simplemente se ha seguido la estructura de colores y se ha hecho uso de la reutilización de componentes previamente creados para otras pantallas, tratando de reciclar el mayor código posible.

Stayn



ENVIAR



Ilustración 13. Mockup Home de inquilino

Ilustración 14. Mockup pantalla de búsqueda

Viviendas que te podrían interesar



Piso en Gran Vía, 13
Calle Gran Vía, 13, Granada

350€/mes



Compañeros con lo que podrías conectar



Carlos Hoyo
Granada, España

Conectar



Carlos Hoyo
Granada, España

Conectar



Ilustración 15. Mockup Home inquilino

Mis anuncios



Piso en Gran Vía, 13
Calle Gran Vía, 13, Granada

350€/mes



Piso en Gran Vía, 13
Calle Gran Vía, 13, Granada

350€/mes



Nuevo anuncio



Ilustración 16. Mockup Home casero

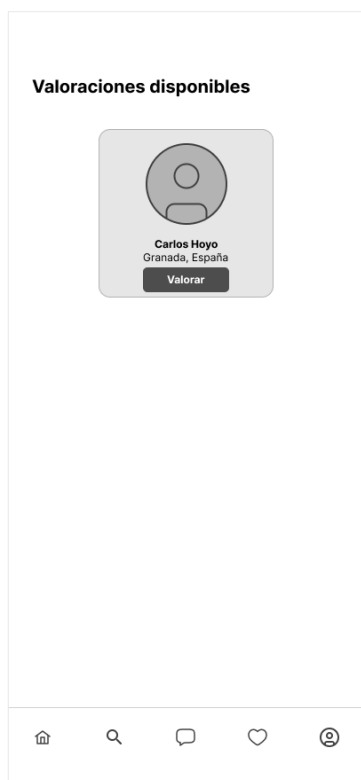


Ilustración 17. Mockup valoraciones pendientes



Ilustración 18. Mockup pantalla valoraciones

Elige los archivos para cada habitacion

Salón

+

Cocina

+

Dormitorio

+

Baño

+

+

ENVIAR











Ilustración 19. Mockup añadir multimedia a anuncio

Tabla 43. Mockups de StayIn

6. IMPLEMENTACIÓN Y DESARROLLO

En este capítulo se va a tratar la implementación de todo el código del proyecto, los problemas que han surgido y cómo se han planteado y resuelto sus respectivas soluciones.

Se ha realizado una implementación y una documentación separados por módulos de desarrollo en relación con la planificación y *sprints* definidos en el capítulo 3.

A continuación, se mostrarán los detalles más relevantes de cada aspecto, comenzando por la implementación de la gestión de usuarios en el sistema, aspecto clave sin el cual no se puede comenzar realmente a avanzar en la programación de la aplicación.

6.1 Gestión integral de perfil y estados (Sprint 1)

En esta primera parte de la implementación se han llevado a cabo todas las tareas correspondientes al registro, inicio de sesión y edición del perfil del usuario, así como la elección de los diferentes roles y estados posibles.

6.1.1 Lógica de base de datos

Para abordar esta primera parte del desarrollo, se han creado las tablas Usuarios, Inquilinos y Caseros, que se encargarán de almacenar los datos de todos los usuarios del sistema para que puedan autenticarse de manera segura.

Un aspecto muy importante de cara al diseño de estas tablas es marcar como no nulos únicamente aquellos valores estrictamente esenciales para el correcto funcionamiento de la aplicación, dejando otros campos que serán necesarios para hacer uso de todas las funcionalidades disponibles en el sistema para más adelante, de manera que el proceso de registro en la aplicación no sea demasiado abrumador, lo que podría dificultar la llegada de nuevos usuarios.

6.1.2 Implementación en el *backend*

Para poder realizar las tareas definidas se han implementado los *endpoints* */api/register*, */api/login* y */api/edit/profile*. En cada uno de ellos se han obtenido los datos del *frontend* y se han realizado consultas SQL a la base de datos para almacenar u obtener los datos correspondientes en cada caso.

- Registro

En el caso del registro de un usuario, se obtienen del *frontend* el nombre de usuario, correo electrónico, nombre, apellidos, rol seleccionado, contraseña, fecha de nacimiento y sexo. La contraseña es encriptada mediante el uso de la función *hash* del módulo *bcrypt*.

La consulta para el registro de un usuario en el sistema se realiza mediante la inserción de los valores obtenidos del *frontend* en la tabla Usuarios.

Una vez insertado el usuario, se realiza la inserción en la tabla correspondiente según el rol que haya escogido. Todo este bloque en el que se realizan varias inserciones para una misma funcionalidad debe ir protegido mediante las sentencias *BEGIN*, *COMMIT* y *ROLLBACK*, de forma que se realicen o todas las inserciones o ninguna, garantizando la consistencia de los datos y evitando registros huérfanos en la base de datos.

- Inicio de sesión

Para realizar el inicio de sesión en el sistema, se obtienen del *frontend* el nombre de usuario, email o número de teléfono y la contraseña y se comprueba la existencia de un usuario con esos datos en el sistema. De esta manera, se permite el inicio de sesión y el acceso a la cuenta en la aplicación mediante esos 3 métodos. El inicio de sesión de los usuarios está protegido mediante el uso de tokens web, conocidos como JSON Web Tokens (JWT). De esta manera, cuando un usuario inicia sesión en la aplicación, se almacenan sus datos relevantes en un token, que servirá para garantizar el acceso a los servicios ofrecidos solo para aquellos usuarios registrados y con el rol correcto. Para ello, se ha

implementado la función *verificarToken* (Código 1), que se asegura de que el usuario esté registrado y tenga un rol asociado.

```
1  const verificarToken = (req, res, next) => {
2    const authHeader = req.headers['authorization'];
3    const token = authHeader && authHeader.split(' ')[1];
4
5    if (!token) {
6      return res.status(401).json({ success: false, message: 'Acceso
7 denegado. No hay token.' });
8    }
9
10   jwt.verify(token, secret_key, (error, decodedUser) => {
11     if (error) {
12       return res.status(403).json({ success: false, message: 'Token
13 inválido o caducado.' });
14     }
15
16     req.user = decodedUser;
17
18     next();
19   });
20 };
```

Código 1. Función *verificarToken*

- Edición del perfil

Para la edición del perfil, se han obtenido del *frontend* todos los datos del perfil del usuario, contando los editados y los no editados. De esta forma, se permite realizar un *UPDATE* de la tabla correspondiente, pasando los datos editados y los no editados, garantizando así la consistencia de las sentencias de edición de la tabla para cualquier valor modificado. Así, al editar un dato, realmente se están modificando todos, con la diferencia de que los que no hayan sido modificados, simplemente se sobrescribirán con los mismos valores que ya hay almacenados.

6.1.3 Implementación en el *frontend*

Para la implementación de la IU del registro e inicio de sesión se ha hecho uso de varias pantallas que han permitido el proceso de la solicitud tanto de los datos necesarios para el registro en la aplicación, como del inicio de sesión de una cuenta ya existente y la recuperación de la contraseña en caso de haberla olvidado. Se han usado componentes personalizados para los formularios (*customForm*, *firstTextInput*, *middleTextInput* y *lastTextInput*) y para la elección de rol (*toggleButtonRole*).



Ilustración 20. Pantalla de login de StayIn (modo claro)

Una vez implementado el registro e inicio de sesión, se deja el camino hecho para comenzar a implementar las diferentes funcionalidades en función del rol de cada usuario.

6.2 Publicación y gestión de anuncios (Sprint 2)

En este módulo, continuando con la implementación descrita en el punto anterior, se han implementado todas las funcionalidades relacionadas con la publicación, edición y eliminación de anuncios por parte de los caseros. Para ello se ha creado, en primer lugar, una pantalla de inicio (*home*) tanto para el usuario como para el casero, que será donde redirigirá la aplicación cada vez que un usuario entre en ella con la sesión iniciada, se registre, o inicie sesión de nuevo.

6.2.1 Lógica de base de datos

Para este módulo, se han creado las tablas *Viviendas* y *Anuncios*, las cuales servirán para almacenar todos los datos necesarios en cuanto a la gestión de inmuebles anunciados en la aplicación.

6.2.2 Implementación en el *backend*

Para llevar a cabo esta funcionalidad, ha sido necesario crear los *endpoints* `/api/home/casero` y `/api/home/inquilino`, en los que se valida la identidad del usuario mediante el uso del token JWT almacenado tras el inicio de sesión exitoso y se concede acceso a la cuenta en cuestión y al área principal de la misma. Una pantalla (*homeCasero* o *homeInquilino*, dependiendo del rol del usuario registrado) se encargará de mostrar sugerencias de pisos y usuarios para aquellos usuarios con rol de inquilino que se encuentren en búsqueda; mientras que, en el caso de los caseros, servirá para la gestión y publicación de sus propios anuncios. También se han necesitado los *endpoints* `/api/postad`, `/api/editad` y `/api/delete/ad`, los cuales sirven para recoger los datos del anuncio y vivienda a publicar y enviarlos a la base de datos, así como para alterar los datos de un anuncio ya almacenado o eliminarlo; respectivamente.

Para llevar a cabo estas acciones, ha hecho falta la modificación de las dos tablas mencionadas, bien para realizar la inserción de un nuevo anuncio y, por ende, una nueva vivienda; bien para la modificación de una instancia ya existente en alguna de las tablas, o bien para la eliminación de un anuncio en el sistema.

Como todos los casos han requerido la alteración de varias tablas de la base de datos, ha sido fundamental el uso de las cláusulas *BEGIN*, *COMMIT* y *ROLLBACK* una vez más para garantizar que no se realice ninguna modificación a medias de la base de datos, lo que generaría inconsistencia.

6.2.3 Implementación en el *frontend*

Para la parte visual de este módulo se han creado diferentes componentes para la visualización de las tarjetas de los anuncios en función del rol del usuario, mostrando para cada uno los aspectos más relevantes. Se han creado varios modales, por un lado, para la visualización de los anuncios por parte del usuario inquilino, de manera que pueda observar todas las imágenes de la vivienda, así como la descripción y las valoraciones de esta, y por otro lado para la visualización y edición del anuncio por parte del casero.

La pantalla principal del casero muestra una lista de sus anuncios publicados y un botón para el acceso al modal de publicación de anuncios, donde se muestra un formulario a rellenar a lo largo de varias pantallas con los datos necesarios para el anuncio, así como la adición de los archivos multimedia requeridos para la aceptación del mismo.

La pantalla principal del inquilino, en cambio, muestra una lista con los anuncios más relevantes según su perfil, seguido de un carrusel con usuarios como compañeros potenciales, en función a sus preferencias.

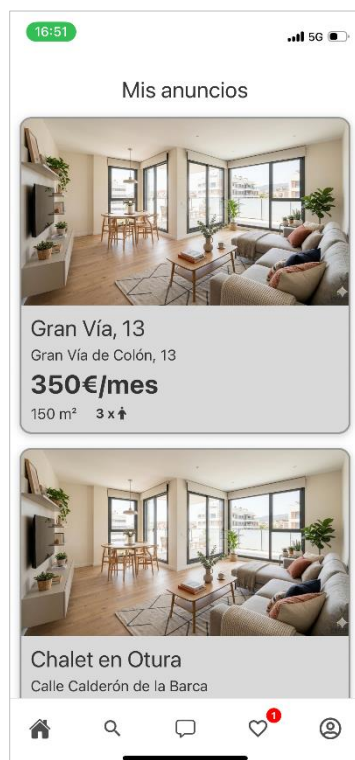


Ilustración 21. Home del usuario Casero



Ilustración 22. Tarjeta de anuncio editable por parte del casero

6.2.4 Desviaciones e imprevistos

Al avanzar en la implementación del proyecto y llegar a la fase del sistema de recomendaciones (6.5), se vio la necesidad de realizar cambios en la estructura definida para la publicación y edición de anuncios, debido a la presencia de las características y preferencias de usuarios y anuncios descritas más adelante, en el punto 6.5.3, obligando a añadir los atributos a la hora de publicar y editar anuncios de viviendas, lo que conlleva a su vez la modificación de los campos de la tabla Anuncios, añadiendo el atributo para almacenar las preferencias en formato JSONB.

Una vez implementado este módulo, se puede dar pie a la interacción entre inquilinos y usuarios mediante solicitudes de registro, que se detalla en el siguiente punto.

6.3 Vinculaciones y social (Sprint 3)

En este módulo se ha llevado a cabo la implementación de las conexiones entre usuarios, permitiendo la solicitud de registro en vivienda por parte de los inquilinos y la correspondiente aceptación por parte de los caseros, así como la posibilidad de comunicación con otros usuarios del sistema mediante un chat integrado en la aplicación.

6.3.1 Lógica de base de datos

Para este módulo se han creado las tablas Estancias y Solicitudes. La primera de ellas se encarga de almacenar las relaciones de convivencia entre diferentes usuarios, la fecha de inicio de la estancia y la fecha de fin, en caso de que ya no tenga efecto. La única manera de comenzar una estancia es mediante una solicitud previa, de manera que la segunda tabla será la encargada de almacenar todas las solicitudes de estancia enviadas y de establecer sus estados a los correspondientes, según estén enviadas; o bien, aceptadas o rechazadas por el usuario receptor.

6.3.2 Implementación en el *backend*

La lógica implementada para esta funcionalidad se ha dividido en dos partes, por un lado, la creación y emisión de la solicitud y, por otro, la respuesta y procesamiento de esta. La implementación seguida garantiza la seguridad e integridad de las operaciones de interacción con la base de datos.

- Emisión de solicitudes

Esta es la lógica por el lado del usuario inquilino, el cual mandará una solicitud al casero por medio del *endpoint* `/api/solicitudes/crear`, donde simplemente se creará una instancia de la tabla solicitudes con el id del usuario en cuestión y del anuncio solicitado.

Se ha garantizado la unicidad de las solicitudes mediante la implementación de un *catch* para el código de error 23505 (violación de unicidad), de manera que, en el caso de producirse algún problema en el sistema que permitiera al usuario mandar una nueva solicitud a un anuncio ya solicitado previamente, se devolvería un error con el mensaje “Ya has enviado una solicitud para este anuncio”.

- Recepción y procesamiento de solicitudes

Para la lógica de la recepción y el procesamiento o respuesta de las solicitudes, es decir, la parte del casero, se han creado dos *endpoints*: `/api/solicitudes/casero` y `/api/solicitudes/procesar`.

El primero de estos *endpoints* es el encargado de obtener las solicitudes pendientes de responder (estado *Pendiente*) del casero. Para ello se obtienen los ids de la solicitud y del anuncio en cuestión, la fecha en la que transcurre el evento, el nombre de usuario del inquilino que ha enviado la solicitud y el título del anuncio solicitado. Esta consulta a la base de datos se lleva a cabo mediante el acceso a las tablas Solicitudes, Anuncios y Usuarios a través de un *JOIN*, como se puede ver en el Código 2, ordenándolos en función de la fecha de solicitud, de forma que aparezcan las más recientes primero.

```
1  const resultado_solicitudes = await client.query(`
2      SELECT
3          s.id_solicitud,
4          s.id_anuncio,
5          s.f_solicitud,
6          u.username,
7          a.titulo AS titulo_anuncio
8      FROM Solicitudes s
9      JOIN Anuncios a ON s.id_anuncio = a.id_anuncio
10     JOIN Usuarios u ON s.id_inquilino = u.id_usuario
```


11	WHERE a.id_casero = \$1 AND s.estado = 'Pendiente'
12	ORDER BY s.f_solicitud DESC;`,
13	[id_casero]);

Código 2. Consulta de obtención de solicitudes

El segundo *endpoint* sirve para el procesamiento de la respuesta del casero, de forma que se acepta o rechaza la solicitud en función de la opción escogida por el usuario y recibida del *frontend*, modificando la instancia de la solicitud al estado *Aceptada* o *Rechazada*. En caso de que la solicitud se acepte, se inserta además una instancia en la tabla Estancias con los datos de los usuarios involucrados.

6.3.3 Implementación en el *frontend*

Para el envío, recepción y procesamiento de solicitudes en el *frontend* se han necesitado, fundamentalmente, las pantallas *home* del inquilino y *social* del casero. A través de la pantalla principal del inquilino se accede a la vista de un anuncio, donde, además de todos los datos y descripciones relevantes del anuncio, se encuentra el botón de *Solicitar*, de manera que, al pulsarlo, se hace uso del *endpoint* `/api/solicitudes/crear` mencionado anteriormente, enviando al *frontend* todos los datos relevantes del usuario inquilino.

Una vez que se ha enviado una solicitud, se debe notificar al usuario casero para que pueda responder a ella. En entornos de producción reales, lo ideal sería mandar una notificación en tiempo real al casero, para que sepa que un usuario está interesado en su anuncio lo antes posible. Sin embargo, para la implementación en este proyecto, este proceso se ha simplificado a efectos prácticos y temporales, de manera que las solicitudes se cargan (mediante la conexión con el *backend*) cada vez que se accede o se recarga la página principal del casero. De esta manera, se asegura no sobrecargar el servidor de peticiones en busca de nuevas solicitudes, pero se mantiene una estructura funcional y fácil de probar para su recepción. Esta llamada al recargar la página se ha realizado mediante el uso de la función `useFocusEffect` y `useCallback` de los módulos *expo-router* y *react*, respectivamente, como se puede ver en el Código 4. El uso de estas dos funciones en lugar del clásico `useEffect` del módulo *react* se debe a que, la navegación de pantallas proporcionada por Expo Router no es como la proporcionada por React, donde al cambiar de una página a otra, la anterior se destruye y, por ende, al volver a ella se vuelve a recargar, haciendo válido el uso de `useEffect` para obtener datos al recargar la página. No obstante, en Expo Router, la navegación se basa en la apilación de unas pantallas encima de otras, por lo que, al cambiar de una pantalla a otra y volver de nuevo a la primera, no se está cargando de nuevo, lo que impediría la solicitud al *backend* de los nuevos datos. Es por esto por lo que, para garantizar la correcta solicitud al servidor, se usan las funciones `useFocusEffect`; que permite la ejecución de peticiones cada vez que la pantalla recupera el foco (se convierte en pantalla activa); junto con `useCallback`, que garantiza la eficiencia computacional y evita los bucles de renderizado infinitos.

De esta manera, se permite una recarga estable entre renderizados y una comunicación con el servidor única y exclusivamente cuando el usuario accede a esa pantalla, además de garantizar una experiencia de usuario (UX) fluida al interactuar con la barra de navegación de la aplicación (componente *NavBar*).

6.4 Sistema de valoraciones (Sprint 4)

En este módulo se lleva a cabo la implementación de una de las funcionalidades más valiosas y útiles de este proyecto, la posibilidad de valorar a otros usuarios con los que se haya establecido una relación de convivencia, de cara a dar *feedback* al resto de usuarios de la aplicación. El sistema de valoraciones está basado en las preguntas definidas en el punto 4.2.4.

6.4.1 Lógica de base de datos

Para la implementación de este sistema, se han creado las tablas *Valoraciones*, *Criterios_Valoracion* y *Puntuacion_Criterios*, cada una de ellas con una tarea concreta. La tabla

Valoraciones servirá para llevar un registro de quién valora a quién y en base a qué estancia. Por otro lado, la tabla `Criterios_Valoracion` sirve para almacenar todas las preguntas que se van a solicitar responder al usuario valorador, las cuales pueden verse tanto en el punto 4.2.4 como en el Código 26 (en el caso de valoración entre usuarios). Por último, la tabla `Puntuacion_Criterios` sirve para almacenar la nota puesta por un usuario a un criterio concreto al valorar a otro. De esta manera y mediante el uso de estas 3 tablas, se pueden almacenar, consultar y modificar las valoraciones de cualquier usuario a otro actor del sistema.

6.4.2 Implementación en el *backend*

Para la implementación del sistema de valoraciones se han usado dos *endpoints* de carga de solicitudes pendientes, cada uno en el “*home*” del usuario en función del rol, `/api/home/casero` y `/api/home/inquilino`, uno para la obtención de las cuestiones en función de la valoración a realizar, `/api/valoraciones/usuario`, y otro para el procesamiento y envío de la valoración una vez realizada, `/api/valoraciones/usuario/enviar`.

- Obtención de las valoraciones pendientes

Para obtener las valoraciones que un usuario tiene pendientes de realizar, simplemente se ha llevado a cabo una consulta a la base de datos de la forma que podemos ver en el Código 3, permitiendo que se devuelvan al *frontend* las valoraciones que haya pendientes para ese usuario, en caso de haberlas.

```
1 SELECT e.id_estancia
2   FROM Estancias e
3  JOIN Viviendas v ON e.id_vivienda = v.id_vivienda
4  LEFT JOIN Valoraciones val ON e.id_estancia = val.id_estancia AND
5 val.id_valorador = $1
6 WHERE
7   (e.id_inquilino = $1 OR v.id_casero = $1)
8   AND e.f_fin_estancia IS NOT NULL
9   AND val.id_valoracion IS NULL;
```

Código 3. Consulta de valoraciones pendientes

- Obtención de las cuestiones de la valoración

En caso de que el usuario tenga alguna valoración pendiente, se realiza una consulta a la tabla `Criterios_Valoracion`, obteniendo así todas las cuestiones para la valoración, en función del tipo que sea.

- Envío de valoración realizada

Cuando todas las preguntas han sido respondidas y el usuario ha presionado el botón de enviar, se realizan dos inserciones; una vez más, protegidas por las sentencias *BEGIN*, *COMMIT* y *ROLLBACK*; en las tablas `Valoraciones` y `Puntuacion_Criterios`, para almacenar la relación de la valoración y la puntuación de cada aspecto de esta, respectivamente.

6.4.3 Implementación en el *frontend*

En este caso, el *frontend* es el encargado de realizar la recarga de las posibles valoraciones del usuario, o al menos de llevar a cabo la llamada que la realice. Para esto, en un entorno de producción, lo ideal sería que la aplicación notificase al usuario en tiempo real de que tiene una valoración pendiente y se lo recordara cada cierto tiempo, incentivando a este a realizarla. Sin embargo, por falta de tiempo y para simplificar este aspecto e implementar un MVP, se ha realizado una petición al *backend* cada vez que el usuario acceda a su pantalla “*home*”, manteniendo la solución acorde a la planteada con las notificaciones del apartado anterior. Esta implementación se puede ver en el Código 4, de manera que, cada vez que el usuario se sitúe en su pantalla principal, se realizará la petición descrita en el Código 3 y se recibirán las valoraciones pendientes, siempre y cuando las

haya. En caso de haberlas, se mostrará la notificación con el número de valoraciones pendientes en el icono de la pantalla social del usuario, como podemos ver en la Ilustración 23 y en la Ilustración 24.

Una vez que entremos en la valoración concreta, se mostrarán todas las cuestiones para responder con estrellas en una escala del 1 al 5, que no podremos enviar hasta que hayamos respondido todas.

```

1 useFocusEffect(
2   useCallback(() => {
3     getAnuncios();
4     getSolicitudes();
5   }, []));

```

Código 4. Función recarga solicitudes y valoraciones pendientes

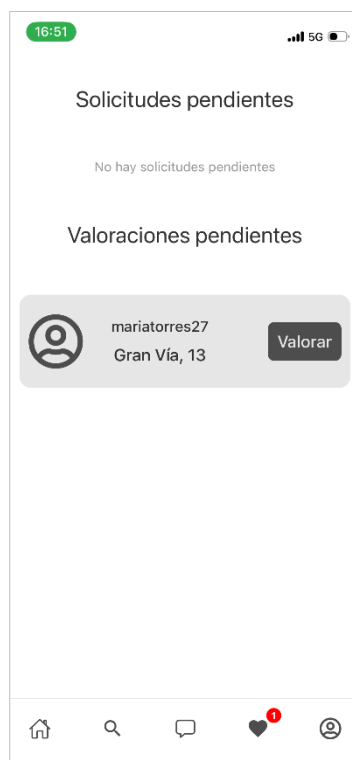


Ilustración 23. Pantalla social con valoración pendiente

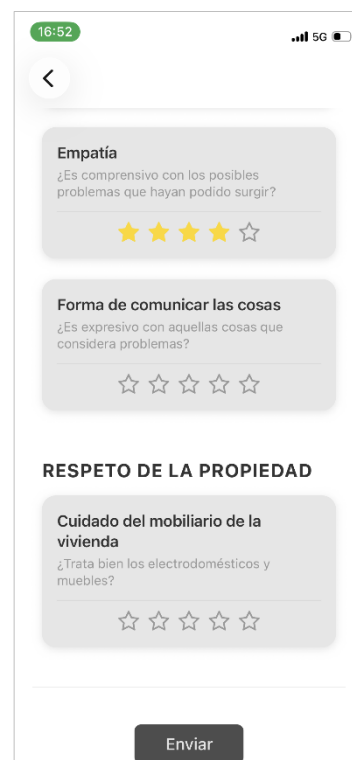


Ilustración 24. Valoración a usuario

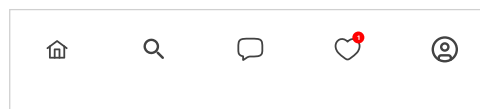


Ilustración 25. Barra de navegación con notificación pendiente

Una vez implementada la estructura necesaria para gestionar el perfil de usuario, la publicación de anuncios y el ecosistema de valoraciones, el sistema cuenta con una base de datos robusta y rica en potenciales interacciones, que se puede usar como base para alimentar el sistema de recomendaciones desde el primer momento. A continuación, se define el planteamiento e implementación del motor de recomendación híbrido desarrollado para las recomendaciones del sistema al usuario, que aprovecha todos estos datos para elevar la eficacia y fiabilidad del proceso de búsqueda de StayIn.

6.5 Sistema de recomendación (Sprint 5)

En este módulo de implementación se han llevado a cabo todas las decisiones de diseño y de planteamiento del sistema de recomendación, tratando de maximizar la compatibilidad entre

usuarios y viviendas, y entre usuarios entre sí. De esta forma, se ha diseñado un sistema de recomendaciones orientado a anuncios y otro orientado a compañeros, ambos basados en una arquitectura híbrida compuesta por capas, diseñado para ser funcional desde el primer usuario y escalables hacia técnicas de aprendizaje automático a medida que el sistema crezca.

Los dos sistemas comparten la arquitectura por capas, compuesta por una base matemática, un análisis semántico mediante NLP y una monitorización de interacciones del usuario, pero difieren en la naturaleza del problema a resolver, como se detalla a lo largo de este capítulo.

6.5.1 Arquitectura general del sistema de recomendación híbrido

El sistema de recomendación planteado en ambos casos responderá al paradigma de sistemas híbridos, combinando en este caso filtrado colaborativo y filtrado basado en contenido, así como interacciones del usuario con la aplicación y el análisis semántico mediante NLP. Los sistemas se organizan en capas, con la disposición que se puede ver en la Tabla 44.

Capa	Técnica	Estado	Propósito
1 ^a	Filtrado basado en contenido	Implementado	Ofrecer recomendaciones objetivas a los usuarios basado en las preferencias introducidas y características ofrecidas por anuncios u otros usuarios
2 ^a	Similitud semántica (NLP)	Implementado	Ofrecer más métricas de similitud entre, bien usuarios y anuncios, o bien usuarios, mediante la comparación semántica de sus descripciones
3 ^a	Feedback implícito del usuario mediante interacciones	Parcial; infraestructura implementada, a falta de integración	Monitorización de las interacciones del usuario con la aplicación para aumentar o disminuir la puntuación obtenida en función de su comportamiento real
4 ^a	Filtrado colaborativo (ML)	Trabajo futuro	Identificación de patrones colectivos entre usuarios, mejorando así la calidad de las recomendaciones. Requiere masa crítica de datos

Tabla 44. Organización por capas del sistema de recomendaciones

Con esta organización por capas se solventa el problema del arranque en frío, haciendo un motor de recomendaciones funcional desde el primer usuario mediante el filtrado basado en contenido combinado con la similitud semántica entre descripciones de usuarios y anuncios y el comportamiento real del usuario mediante sus interacciones con la aplicación.

6.5.2 Cálculo de la puntuación: capas comunes a ambos motores

Las capas 1, 2 y 3 descritas en la Tabla 44, aunque difieren en cuanto a implementación en ciertos apartados, son comunes a ambos motores. A continuación, se describe cómo funcionan cada una de ellas.

- Capa 1. Filtrado basado en contenido (la base matemática)

En esta primera capa del sistema de recomendaciones se realiza un cruce entre las preferencias de búsqueda del usuario con las características presentadas por las entidades del sistema, anuncios u otros usuarios, según el caso.

En cada uno de los sistemas de recomendaciones este cruce se realiza de una manera distinta, debido a las diferencias en la naturaleza de ambos problemas, realizando un cruce unidireccional en el caso de los anuncios, donde solo es necesario comparar las preferencias del usuario con las características del anuncio; mientras que, en el caso de los usuarios, es necesario realizar un doble cruce de las preferencias de un usuario con las características ofrecidas por el otro, y viceversa, debido a la bidireccionalidad que se presenta en este problema. Estas diferencias se detallan en los puntos 6.5.3 y 6.5.4, respectivamente, además de en la comparativa relacionada entre ambos sistemas de recomendación en la Tabla 50.

- Capa 2. Similitud semántica (NLP)

Esta segunda capa, a diferencia de la anterior, es exactamente igual en ambos sistemas de recomendación. Una vez que tenemos esa base algorítmica implementada, la cruzaremos con la puntuación obtenida entre la relación de la descripción del usuario y de la entidad. Para esto usamos un modelo de Procesamiento de Lenguaje Natural (NLP), que se encargará de analizar el texto de la descripción del usuario y del anuncio/posible compañero, calculando así la similitud entre uno y otro.

Para poder implementar esto, se ha hecho uso del modelo *paraphrase-multilingual-MiniLM-L12-v2* [107], un modelo ligero, perfectamente funcional en español, ya que lo entiende nativamente, y especializado en analizar dos textos y ver si significan o no lo mismo (como podemos ver con la etiqueta *paraphrase*).

Para este modelo se ha instalado la librería *sentence-transformer*, mediante el siguiente comando:

```
pip install sentence-transformer
```

Una vez instalado el modelo, se carga en el programa y se procesan ambos textos mediante la función *encode*, convirtiendo ambas descripciones en arrays multidimensionales (tensores) para que se puedan tratar matemáticamente sin problema.

Al obtener los tensores, se puede calcular la similitud entre ambos. Para esto, se hace uso de la Similitud del Coseno, que podemos ver en la Ecuación 1, que nos permite calcular el grado de semejanza entre dos vectores en un espacio multidimensional.

$$\cos \theta = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

Ecuación 1. Cálculo de la Similitud del Coseno

De esta forma, tendremos varios posibles casos en función de la similitud de ambos textos. Si significan prácticamente los mismo, los vectores apuntarán en la misma dirección (o casi la misma), por lo que el ángulo (θ) será de 0 grados y, por tanto, el coseno (es decir, la similitud), valdrá 1.

En el caso de que signifiquen cosas totalmente distintas, el ángulo formado entre ambos vectores será de 90 grados y la similitud 0.

Si los textos significan cosas opuestas, los vectores apuntarán a direcciones también opuestas y, por tanto, el ángulo será de 180 grados y la similitud valdrá -1.

- Normalización del resultado y calibración mediante umbral de éxito

Como se puede inferir en el apartado anterior, tras calcular la similitud entre ambas descripciones, tendremos un valor en el rango $[-1,1]$, lo cual no es demasiado útil para el caso de uso en cuestión. Es por esto por lo que es necesario llevar a cabo una normalización del resultado, llevándolo a una escala que podamos interpretar correctamente, en este caso $[0,1]$.

Para esto, hacemos uso de la Ecuación 2, donde, al sumar 1 al valor de la similitud, pasamos el rango de $[-1,1]$ a $[0,2]$ y, al dividir entre 2, al rango $[0,1]$. Multiplicamos por 100 y obtenemos el resultado en un porcentaje.

$$Score_{NLP} = \frac{similitud + 1}{2} \times 100$$

Ecuación 2. Normalización a rango $[0,100]$

Como se ha comentado anteriormente, un NLP devuelve 1 cuando dos textos son idénticos, por lo que podemos asumir que, en casos prácticos, obtener un 100% de similitud mediante el NLP es casi imposible. Sin embargo, lo que se busca en este caso no es que los textos sean idénticos, sino que vengan a decir lo mismo, es decir, que apunten en la misma dirección general. Por esto, se ha decidido aplicar un umbral de éxito, de manera que dos textos que obtengan una similitud del NLP de 0,4 o mayor, serán tratados como una similitud del 100% automáticamente, como se puede observar en la Ecuación 3. De esta manera se penaliza únicamente aquellas descripciones con significados claramente distintos, evitando penalizaciones injustas a aquellas descripciones compatibles pero redactadas de manera diferente.

Este valor de umbral ha sido calibrado a partir de la observación de datos representativos, queda como trabajo pendiente el ajuste a partir de datos empíricos de descripciones reales analizadas, para observar cuál puede ser un valor óptimo con el objetivo de mejorar la calidad de las recomendaciones del sistema.

$$Score = \max\left(0, \frac{Similitud}{Umbral} \times 100\right), \quad si \text{ Similitud} < Umbral$$

$$Score = 100, \quad si \text{ Similitud} \geq Umbral$$

Ecuación 3. Cálculo de la puntuación aplicando umbral

Ejemplo práctico

Supongamos que tenemos un usuario y un anuncio de una vivienda, cuyas descripciones son "Soy un estudiante tranquilo, me gusta el silencio y estudiar en casa. Odio el ruido" y "Buscamos a alguien calmado, el piso es un templo de estudio. No se admiten fiestas", respectivamente.

Como se puede observar, ambos textos, aunque no idénticos, tienen significados muy similares. Cabría esperar que, ante este caso, el NLP diese una puntuación bastante alta. No obstante, al analizar ambas descripciones, obtenemos simplemente un 65,31% de similitud semántica, lo que hace que, en un caso práctico, la puntuación obtenida de la primera capa no se vea realmente potenciada por la similitud de las descripciones, cuando en realidad podría ser un caso de dos entidades realmente compatibles. Si desglosamos el cálculo de ese 65,31% mediante la Ecuación 2, vemos que la similitud calculada por el NLP para esos dos textos es 0,306.

En base a este caso, podemos ver que estos dos textos, siendo bastante similares en cuanto a significado, superan escasamente un 0,3 sobre 1. Es por esto por lo que se ha decidido establecer un umbral de éxito de 0,4, como se detalla en el punto anterior. De esta forma, si aplicamos la Ecuación 3, observamos cómo la puntuación de la similitud semántica una vez normalizada asciende al 76,55%, lo cual representa mejor la similitud en el significado de las descripciones y potencia realmente la puntuación matemática obtenida de la primera capa del sistema de recomendación.

Por este motivo, se elige la normalización definida en la Ecuación 3 en lugar de la definida en la Ecuación 2, ya que representa mejor la similitud semántica entre ambos textos.

- Capa 3. Feedback implícito del usuario e interacciones

Por último, el sistema se encargará de recopilar las interacciones que tenga cada usuario con la aplicación. Las interacciones tenidas en cuenta se pueden ver representadas en la Tabla 45. Esto servirá para ver realmente qué interés muestra el usuario en los diferentes anuncios o usuarios ofrecidos por el sistema, ya que por mucho que se trate de encontrar la mayor afinidad en base a matemáticas e IA, el feedback del propio usuario con el sistema siempre será mucho más valioso, por lo que será útil para combinarlo con el resto de las puntuaciones, sacando una lista final de recomendaciones que se acerque lo máximo posible a lo que al usuario le puede interesar realmente.

El uso de estas interacciones se integrará a la lista de anuncios o usuarios basados en la afinidad descrita anteriormente, a modo de potenciador en función de la interacción que haya realizado el usuario.

Nombre de la interacción	Descripción	Δ Feedback
Solicitud de contacto (mensaje)	Es el mayor indicativo de que al usuario le interesa el anuncio o usuario en cuestión	+15
Guardar en favoritos	El usuario no quiere perder de vista el anuncio o usuario del sistema y se lo guarda para ver más adelante.	+10
Descartar	El usuario rechaza activamente una sugerencia del sistema, indicando que no está interesado, por lo que el sistema reduce drásticamente su prioridad.	-80
Reportar anuncio/usuario	De la misma manera que con el anterior, el usuario demuestra activamente que no está interesado en esa sugerencia, llevándolo a un nivel más y mostrando su descontento por algún motivo concreto, reportando el anuncio o usuario.	-100
Tasa de clic	El usuario pulsa en la sugerencia, lo que demuestra que está interesado en conocer más detalles al respecto	+2
Dwell time	Representa el tiempo que un usuario pasa dentro de un anuncio o perfil de usuario, lo que pueda dejar ver el interés mostrado por el mismo.	Hasta +5
Interacción con la galería > 50% (en el caso de anuncios)	Cuando el usuario ve más del 50% de las imágenes mostradas en el anuncio, lo que demuestra que puede estar interesado en ver exactamente qué se está ofreciendo.	+3
Expansión de texto	Si la descripción del usuario o anuncio es demasiado larga como para mostrarla al completo, que el usuario pulse el botón de “ver más” puede	+2

	demostrar interés por el mismo.	
--	---------------------------------	--

Tabla 45. Interacciones tenidas en cuenta

Todas estas interacciones serán almacenadas en las tablas Interacciones_Companeros e Interacciones_Anuncios, lo que, más adelante, permitiría el entrenamiento de un modelo supervisado que complementaría la arquitectura híbrida descrita a lo largo de todo este capítulo.

Cabe decir que todos los valores aquí definidos son meramente impuestos en función a importancia teórica y no tienen, por tanto, su base en ningún estudio ni prueba realizados, ya que para eso haría falta un mayor volumen de usuario e interacciones de los mismos con los que poder ajustar los valores de cada interacción, de manera que se optimizara al máximo la calidad de las recomendaciones a los usuarios en base a su comportamiento.

- Matriz de utilidad implícita

Con el objetivo de preparar la infraestructura de datos necesaria para el filtrado colaborativo de la capa 4, se ha implementado una matriz de utilidad almacenada en la base de datos. Esta matriz almacenará, para cada par (usuario/entidad, siendo una entidad un anuncio u otro usuario), un *utility score* [108] que resume todas sus interacciones históricas incorporando un factor de decaimiento temporal exponencial (Ecuación 4), técnica introducida por Ding y Li [109], de manera que siempre tengan más peso las interacciones más recientes. Este *utility score* se calcula a través de la fórmula definida en la Ecuación 5, donde se adapta la fórmula del decaimiento temporal exponencial, adaptándola al caso de uso de este sistema, realizando la suma de los pesos de todas las interacciones llevadas a cabo por el usuario y aplicándola a la fórmula.

$$N(t) = N_0 \times e^{(-\lambda t)}$$

Ecuación 4. Cálculo del Decaimiento Temporal Exponencial

Donde

N_0 : valor inicial.

t : la antigüedad de la interacción

λ : tasa de decaimiento

$$Utility Score(u, i) = \sum_{k=1}^n Peso_k \times e^{(-\lambda t_k)}$$

Ecuación 5. Cálculo del Utility Score

Donde

n : número de interacciones registradas entre el usuario u y la entidad i .

$Peso_k$: peso asociado a la interacción k .

t_k : antigüedad de la interacción k , medida en días como la diferencia entre el instante actual y el instante en que se produjo.

El único parámetro libre de este modelo es la tasa de decaimiento λ . En lugar de asignar un valor arbitrario sin justificación, se ha optado por parametrizarlo en función de su semivida, es decir, el número de días que deben transcurrir para que una interacción reduzca su peso a la mitad. Imponiendo esta condición sobre la Ecuación 4, obtenemos la relación directa entre ambos, como vemos en la Ecuación 6.

$$e^{(-\lambda h)} = \frac{1}{2} \Rightarrow \lambda = \frac{\ln(2)}{h}$$

Ecuación 6. Relación entre la tasa de decaimiento temporal exponencial y la semivida

De esta forma, se hace depender el valor de λ de un valor temporal, que depende de la velocidad a la que cambian los intereses del usuario en el caso de uso concreto de esta aplicación. En ámbitos como la recomendación de películas o música, este proceso es lento; de hecho, ciertos

estudios sobre valoraciones de películas sitúan la semivida óptima en torno a los 150 días [110]. Sin embargo, la búsqueda de vivienda y de compañeros de piso presenta una naturaleza radicalmente distinta: se trata de un proceso corto, intenso y con un final definido. Un usuario permanece activo durante semanas, no meses, y abandona la búsqueda en cuanto encuentra una vivienda. En consecuencia, una interacción de varias semanas de antigüedad pierde con rapidez su valor predictivo, pues el usuario puede haber modificado su presupuesto, cambiado su zona de interés o resuelto por completo su búsqueda.

Atendiendo a esta naturaleza, se ha fijado una semivida de $h = 14$ días (dos semanas), que se corresponde con una tasa de decaimiento $\lambda = \ln(2)/14 \approx 0,0495 \text{ día}^{-1}$. Este valor establece un equilibrio adecuado: concede un peso elevado a la actividad de la última semana, que se considera la más representativa de la intención actual del usuario, al tiempo que atenúa con rapidez las interacciones antiguas. Con el fin de verificar que el comportamiento resultante es coherente con el dominio, la Tabla 46 recoge el peso que conserva una interacción en función de su antigüedad para la semivida escogida.

Antigüedad de la interacción	Peso conservado
1 día	95,2%
1 semana	70,7%
2 semanas (=h)	50,0%
3 semanas	35,4%
1 mes	22,6%
2 meses	5,1%
3 meses	1,2%

Tabla 46. Peso conservado por una interacción según su antigüedad ($h = 14$ días)

Como puede observarse, una interacción mantiene prácticamente todo su peso durante la primera semana, lo reduce a la mitad a las dos semanas y resulta ya casi residual transcurridos dos meses, un perfil que se ajusta con precisión a la duración típica de un proceso de búsqueda de vivienda.

El valor de h constituye una decisión de diseño basada en la naturaleza del dominio ante la ausencia de un volumen suficientemente grande de datos históricos para su calibración. Se trata, por tanto, de un valor susceptible de cambios y reajustes mediante pruebas una vez que el sistema conste de un conjunto de interacciones suficientemente representativo, por lo que se deja como trabajo a futuro la optimización del valor en cuestión.

- Puntuación final incorporando el feedback

Una vez que tenemos una puntuación matemática mediante la aplicación de la capa 1 y 2 y un *feedback* obtenido a través de las interacciones del usuario, se integran todos los valores mediante la fórmula definida en la Ecuación 7.

$$Score\ final = (\alpha \times Score\ matemático) + (\beta \times Score\ NLP) + \Delta Feedback$$

Ecuación 7. Cálculo del Score Final para una recomendación

Donde:

α es el peso del *score* matemático (80%).

β es el peso aplicado al *score* del NLP (20%).

$\Delta Feedback$ es el potenciador aplicado por las interacciones del usuario (limitando el *Score final* al rango [0,100]).

La decisión de los porcentajes asignados tanto al cálculo matemático como a la similitud semántica entre descripciones se basa en la mayor fiabilidad presentada por el contenido estructurado del cruce de los datos de las distintas entidades participantes. No obstante, los valores exactos han sido seleccionados como estimación y están sujetos a cambios con el paso del tiempo y,

sobre todo, con el tratamiento de datos y usuarios reales, en cuyo caso se realizaría un estudio más exhaustivo de los valores pertinentes según cada caso.

Por otro lado, el feedback se recopila y materializa en la matriz de utilidad, pero su incorporación al ranking en vivo queda pendiente hasta tener bien definidas e implementadas todas las interacciones del sistema, por lo que en el código actual no se está aplicando ese incremento.

- **Capa 4. Filtrado colaborativo**

Una vez la aplicación tenga una cantidad suficientemente grande de usuarios y, por tanto, de datos e interacciones, se podrá llevar a cabo la implementación de la capa 4. A diferencia de la capa 1, en esta no se evalúan las preferencias de los usuarios ni las características ofrecidas por las entidades, sino que se detectan patrones de comportamiento colectivo, de manera que, si dos usuarios muestran un patrón de interacciones similar, es muy probable que una entidad que interesa a uno de ellos, también interese al otro.

La idea de esta capa sería implementar varias técnicas de filtrado colaborativo definidas en el apartado de Sistemas de recomendación: el filtrado colaborativo basado en usuario y el filtrado colaborativo basado en ítems.

La infraestructura para la correcta implementación de esta capa ya estaría implementada gracias a la matriz de utilidad definida anteriormente, constituyendo la entrada necesaria para cualquier algoritmo de procesamiento de datos mediante *Machine Learning*.

El entrenamiento del modelo se deja como trabajo futuro debido a que no se dispone del volumen de datos necesario para poder llevarlo a cabo correctamente, y realizarlo mediante datos insuficientes o generados de manera aleatoria o inventada produciría predicciones poco fiables, perjudicando la experiencia de usuario y quedando fuera del alcance de este proyecto.

6.5.3 Anuncios

En primer lugar, antes de implementar nada de código, se ha realizado un trabajo previo de estudio sobre qué factores deben determinar el descarte automático de un anuncio a un determinado usuario del sistema, es decir, los filtros barrera o filtros duros (*hard constraints*); y qué aspectos de una recomendación pueden ser más o menos importantes y, por tanto, tener más peso en la recomendación de ese anuncio, lo que se conoce como las variables de puntuación o filtros blandos (*soft constraints*).

- **Filtros barrera**

En este caso, se han tenido en consideración aquellos factores que determinarían automáticamente la no compatibilidad entre un usuario y una vivienda anunciada en la aplicación. Se han escogido como los más importantes el presupuesto máximo, la ubicación del piso y el permiso de vivir con mascotas, cuyas razones se desglosan a continuación.

- **Presupuesto máximo**

El presupuesto máximo especificado por un usuario debe ser el factor más determinante para decidir si un piso es interesante para él o no, ya que representa directamente las limitaciones reales del usuario a la hora de escoger una vivienda. Es por esto por lo que se realiza un filtro cortante para todos aquellos anuncios con un precio mayor al presupuesto máximo indicado. No obstante, para el tipo de tareas como buscar una vivienda, siempre puede haber alguna excepción para aquellos casos en los que un anuncio que cubre perfectamente todos los requisitos del interesado, pero se sobrepasa mínimamente el presupuesto establecido. Es por esto por lo que se han considerado también aquellos anuncios que superen las premisas salariales hasta en un 5% pero obtengan una puntuación muy alta para ese usuario, ya que se entiende que podría llegar a interesarle en caso de verlo.

- **Ubicación del inmueble**

Este filtro hace referencia no a la zona en sí de la vivienda (barrio, código postal, etc.) sino, más bien, a la ciudad o municipio en sí donde el usuario está interesado, de manera que no se

recomiende un anuncio de Sevilla a un usuario interesado en encontrar viviendas en Granada, por ejemplo.

- Mascotas

Este filtro es el menos común de los filtros barrera, ya que no en todos los casos será necesario aplicarlo porque no todos los usuarios tienen mascota y, por tanto, no tiene por qué interesarles si un anuncio permite o no la presencia de mascotas. Sin embargo, será crucial para aquellos usuarios que sí la tengan y, por tanto, sea un requisito indispensable que se les permita meterla en la vivienda.

- Variables de puntuación

Una vez que un anuncio pase los filtros duros del usuario, serán las variables de puntuación las que determinen la posición final de ese anuncio dentro de la lista de recomendados para el usuario.

Para esto, se han tenido en cuenta los que se han considerado como los aspectos más relevantes que un anuncio debe cumplir a la hora de buscar vivienda, que podemos ver a continuación.

- Economía y contrato

Incluye todas las restricciones a nivel de dinero extra que el usuario tendrá que suponer entrando al piso.

Gastos incluidos: se dará la opción de marcar el tipo de gastos incluidos a la hora de publicar un anuncio y de buscar, pudiendo marcar agua, luz, internet y comunidad.

Meses de fianza máximos: se dará la posibilidad de introducir el número máximo de meses a pagar para la fianza del inmueble.

Estancia mínima: determinará si la estancia en la vivienda es de corta, media o larga duración.

- Convivencia y normas

Este apartado sería útil en aquellos casos en los que son los mismos inquilinos del piso los que se están encargando de la búsqueda de compañeros (o ayudando al casero), de manera que puedan especificar qué se busca de la persona que entre; o en los que el casero quiere mantener un orden concreto en su vivienda.

- Habitación

Determinará las características concretas de la habitación a ocupar por el usuario interesado, con aspectos determinantes como el tipo de cama, las prestaciones disponibles, etc.

- Características de la vivienda

En este campo se podrán especificar las características generales de las que disponga la vivienda anunciada y que, por tanto, estarían disponibles para todo usuario que entrara como inquilino.

- Ejemplo de representación en formato JSON para la integración en el sistema

En el Código 5 podemos ver un ejemplo de un JSON que representaría las características de un anuncio.

```
1 {
2   "economia_contrato": {
3     "gastos_incluidos": ["agua", "internet"],
4     "meses_fianza_max": 1,
5     "estancia_minima_meses": 6
6   },
7   "convivencia_normas": {
8     "visitas": "ocasional",
9     "limpieza": "turnos_estrictos",
```

```

10     "ambiente": "estudio",
11     "ocupacion_inquilinos": ["solo_estudiantes"],
12     "genero_inquilinos": "femenino"
13 },
14     "habitacion": {
15         "cama_doble": true,
16         "zona_estudio": true,
17         "bano_privado": false
18     },
19     "caracteristicas_vivienda": {
20         "tipo_inmueble": ["piso"],
21         "banos_minimos": 2,
22         "ascensor": true,
23         "climatizacion": "calefaccion_central"
24     }
25 }

```

Código 5. JSON de ejemplo de características de una vivienda anunciada

- Cálculo de la puntuación matemática

En el motor de recomendaciones de anuncios, el cruce matemático hace uso de las preferencias del usuario con las características ofrecidas por cada anuncio. Para realizar este cruce, se ha llevado a cabo una asignación de pesos a cada uno de los módulos y submódulos que componen las características de un anuncio. Esto es, cada módulo (economía_contrato, convivencia_normas, habitación y características_vivienda) tendrá un peso asociado, de forma que la suma de los pesos de todos los módulos dé un total de 100. A su vez, el peso de cada módulo estará conformado por unos pesos internos asociados a cada submódulo que lo compone. De esta forma, los campos internos de un módulo tendrán un valor arbitrario asignado en cada caso en función de lo que se considera su importancia en la puntuación global, de forma que todos los pesos de los campos internos de un módulo sumen, de nuevo, 100, que será normalizado al valor sobre la puntuación final que tenga ese módulo. La asignación de los pesos se puede ver en el Código 28.

Para el cálculo de la puntuación matemática del sistema de recomendaciones de anuncios se han organizado las preferencias de un usuario (y las características, ya que son las mismas) en una jerarquía de dos niveles. Esta jerarquía se puede ver representada en el Código 28 mediante dos bloques: *módulos* e *internos*.

- Nivel superior (módulos)

Cada módulo representa un aspecto relevante de la convivencia en una vivienda, por lo que van desde características físicas de la vivienda hasta normas de convivencia o gastos. Cada módulo tiene un peso asociado, de manera que la suma de los pesos de todos dé como resultado 100, por lo que los pesos se pueden tratar directamente como porcentajes, como se puede ver en la Tabla 47.

Módulo	Peso	Significado
economia_contrato	40%	Condiciones económicas y contractuales (lo más decisivo)
convivencia_normas	30%	Normas de convivencia y estilo de vida
caracteristicas_vivienda	15%	Características físicas del inmueble
habitacion	15%	Características de la habitación privada

Tabla 47. Representación módulos de preferencias en anuncios

- Nivel inferior (campos internos)

Dentro de cada módulo, hay también una serie de campos con un peso asociado, cuya suma dentro de cada módulo será también de 100, representando, por tanto, el 100% del peso del módulo en cuestión. Esto puede verse de manera más clara en la Tabla 48.

Módulo	Peso M.	Interno	Peso I.
economia_contrato	40%	gastos_incluidos	50%
		estancia_minima_meses	30%
		meses_fianza_max	20%
convivencia_normas	30%	ambiente	40%
		limpieza	30%
		visitas	15%
		ocupación_inquilinos	9%
		genero_inquilino	4%
		lgtb-friendly	2%
habitacion	15%	bano_privado	30%
		cama_doble	35%
		zona_estudio	35%
caracteristicas_vivienda	15%	tipo_inmueble	40%
		banos_minimos	30%
		climatización	20%
		ascensor	10%

Tabla 48. Pesos de módulos y campos internos en preferencias de anuncios

Esta composición permite ajustar la importancia de los diferentes aspectos de la convivencia, dándole mucha más importancia a qué gastos de vivir en ese inmueble están o no incluidos, que a que el inmueble cuente o no con ascensor (estos pesos están sujetos cambios y lo ideal es que vayan en función de cada usuario, ya que la importancia de uno u otro aspecto puede cambiar considerablemente dependiendo de cada caso).

- Enrutador (patrón *Dispatcher*)

A la hora de cruzar los datos del usuario con los del piso, debido a la variedad de posibles valores en los diferentes campos internos de las características, no todos los valores se pueden evaluar de la misma forma, ya que no tendría sentido comparar, por ejemplo, la lista de gastos incluidos con el número de meses de fianza a pagar, puesto que el formato del valor de cada uno es completamente distinto.

Para arreglar este problema evitando el uso de largas cadenas de *if/else*, se ha hecho uso del patrón *Dispatcher*, definiendo en el constructor de la clase *MotorRecomendacion* el atributo *self.enrutador*, un diccionario encargado de asignar un método de comparación a cada campo interno según el caso, como se puede ver en el Código 30, contenido en el capítulo 8 para facilitar la lectura, debido a su extensión. Este fragmento de código muestra la inicialización del enrutador, como ya se ha comentado, implementando un patrón de despacho por tabla (*Dispatch Table*), realizando la asignación de una función a cada campo interno del objeto de características. De esta manera, mediante la iteración del objeto JSON se pueden obtener las funciones definidas para cada campo en función del tipo de valor almacenado, evitando una implementación más rudimentaria y difícil de leer mediante bloques largos de sentencias *if/else* evaluando cada caso concreto y aumentando la escalabilidad del sistema, ya que añadir un nuevo criterio simplemente conlleva añadir una entrada al diccionario y asociarle una función definida.

- Funciones de evaluación

Como se ha comentado en el punto anterior, el enrutador se encarga de asignar una función de comparación de valores a cada campo interno del objeto JSON de características, por lo que, a continuación, se van a definir todas las funciones implementadas, su funcionamiento y para qué tipo de casos se usan.

Todas las funciones descritas a continuación para el cálculo de la puntuación matemática en el caso de recomendaciones de viviendas reciben como parámetros un objeto, *deseado* (tipo lista,

entero o el tipo correspondiente según el caso) y un objeto, *real*, del mismo tipo que *deseado*, que representan las preferencias del usuario y las características del anuncio para ese campo concreto, respectivamente.

- Función evaluar_array

Esta función sirve para evaluar características con varios posibles valores, como pueden ser los campos *gastos_incluidos* o *tipo_inmueble*, en los que únicamente se penalizan aquellos valores requeridos por el usuario que no se encuentren presentes en las características del anuncio. Para el cálculo de la puntuación de este campo se usa la Ecuación 8.

$$Score = \frac{|pedidos \cap ofrecidos|}{|pedidos|} \times 100$$

Ecuación 8. Cálculo de score en evaluar_array

Donde:

pedidos: el conjunto de valores pedidos por el usuario (presentes en el objeto de preferencias del usuario).

ofrecidos: el conjunto de valores ofrecidos en el anuncio (presentes en el objeto de características del anuncio).

De esta manera, mediante la intersección realizada entre los valores pedidos y los ofrecidos se obtienen, al aplicar el valor absoluto, el número de aciertos (es decir, coincidencias) que hay en el objeto de características del anuncio respecto al de preferencias del usuario. Al dividirlo entre el número de valores pedidos y multiplicarlo por 100, se obtiene el porcentaje de acierto del objeto de características respecto al de preferencias. En el caso de que el usuario no haya solicitado nada (el valor es vacío para las preferencias) el resultado es 100 (no representa una característica relevante para él).

- Función evaluar_minimo

En esta función se define un umbral mínimo, de forma que todo lo que sea igual o superior a él será evaluado automáticamente como un 100%, pero se penalizará linealmente cada unidad de diferencia respecto a lo solicitado en función de una penalización fija definida en el constructor de la clase *MotorRecomendacion*, siendo la nota mínima permitida 0%, evitando así los valores negativos. Esta función se usa para casos en los que “más es mejor”, como puede ser el caso de *banos_minimos*. Por ejemplo, el usuario requiere que el inmueble cuente con, al menos, 2 baños. Si el inmueble cuenta con 2 o más baños, la puntuación obtenida es del 100%, ya que la oferta cubre o excede las expectativas. En su lugar, si el inmueble cuenta con un solo baño, la puntuación se penalizará respecto al valor de la penalización fija. Este cálculo se ve reflejado en la Ecuación 9.

$$Score = \begin{cases} \max(0, 100 - (deseado - real) \times penalizacion) & , \text{ si } deseado > real \\ 100 & , \text{ si } deseado \leq real \end{cases}$$

Ecuación 9. Cálculo del score en evaluar_minimo

Donde

deseado: el valor obtenido de las preferencias del usuario

real: el valor obtenido de las características de la vivienda

penalización: valor del atributo penalización declarado como atributo de *MotorRecomendaciones*.

Ejemplo 1

Suponiendo el ejemplo anterior, donde *deseado* = 2, *real* = 3, *penalización* = 25, se aplicaría el segundo caso de la ecuación definida, por lo que obtendríamos *score* = 100%.

Ejemplo 2

Si, en este caso, contamos con los siguientes valores: deseado = 3, real = 1, penalización = 25. Aplicando la Ecuación 9, obtendríamos un resultado de score = 50%.

- Función evaluar_friccion

Este es el caso inverso del anterior, donde “menos es mejor”, como puede suceder con *meses_fianza* o *estancia_minima_meses*, donde cualquier valor que supere el indicado por el usuario, impactará negativamente al interés por ese anuncio. En este caso, el valor deseado por el usuario actúa como el tope de tolerancia, de manera que, cada unidad que lo exceda supondrá una penalización en la puntuación, mientras que, en el caso de encontrarse el valor real por debajo del deseado, la puntuación será automáticamente del 100%, como se puede observar en la Ecuación 10.

$$Score = \begin{cases} \max(0, 100 - (real - deseado) \times penalizacion) & , \text{ si } real > deseado \\ 100 & , \text{ si } real \leq deseado \end{cases}$$

Ecuación 10. Cálculo del score en evaluar_friccion

Donde

deseado: el valor obtenido de las preferencias del usuario

real: el valor obtenido de las características de la vivienda

penalización: valor del atributo penalización declarado como atributo de *MotorRecomendaciones*.

Ejemplo

Supongamos un caso en el que un usuario guarda en sus preferencias del perfil que busca anuncios donde se exija, como máximo, una estancia mínima de 6 meses (*estancia_minima_meses* = 6). Para ambos casos se va a tomar una penalización = 25.

Caso 1

El anuncio para evaluar ofrece estancia mínima de 3 meses. Aplicando la Ecuación 10, obtenemos una puntuación del 100%.

Caso 2

El anuncio para evaluar ofrece estancia mínima de 6 meses. Aplicando la Ecuación 10, obtenemos una puntuación del 100%.

Caso 3

El anuncio para evaluar ofrece estancia mínima de 9 meses. Aplicando la Ecuación 10, obtenemos una puntuación del 25%.

Caso 4

El anuncio para evaluar ofrece estancia mínima de 12 meses. Aplicando la Ecuación 10, obtenemos una puntuación del 0% (sin la condición obtendríamos una puntuación de -50%).

- Función evaluar_exacta

Esta función se aplica a los casos en los que se tiene un valor de sí o no. Se aplica un cálculo de la puntuación simple, mediante coincidencia binaria: si los valores de ambos objetos son iguales, 100%, si son distintos, 0%. Esta función se aplica a campos como *baño_privado* o *ascensor*.

- Función evaluar_exacta_indiferente

Es el mismo caso de la función anterior aplicado a aquellos que también acepten el valor “indiferente” en las preferencias del usuario, en cuyo caso el valor será automáticamente 100%, independientemente del valor de las características del anuncio. Esta función se aplica a campos como *genero_inquilinos* y *ocupación_inquilinos*.

- Función evaluar_proximidad

Esta función se aplica a casos para variables cualitativas con un orden natural (*visitas*, *limpieza* o *ambiente*), donde los posibles valores vienen ordenados de forma que el primer valor del *array* sea el opuesto (o más contrario) del último, como se puede ver en el Código 29.

La puntuación obtenida decae según la distancia entre las posiciones dentro del *array* del valor deseado y el obtenido, como se puede observar en la Ecuación 11.

$$Score = \begin{cases} 100 - \frac{|id_{deseado} - id_{real}|}{longitud - 1} \times 100 & , \quad si \ id_{deseado}, id_{real} \neq "indiferente" \\ 100 & , \quad si \ id_{deseado} = indiferente \ \text{ó} \ id_{real} = indiferente \end{cases}$$

Ecuación 11. Cálculo del score en evaluar_proximidad

Donde:

id_{deseado}: posición dentro del *array* del valor obtenido de las preferencias del usuario.

id_{real}: posición dentro del *array* del valor obtenido de las características del anuncio.

longitud: número de elementos del *array*. Se le resta 1 para obtener la distancia máxima entre dos elementos dentro de esa lista.

Así, una diferencia de un escalón penaliza menos que una de dos. Mientras que la coincidencia exacta de posiciones indica que son el mismo valor y, por tanto, da una puntuación de 100%. Por otro lado, el valor indiferente por alguna de las dos partes también supone el 100% de la puntuación.

Ejemplo

Supongamos que tenemos un usuario y un anuncio, cuyas características están siendo evaluadas por el motor de recomendación de la aplicación. Al llegar al campo *ambiente*, encontramos los siguientes valores:

Caso 1

Preferencias usuario → *ambiente: estudio*

Características anuncio → *ambiente: animado*

Como podemos comprobar en el Código 29, los dos valores están separados por 2 unidades, dentro de un *array* de longitud 3, por lo que, aplicando el cálculo definido en la Ecuación 11, la puntuación obtenida sería 0%, penalizando por completo los valores en extremos opuestos.

Caso 2

Preferencias usuario → *ambiente: tranquilo*

Características anuncio → *ambiente: animado*

En este caso, la distancia entre los valores es solamente de 1 unidad, por lo que la penalización esta vez, aplicando de nuevo la Ecuación 11, sería de un 50%.

Caso 3

Preferencias usuario → *ambiente: tranquilo*

Características anuncio → *ambiente: indiferente*

En este caso, al ser uno de los dos valores indiferente, no haría falta siquiera entrar en cálculos, aplicándose automáticamente un 100% en la puntuación de este campo interno.

De esta forma, podemos claramente comprobar cómo el funcionamiento del motor está pensado para potenciar aquellos valores que estén lo más cercanos posible en el espacio, ya que se considera que serán más compatibles, sobre el papel.

- Cálculo de la puntuación matemática global para anuncios

Una vez obtenidas todas las puntuaciones anteriores, la función *calcular_score_vivienda* se encarga de obtener la puntuación total como una media ponderada de dos niveles con renormalización dinámica.

El método se encarga de recorrer cada módulo de los datos obtenidos del usuario y del anuncio y, dentro de él, cada campo interno. Para cada uno de ellos, se consulta al enrutador para obtener la función correspondiente, se ejecuta y se acumula la nota multiplicada por su peso interno, llevando además la cuenta del peso realmente activo. El funcionamiento se puede observar en el Código 6.

```

1 nota_caracteristica = funcion(valor_usuario, valor_anuncio) # 0-100
2 score_interno_acumulado += nota_caracteristica * (peso_interno / 100.0)
3 peso_interno_activo      += peso_interno

```

Código 6. Pseudocódigo de la acumulación de pesos en recomendaciones de anuncios

Si el usuario no ha definido alguna de las características (*valor_usuario = None*), se omite con *continue* y su peso no entra en *peso_interno_activo*.

Al terminar un módulo, se obtiene su nota dividiendo por el peso interno activo, como se muestra en el fragmento definido en el Código 7.

```

1 nota_modulo = (score_interno_acumulado / peso_interno_activo) * 100.0

```

Código 7. Cálculo de la nota de un módulo en recomendaciones de anuncios

Esto, realmente equivale al cálculo de la media ponderada sobre las características presentes, como se muestra en la Ecuación 12.

$$nota_{modulo} = \frac{\sum_{i=1}^n nota_i \times peso_i}{\sum_{i=1}^n peso_i}$$

Ecuación 12. Cálculo de la puntuación de cada módulo

De la misma forma que ocurre con los campos internos, para calcular la puntuación global, la nota de cada módulo se pondera por el peso del mismo, acumulando solo los pesos activos, como se muestra en el Código 8 y la Ecuación 13.

```

1 score_total_acumulado += nota_modulo * (peso_del_modulo / 100.0)
2 peso_modulo_activo    += peso_del_modulo
3
4 ...
5 score_matematico = (score_total_acumulado / peso_modulo_activo) * 100.0

```

Código 8. Cálculo del score matemático para recomendaciones de anuncios

$$Score_{matemático} = \frac{\sum_{i=1}^n notaModulo_i \times pesoModulo_i}{\sum_{i=1}^n pesoModulo_i}$$

Ecuación 13. Cálculo del score matemático en anuncios

La clave del algoritmo; cumpliendo con una de las decisiones tomadas durante el desarrollo del Sprint 1, sobre evitar abrumar al usuario pidiéndole una cantidad ingente de datos y formularios por rellenar; es la renormalización dinámica de los resultados, haciendo que la ausencia de valores en campos internos de las características de un usuario no penalicen en la puntuación obtenida, sino que el peso se redistribuye proporcionalmente entre los que sí están definidos. El resultado queda siempre acotado en el rango [0,100], independientemente de cuántos campos haya rellenado el usuario.

- Ejemplo de cálculo de puntuación de un anuncio

A continuación, en la Tabla 49, se muestra un ejemplo del cruce de las características de un usuario con los valores de un piso. Nótese que no están presentes todos los campos, por lo que el usuario ha dejado campos sin definir en este ejemplo.

Módulo(peso)	Interno (peso)	Usuario	Anuncio	Explicación	Nota
economia_contrato (40)	gastos_incluidos (50)	agua, internet	agua	1 de 2	50
	estancia_minima_meses (30)	6	6	6 de 6	100
	meses_fianza_max (20)	1	2	+1 mes · 25	75
convivencia_normas (30)	ambiente (40)	estudio	animado	Distancia 2 de 2	0
	limpieza (30)	turnos_estri ctos	a_menudo	Distancia 1 de 2	50
	visitas (15)	ocasional	ocasional	Coinciden	100
	ocupación_inquilinos (9)	estudiante	estudiante	Coinciden	100
	genero_inquilino (4)	indiferente	mujer	Indiferente (comodín)	100
	lgtb-friendly (2)	sí	sí	coincide	100
Características_vivie nda (15)	Tipo_inmueble (40)	piso, ático	piso	1 de 2	50
	banos_minimos (30)	2	3	Más de lo que pide	100

Tabla 49. Ejemplo de recomendación de anuncio

- Cálculos de puntuaciones

$$\text{Nota economía} = 50 \cdot 0,5 + 100 \cdot 0,3 + 75 \cdot 0,2 = 70$$

$$\text{Nota convivencia} = 0 \cdot 0,4 + 50 \cdot 0,3 + 100 \cdot 0,15 + 100 \cdot 0,09 + 100 \cdot 0,04 + 100 \cdot 0,02 = 45$$

Nota características = aquí el usuario solo rellenó 2 de los 4 atributos (faltan climatización y ascensor), así que el sistema renormaliza sobre los pesos activos (40+30=70): $(50 \cdot 0,4 + 100 \cdot 0,3) / 0,70 = 71,43$

El módulo habitación no se rellenó, así que también se descarta y la media global se renormaliza sobre los módulos activos (40+30+15 = 85): Score matemático = $(70 \cdot 0,4 + 45 \cdot 0,3 + 71,43 \cdot 0,15) / 0,85 = 61,43$

$$\text{Score final} = 61,43 \cdot \alpha + 50 \cdot \beta = 61,43 \cdot 0,8 + 50 \cdot 0,2 = 59,14$$

6.5.4 Compañeros

Para la implementación del sistema de recomendaciones sobre posibles compañeros, se han realizado varias modificaciones en el planteamiento del problema y, por tanto, de la solución escogida para resolverlo. Esto se debe principalmente a la diferencia en la naturaleza del problema a resolver, ya que, a pesar de ambos tener la misma finalidad; recomendar al usuario basado en sus preferencias e interacciones; la direccionalidad no es la misma.

- Naturaleza del problema. La bidireccionalidad

A diferencia de los anuncios, donde la relación definida es unidireccional; el inmueble ofrecido tiene que ser compatible con las preferencias del usuario, lo que ya garantiza que el usuario sea compatible con las características del anuncio; el caso de los compañeros es bidireccional. Supongamos que tenemos un usuario A y un usuario B; por un lado, el usuario B debe ser compatible con lo que el usuario A está buscando, pero eso no garantiza que lo que el usuario A ofrece sea compatible con lo que el usuario B busca.

En este caso, tendríamos una doble comprobación que antes no era necesaria; si se cruzan las preferencias de un usuario con las características de un anuncio y se obtiene una puntuación total del 90%, significa que ese usuario y ese anuncio son altamente compatibles sobre el papel.

Por otro lado, si se cruzan las preferencias de un usuario A con las características de un usuario B, se puede obtener una gran puntuación y asumir que el usuario B es compatible con lo que

el usuario A busca. No obstante, al hacerlo de manera inversa, podría ocurrir que el usuario A no se asemeje tanto a lo que el usuario B está buscando y, por tanto, no ser compatibles finalmente.

Por tanto, como queda claro, este problema no se puede abordar de la misma manera que se aborda el caso de los anuncios, sino que deben realizarse ciertas modificaciones en el planteamiento de su solución.

- Resolución del problema

Para la resolución del problema planteado, vamos a tener, en primer lugar, un modelado de datos basado en la dualidad de los perfiles de los usuarios y un cálculo de sus afinidades adaptado a estas características.

- Modelado de datos: dualidad del perfil

Para resolver este cruce, el sistema modela a cada usuario en un espacio multidimensional de características de convivencia, basadas en las definidas en el punto Valoraciones, asignando un valor numérico a cada uno de estos apartados, que conformarán las coordenadas de cada uno de los siguientes vectores.

- Las preferencias del usuario (lo que busca): se obtienen mediante un formulario ofrecido al usuario.
- La realidad (lo que ofrece): representa lo que el usuario aporta realmente a la convivencia. Se construye combinando dos fuentes: la autovaloración inicial del usuario, obtenida mediante el test de personalidad, y las valoraciones recibidas de otros usuarios con los que ha convivido. Ambas se integran mediante una media bayesiana, de modo que el peso de las valoraciones reales crece a medida que el usuario acumula histórico, como se detalla en el siguiente punto.

- Test de personalidad

La realidad de qué ofrece un usuario es fácilmente identificable en este sistema, gracias al sistema de valoraciones ofrecido, donde podremos ver qué opinan los antiguos compañeros y compañeras de un usuario sobre la convivencia con ellos. Sin embargo, aquí nos encontramos con el problema de todo sistema de recomendaciones una vez más, el arranque en frío. No se pueden ofrecer valoraciones de un perfil hasta que ese usuario haya convivido con alguien antes y esos usuarios le hayan dejado una valoración. Para solventar este problema, se plantea un pequeño test al usuario para responder las preguntas definidas en el punto Valoraciones calificándolas del 1 al 5, como se puede comprobar en el Código 9. Con esto, tenemos una realidad (la del mismo usuario) sobre qué ofrece una persona en la convivencia. Obviamente, esta aproximación en muchos casos podrá no representar fielmente la realidad de la convivencia con ese usuario, pero es entonces cuando entran en juego las valoraciones del resto de los usuarios de la plataforma, equilibrando lo que este realmente ofrece.

```

1  {
2      "higiene_y_orden": {
3          "frecuencia_limpieza": 5,
4          "gestion_vajilla": 4,
5          "zonas_comunes": 5
6      },
7      "economia_y_responsabilidad": {
8          "puntualidad_pagos": 5,
9          "gastos_compartidos": 4,
10         "consumo_responsable": 3
11     },
12     "ruido_y_horarios": {
13         "respeto_descanso": 5
14     },
15     "visitas": {
16         "frecuencia_visitas": 2,
17         "respeto_zonas_comunes": 4
18     },
19     "comunicacion": {
20         "resolucion_conflictos": 4,
21         "empatia": 5,
22         "expresividad": 3
23     },
24     "respeto_propiedad": {
25         "cuidado_mobiliario": 5
26     }
27 }

```

Código 9. JSON Características usuarios

- Equilibrado del vector de realidad

Como se ha comentado anteriormente, la autovaloración del test constituye únicamente una aproximación inicial: difícilmente coincide con la percepción que tienen de un usuario quienes han convivido con él. Por ello, el vector de realidad no se fija con los valores del test ni se sobrescribe por completo con las valoraciones recibidas, sino que ambas fuentes se equilibran mediante una media bayesiana (o estimador por contracción), tratando la autovaloración como un valor a priori (*prior*) y las valoraciones de otros usuarios como evidencia. Para cada criterio de convivencia c , el valor del vector de realidad se calcula según la Ecuación 14.

$$Realidad_c = \frac{w \times test_c + \sum_{j=1}^n p_{cj}}{w + n}$$

Ecuación 14. Cálculo del equilibrado del vector de realidad

Donde

$test_c$: puntuación del criterio c en la autovaloración del test.

p_{cj} : puntuación del criterio c en la j -ésima valoración recibida de otros usuarios.

n : número de valoraciones.

w : peso asignado al prior, expresado como el número de valoraciones reales equivalentes que pesa la autovaloración.

Este planteamiento resuelve simultáneamente dos problemas. En el arranque en frío, cuando el usuario aún no ha recibido valoraciones ($n_c = 0$), el valor se reduce a su autovaloración ($realidad_c = test_c$), garantizando que el sistema disponga de un vector funcional desde el primer momento. A medida que el usuario acumula valoraciones reales, el término de la evidencia gana peso y la autovaloración se diluye progresivamente, de modo que el vector converge hacia la percepción

real del resto de usuarios sin que una única valoración temprana pueda distorsionarlo de forma brusca. Se ha fijado $w = 3$, lo que implica que, a partir de aproximadamente tres valoraciones recibidas, la opinión externa pesa tanto como la autovaloración; al igual que el resto de los parámetros del sistema, es una decisión de diseño ajustable empíricamente con datos reales.

Para garantizar que este vector se mantenga siempre actualizado sin recalcularlo en cada recomendación, su cómputo se ha delegado a un *trigger* de PostgreSQL (Código 27) que se ejecuta ante cualquier inserción, modificación o borrado en la tabla *Puntuacion_Criterios*. El *trigger* recupera la autovaloración almacenada en la columna *vector_test* de la tabla *Inquilino*, la combina con las valoraciones recibidas de otros usuarios según la Ecuación 14 y reconstruye el vector de realidad en formato JSONB, respetando la misma estructura anidada que el vector de preferencias para que ambos sean comparables por el motor de recomendación. De esta forma, la lógica de actualización reside en la propia base de datos y el microservicio se limita a leer un vector ya consolidado.

Ejemplo

Supongamos que un usuario se autovalora con un 5 en el criterio *frecuencia_limpieza* mediante el test de personalidad y, más adelante, recibe una primera valoración de 2 en ese mismo criterio. Su vector de realidad pasaría a $(3 \cdot 5 + 2)/(3 + 1) = 4,25$; tras tres valoraciones de 2, 2 y 3, descendería a $(3 \cdot 5 + 7)/(3 + 3) = 3,67$, acercándose poco a poco a la valoración percibida por sus compañeros.

- Cálculo de la afinidad bidireccional

Para calcular la puntuación total de afinidad entre dos usuarios, se debe realizar el mismo cálculo dos veces, pero de manera inversa.

- Cálculo de la afinidad de un usuario hacia otro

Supongamos que tenemos dos usuarios a comparar, el usuario A y el usuario B. Como ya hemos comentado, cada usuario cuenta con dos vectores asociados, de manera que, para calcular la afinidad de B hacia A, debemos cruzar el vector de preferencias de A con el vector de preferencias de B, midiendo la distancia existente entre ambos y viendo, por tanto, su distancia.

Para este caso concreto, se ha hecho uso de la distancia euclidiana (Ecuación 15), que mide la distancia de dos vectores en línea recta mediante el teorema de Pitágoras, penalizando de forma cuadrática las grandes diferencias en un rasgo específico [111].

$$d(P, Q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

Ecuación 15. Cálculo de la Distancia Euclidiana

Donde:

- P es el vector de Preferencias del usuario evaluador.
- Q es el vector de Realidad del usuario a evaluar.
- n es el número total de criterios de convivencia evaluados.

Dado que el objetivo es presentar la compatibilidad entre usuarios en una medida entendible, este resultado es normalizado siguiendo la fórmula definida en la Ecuación 16.

$$Afinidad_{A \rightarrow B} = \left(1 - \frac{d}{d_{Max}}\right) \times 100$$

Ecuación 16. Cálculo de la afinidad entre usuarios

Donde:

- d : es la distancia calculada entre el vector de preferencias de A con el vector de realidad de B mediante la fórmula del Cálculo de la Distancia Euclidiana.

- D_{Max} : es la distancia máxima posible entre un vector y otro, que en este caso sería calculada mediante la fórmula definida en la Ecuación 17.

$$d_{Max} = \sqrt{n \times 4^2} = \sqrt{n \times 16}$$

Ecuación 17. Cálculo de la Distancia Máxima

Donde:

- n es el número total de criterios de convivencia evaluados.
- Como sabemos que la puntuación de cada aspecto de la convivencia (o, dicho de otra manera, las coordenadas de cada vector) son valores numéricos que van del 1 al 5, la diferencia máxima entre cada una de ellas será de 4 unidades.

- Cálculo de la afinidad real

Una vez que tenemos el nivel de compatibilidad entre dos usuarios, necesitamos un método para calcular su afinidad real. Para este cálculo se ha decidido usar la media armónica, que castiga fuertemente la nota final si una de las dos puntuaciones es muy baja [112]. La fórmula de la media armónica la vemos en la Ecuación 18.

$$H = \frac{n}{\sum_{i=1}^n \frac{1}{x_i}}$$

Ecuación 18. Cálculo de la Media Armónica

Donde:

- n es el número de elementos.
- x_i son los valores a tener en cuenta para el cálculo de la media armónica.

Aplicando este cálculo al caso de uso descrito, obtenemos la siguiente fórmula (Ecuación 19) para el cálculo de la puntuación final de afinidad entre dos usuarios:

$$Puntuación\ final = \frac{2 \times Afinidad_{A \rightarrow B} \times Afinidad_{B \rightarrow A}}{Afinidad_{A \rightarrow B} + Afinidad_{B \rightarrow A}}$$

Ecuación 19. Cálculo de la puntuación final entre usuarios

Se ha elegido la media armónica en lugar de la aritmética o geométrica precisamente por esa penalización ante los extremos bajos, ya que, siguiendo el principio empleado por Van Rijsbergen para construir la métrica F1 [113], cuando se combinan dos dimensiones complementarias e igualmente necesarias, la media aritmética enmascara incompatibilidades severas en una de ellas, mientras que la media armónica las hace explícitas. Si nos fuéramos a un caso extremo; donde la afinidad del Usuario A con el Usuario B fuera de un 88%, mientras que la afinidad del Usuario B con el Usuario A fuese, por ejemplo, de un 25%; la media aritmética nos daría un resultado final del 56,5% de afinidad, lo que supondría una compatibilidad de algo más de la mitad de la puntuación posible entre ambos usuarios; mientras que la media geométrica nos daría una compatibilidad del 46,9%, lo que también es bastante cercano al 50%, aunque la puntuación se ve más penalizada en este caso. Sin embargo, si ese mismo caso lo aplicamos a la configuración establecida (con la media armónica), el resultado obtenido sería del 38,9%, reflejando una baja compatibilidad entre ellos y que, sobre el papel, la convivencia sería un desastre. De esta forma, el algoritmo implementado establece un mecanismo de seguridad, obligando a que, para que dos usuarios tengan una recomendación alta en la plataforma, la compatibilidad debe ser sólida y recíproca en ambas direcciones.

6.5.5 Gestión de la memoria

Para evitar duplicar el modelo de lenguaje en la memoria RAM, que supone una cantidad considerable de espacio y una latencia notable al cargarlo desde disco, el modelo NLP no se instancia bajo demanda en cada petición, sino que se carga una única vez en el atributo *self.modelo_nlp*, durante

la construcción del motor de recomendación (siguiendo el patrón *Singleton* [114]), en el momento en el que FastAPI importa el módulo *engine*, de modo que el modelo queda residente en memoria durante todo el ciclo de vida del servicio.

La afinidad semántica entre compañeros no recibe, por tanto, una instancia propia del modelo, sino que reutiliza por composición la que ya reside en el motor de recomendación de viviendas: durante el cálculo del *ranking*, se invoca *motor_global.evaluar_semantica(...)* para obtener la puntuación del NLP y esta se transfiere como argumento a *motor_social.scoring_usuarios(...)*. De esta forma, ambos motores comparten una única instancia del modelo NLP en memoria, evitando copias innecesarias que empeorarían el rendimiento del sistema.

En el fragmento mostrado en el Código 10, se puede ver el mecanismo utilizado, instanciando una única vez cada motor (el modelo NLP se carga en la construcción de *MotorRecomendacion*).

Posteriormente, como se muestra en el Código 11, dentro del cálculo del *ranking* de compañeros, el *Score NLP* se obtiene del motor que ya tiene el modelo cargado y se inyecta por composición en el motor social.

```
1 # engine.py - instanciación única a nivel de módulo (Singleton)
2 motor_global = MotorRecomendacion(penalizacion_unidades=25) # carga el
3 modelo NLP en RAM
4 motor_social = MotorCompaneros() # solo lógica
5 algebraica
6
```

Código 10. Instanciación de motores de recomendaciones

```
1 # engine.py - reutilización del modelo ya residente en memoria
2 # Score NLP aprovechando el modelo que ya está en RAM (no se vuelve a
3 cargar)
4 score_nlp = motor_global.evaluar_semantica(descripcion_usuario, desc_b)
5
6
7 # El Score NLP se transfiere como argumento al motor social (Score
8 Matemático + NLP)
9 score_base = motor_social.scoring_usuarios(
10     perfil_social_a, real_a, perfil_social_b, real_b, score_nlp
11 )
12
```

Código 11. Inyección por composición del modelo NLP en motor social

6.5.6 Diferencias clave entre ambos motores

Como se ha comentado a lo largo de este capítulo, ambos motores de recomendación implementados tienen ligeras diferencias entre sí, pese a compartir gran parte de la lógica diseñada. En la Tabla 50 pueden observarse claramente las diferencias entre ambos motores.

Aspecto	Motor de anuncios	Motor de compañeros
Relación	Unidireccional (usuario → Anuncio)	Bidireccional (Usuario ↔ Usuario)
Base matemática	Cruce de preferencias del usuario con características del anuncio	Distancia euclídea entre vectores de preferencias y realidad de los usuarios
Score final	Puntuación del anuncio para el usuario	Media armónica de las dos puntuaciones obtenidas
Filtros barrera	Presupuesto, ciudad, mascotas	Ciudad
Cold start	Preferencias del usuario y características del anuncio	Test de personalidad como aproximación inicial a la realidad del usuario
Modelo de datos	Un único vector por entidad	Dos vectores por usuario: preferencias y realidad

Tabla 50. Diferencias entre motores de recomendación

6.5.7 Reflejo en el *frontend* de las recomendaciones calculadas

En este apartado se muestran capturas de la pantalla *home* del usuario con rol de Inquilino, con las recomendaciones generadas por el sistema.



Ilustración 26. Home del inquilino con recomendaciones



Ilustración 27. Pop Up para test de personalidad

6.5.8 Desviaciones y complicaciones

Durante la implementación de este módulo han surgido diversas complicaciones y contratiempos no tenidos en cuenta desde el principio.

En primer lugar, no ha sido hasta el planteamiento de la solución del sistema de recomendaciones que se han abordado (y descubierto) las diferencias en las direccionalidades de ambos problemas, lo que ha supuesto la necesidad de implementar dos sistemas de recomendación específicos para un problema que, de primeras, se planeaba resolver con una única solución, por lo que ha sido necesario adaptar la implementación de manera que ambos sistemas compartan la mayor parte de la lógica y el código.

Por otro lado, durante el desarrollo del sprint ha surgido la idea de incluir un Test de personalidad para solventar el problema del arranque en frío para las recomendaciones de usuarios, una solución que no estaba clara hasta la aparición real del problema una vez comenzada su implementación.

Con todas estas funcionalidades ya en uso, el siguiente paso natural es la implementación de un sistema de búsqueda basado en los filtros definidos a lo largo de este capítulo.

6.6 Sistema de búsqueda (Sprint 5)

Partiendo de la base del sistema de recomendaciones, el siguiente módulo se centra en la implementación de un sistema de búsqueda, de manera que el usuario pueda realizar una búsqueda personalizada según sus parámetros y sea el propio sistema de recomendaciones el que se encargue de extraer la lista de usuarios o anuncios, según el caso, que más se adapte a los filtros introducidos.

El sistema de búsqueda de la aplicación se ha diseñado de forma que se dé una separación entre la pantalla principal de recomendaciones del sistema (en el *home*) y las búsquedas exploratorias por parte del usuario. Esto permite a los usuarios realizar consultas puntuales sobre

anuncios o compañeros con ciertas características y en determinadas zonas o ciudades sin sobrescribir las preferencias almacenadas en su perfil.

6.6.1 Contratiempos y modificaciones necesarias en el sistema

Durante la implementación del sistema de búsqueda de la aplicación han surgido varias necesidades no planteadas anteriormente.

- Columna ciudad en las tablas Inquilino y Vivienda

Pese a ser la ciudad un atributo bastante importante a la hora de emparejar diferentes entidades del sistema entre sí, hasta ahora no había sido realmente necesaria, ya que el motor de recomendaciones del sistema trabaja en función de un vector de códigos postales, más que de nombres de ciudades o barrios. No obstante, a la hora de buscar, surge la necesidad de poder especificar en qué ciudad, zona o barrio se está interesado. Debido a esto, se añade la columna Ciudad a ambas tablas, como cadena de texto de 50 caracteres como máximo.

- Problemas con el componente GooglePlacesAutocomplete

La integración del componente de Google ha generado problemas de integración debido a pequeños detalles en la implementación, debido al sistema operativo del dispositivo de pruebas para el desarrollo del proyecto (iOS), donde se presentan incompatibilidades con ciertos componentes de React Native, que no permiten el correcto funcionamiento del sistema sin añadir ciertas *props*, como se puede ver en el Código 31.

6.6.2 Conexiones externas y APIs necesarias para la implementación

El diseño del sistema de búsqueda parte de una premisa relativa a la experiencia de usuario: no resulta adecuado exigir al usuario la introducción de una dirección postal concreta manualmente, dado que la probabilidad de que existan anuncios localizados exactamente en dicha dirección es reducida, haciendo además necesaria la exactitud semántica entre la dirección almacenada en el anuncio y la introducida por el usuario. Análogamente, una búsqueda a nivel de municipio que devuelva la totalidad de los anuncios de una ciudad resultaría poco útil, puesto que el usuario suele estar interesado en barrios o zonas determinadas y no en la ciudad al completo.

A partir de esta necesidad, se ha llevado a cabo una búsqueda de servicios externos y APIs capaces de proporcionar un sistema de búsqueda por zonas. La implementación manual de esta funcionalidad; que requeriría la construcción y el mantenimiento de un *dataset* geográfico propio con la delimitación de barrios y zonas; se ha descartado por resultar demasiado compleja e impracticable en el marco temporal de este proyecto.

La diferencia entre la implementación esperada y la obtenida en esta primera versión funcional del sistema de búsqueda es que, para buscar un barrio cualquiera, el usuario tendrá que saber de antemano qué barrio concreto quiere buscar en la aplicación, con el sistema dándole sugerencias respecto al texto introducido en ese momento. La implementación óptima y programada para desarrollar en un futuro sería que, al buscar una ciudad, zona o barrio, el sistema diese sugerencias concretas en relación con esa búsqueda (barrios, zonas, etc.).

- Servicios externos y APIs escogidas

Para la selección del software se ha realizado un estudio comparativo de los distintos productos con capacidad de ofrecer los servicios requeridos por el sistema. En la Tabla 51 se recogen los criterios de evaluación empleados y el comportamiento de cada alternativa frente a ellos. Las alternativas consideradas han sido las siguientes:

Google Maps Platform (Places API) [115]

Mapbox [116]

HERE [117]

OpenStreetMap (junto con bibliotecas como Leaflet) [118], [119]

Criterio	Google Maps Platform	Mapbox	HERE	Leaflet + OpenStreetMap
Webs activas que lo usan	Alrededor de 2,5 millones	Alrededor de 273 mil	Alrededor de 10 mil	Alrededor de 517 mil
Base de datos	La más completa del mundo, cobertura en más de 200 países y más de 200 millones de lugares	Basada en datos abiertos (OSM, Microsoft Open Maps, Wikidata); menor densidad de POIs comerciales	Fuerte en automoción/logística, más débil en POIs de consumo	Datos colaborativos (crowdsourced), que pueden ser menos detallados o actualizados
Búsqueda de lugares y autocompletado	Nativo (Places API: Text Search, Nearby Search, Autocomplete, Place Details)	Geocoding API con autocompletado, menos rico en metadatos (horarios, reseñas, fotos)	Geocoding & Search API (autocomplete, discover, browse), orientada a logística y con menos POIs de consumo	Leaflet solo renderiza mapas; la búsqueda/geocodificación requiere Nominatim u otros, sin autocompletado nativo y con uso público limitado
Modelo de coste	Pago por uso por SKU; desde marzo de 2025, cuota mensual gratuita por SKU (10.000 Essentials / 5.000 Pro / 1.000 Enterprise) en lugar del antiguo crédito de 200 \$; SDK móvil gratuito; ~2-40 \$/1.000 según SKU	Pago por uso con plan gratuito amplio (~50.000 cargas de mapa y ~100.000 peticiones de geocodificación temporal al mes); después, desde ~0,75 \$/1.000	Plan Freemium con ~250.000 transacciones gratis al mes; después ~1 \$/1.000; plan Pro (~449 \$/mes, 1 millón)	Sin coste de licencia; el Nominatim público está limitado y un uso en producción exige autoalojar el servicio (servidor de gran capacidad e infraestructura propia)
Licencia/restricciones	Propietaria; requiere clave de API y facturación activa; sus condiciones prohíben cachear los resultados de geocodificación y mostrarlos sobre mapas que no sean de Google	Propietaria (SDK y condiciones de uso), aunque construida sobre datos abiertos; no permite almacenar los resultados salvo con la Permanent Geocoding API (de pago); pueden usarse sobre cualquier mapa	Propietaria; los planes estándar no permiten el almacenamiento permanente de los resultados de geocodificación	Abierta: datos de OSM bajo ODbL (atribución y compartir-igual) y Leaflet bajo licencia BSD permisiva
SDKs y sistema	Maps SDK para Android	Mapbox GL JS (web), Maps y Navigation SDK para	HERE SDK para Android, iOS y	Leaflet es una librería JS para

	e iOS, Maps JavaScript API y servicios web; amplio ecosistema y soporte en React Native	iOS/Android y librerías para React Native	Flutter, además de API JavaScript y servicios REST	web; en móvil no hay SDK oficial (se recurre a react-native-maps con teselas de OSM o a WebView)
Beneficio para StayIn	Mejor opción: la base de POIs más completa y la búsqueda/autocompletado más rica, idóneas para la búsqueda por zonas; estándar de la industria y fiable	Gran personalización visual, pero catálogo de lugares menos completo para la búsqueda por zonas	Orientada a logística y automoción, un dominio alejado de las necesidades de la aplicación	Coste nulo, pero traslada al desarrollador la calidad de los datos y el mantenimiento de la geocodificación, incompatible con las necesidades de fiabilidad

Tabla 51. Comparación de servicios externos para sistema de búsqueda

El comportamiento esperado de este módulo, tal como se ha expuesto, consiste en proporcionar al usuario un sistema de búsqueda asistida: al introducir una ciudad, un barrio o una zona, el sistema debe devolver sugerencias de zonas pertinentes. A modo de ejemplo, ante la entrada «Granada», el sistema debería ser capaz de sugerir sus distintos barrios (Centro, Zaidín, Pajaritos, etc.).

No obstante, la implementación completa de esta funcionalidad requiere del uso de una base de datos geográfica externa; que debería ser importada a la base de datos de la aplicación; así como el diseño de una búsqueda compuesta por varias capas. Estos requisitos incrementan de forma notable la complejidad y el tiempo de desarrollo del módulo. Por este motivo, se ha optado por desarrollar una versión simplificada y funcional para el alcance del presente proyecto, posponiendo la implementación definitiva basada en una base de datos externa como línea de trabajo futuro, previa al despliegue de una versión de producción de la aplicación.

- Justificación de la elección

Una vez acotado el alcance, la elección entre los servicios externos accesibles mediante API ha sido relativamente sencilla, debido a la consolidación de la API de Google como estándar de la industria para la integración de mapas y datos de lugares, superando en un orden de magnitud a la del siguiente proveedor comercial, con más de dos millones de aplicaciones y sitios web activos que emplean los productos principales de Google Maps Platform [120], [121]. En particular, la Places API proporciona la base de datos de puntos de interés más extensa del mercado, con cobertura en más de doscientos países [122], lo que la convierte en la opción idónea para el motor de búsqueda de esta aplicación.

Frente a ella, las alternativas evaluadas presentan limitaciones para el caso de uso concreto, tal como se resume en la Tabla 51. Mapbox, pese a ofrecer una gran capacidad de personalización visual, dispone de un catálogo de lugares menos completo, al estar construido sobre fuentes de datos abiertas [116]. HERE Technologies, por su parte, se orienta principalmente al ámbito de la logística y la automoción [117], un dominio alejado de los requisitos de la aplicación. Finalmente, la combinación de OpenStreetMap y Leaflet, aunque elimina los costes de licencia, traslada al desarrollador la responsabilidad sobre la calidad y la actualización de los datos, así como sobre el mantenimiento de la infraestructura de geocodificación [118], [119], lo que no cumple con las necesidades de fiabilidad de este proyecto.

6.6.3 Arquitectura e implementación del sistema de búsqueda

El sistema de búsqueda de la aplicación se ha diseñado bajo un paradigma de "Consultas Volátiles", separando conceptualmente el *feed* principal de recomendaciones (basado en preferencias persistentes) de las búsquedas exploratorias. Esto permite a los usuarios realizar consultas puntuales en diferentes ciudades o bajo distintas restricciones económicas y sociales sin sobrescribir su perfil permanente en la base de datos.

La implementación atraviesa las tres capas principales del sistema: *Frontend* móvil, integración de APIs de terceros y el Motor de Inteligencia Artificial en el *Backend*.

- Interfaz de usuario del sistema de búsqueda

La interfaz del sistema de búsqueda se ha construido con el objetivo de ofrecer una experiencia de usuario fluida e intuitiva, dividida en búsqueda de anuncios y búsqueda de compañeros.

- Autocompletado geográfico (Google Places API)

Se ha integrado la librería *react-native-google-places-autocomplete* operando dentro de un componente *Modal* a pantalla completa, evitando así problemas de renderizado generados por la implementación de una lista (las sugerencias de la barra de búsqueda) dentro de un componente *ScrollView*, problema generado en Expo al anidar listas deslizables con la misma orientación, en este caso vertical. Para garantizar que los resultados sugeridos por el componente sean estrictamente ciudades, pueblos y barrios, evitando puntos de interés, se ha aplicado el filtro *types: 'geocode'* restringido al territorio español (*components: 'country:es'*). La implementación del componente se puede ver en el Código 31.

- Gestión de valores nulos

Se han incorporado medidas de protección ausentes en la primera implementación del motor de recomendaciones, de manera que la ausencia de valores en campos a comparar, como un *NULL* proveniente de la base de datos en la descripción, por ejemplo, no den lugar a un error que produzca la caída de todo el servicio de recomendación y, consecuentemente, del sistema de búsqueda.

Con la implementación del sistema de búsqueda, queda prácticamente cerrada la fase de implementación de funcionalidades de la aplicación en sí, dando paso al desarrollo de un panel de administración que permita gestionar los aspectos clave del sistema de forma sencilla, cumpliendo con todas las restricciones y funcionalidades descritas a lo largo del documento.

6.7 Panel de administración (Sprint 6)

El módulo de administración supone un aspecto crítico en el funcionamiento de StayIn, proporcionando un entorno seguro y centralizado para la supervisión, gestión y moderación de las principales entidades del sistema: anuncios, usuarios y valoraciones. Se ha desarrollado siguiendo principios de separación de responsabilidades, seguridad por capas y optimización de la experiencia de usuario.

6.7.1 Seguridad y control de acceso

Dada la sensibilidad de las operaciones disponibles en este panel, se ha llevado a cabo la implementación de una doble capa de seguridad (doble *middleware*) para controlar el acceso al servicio de administración únicamente a aquellos usuarios con los correspondientes permisos. La implementación de esta doble capa se ha basado en el uso de *JSON Web Tokens (JWT)*.

- Protección en el *backend*

Se ha diseñado un *middleware* específico (*verificarAdmin*) para trabajar junto al ya implementado para usuarios (*verificarToken*), de manera que, mediante la decodificación del token a través del objeto *req.user*, se verifique el rol del usuario actual. En caso de que el rol coincida exactamente con la cadena "Administrador", se concederá el acceso. En caso contrario, el sistema

rechaza la petición con un código *HTTP 403 (Forbidden)*, impidiendo así el acceso al panel de administración para todo usuario que no esté registrado como administrador del sistema.

- Gestión de sesión en el *frontend*

Para garantizar que el token de sesión del usuario viaje de forma segura y automatizada en cada petición al servidor, se ha hecho uso de la librería *Axios* [123], de JavaScript, configurando una instancia de un cliente HTTP de *Axios* (*adminApi*), permitiendo así la inyección del token en las cabeceras de las peticiones de manera dinámica.

- Creación del primer usuario administrador

Debido a la naturaleza del administrador, el registro de un usuario de este tipo no se puede realizar como la de los caseros e inquilinos, mediante un formulario pidiendo los datos directamente al usuario. Tampoco se puede simplemente añadir un usuario a través de una sentencia SQL en la base de datos introduciendo la contraseña en texto plano, ya que almacenarla de esa manera en la tabla Usuarios supondría un problema grave de seguridad, además de una violación de la privacidad de datos. Por estos motivos, para la creación del primer administrador del sistema, se ha realizado la encriptación de la contraseña mediante *bcrypt* a través de la consola de node, insertando, ahora sí, esa contraseña encriptada y segura mediante una sentencia SQL en la base de datos, garantizando la seguridad y consistencia de esta y dando lugar al primer administrador de la plataforma. Este problema solamente se da para el primer caso, ya que mediante el acceso a esa sesión se podrán crear más usuarios con el mismo rol.

6.7.2 Componentes del panel de administración

El panel de administración facilita la gestión del sistema para aquellos usuarios autorizados a ello, por lo que debe mostrar todas las características y datos claves del sistema. En este caso, el panel de administrador está dividido en los siguientes submódulos.

- Dashboard

Es la primera vista del módulo de administración una vez iniciada la sesión. Muestra unas estadísticas simples del sistema, como son el número de usuarios registrados en él, el número de anuncios publicados y el número de usuarios de cada rol registrados. La funcionalidad ahora mismo es básica, con la intención de mejorarlo a futuro, mostrando estadísticas complejas y en tiempo real del sistema, para lo cual es necesaria una mayor carga de usuarios en la aplicación.

A nivel de implementación, se realiza un *COUNT* de todas las tablas en cuestión (Usuarios, Anuncios y Valoraciones), de manera concurrente mediante *Promise.all()*, como se puede ver en el Código 12, reduciendo así el tiempo de respuesta del servidor y devolviendo un objeto JSON consolidado.

```
1  const [resUsuarios, resAnuncios, resInquilinos, resCaseros] = await
2  Promise.all([
3
4      client.query('SELECT COUNT(*) FROM Usuarios'),
5      client.query('SELECT COUNT(*) FROM Anuncios'),
6      client.query('SELECT COUNT(*) FROM Inquilino'),
7      client.query('SELECT COUNT(*) FROM Caseros')
8  ]);
```

Código 12. Consulta datos Dashboard administrador

- Gestión de usuarios

El submódulo de usuarios permite la creación, eliminación y búsqueda de usuarios en la aplicación, mostrando los datos más relevantes de estos, como son: id, nombre, apellidos, nombre de usuario, correo asociado, rol, fecha de nacimiento y género.

- Creación e integridad de los datos

El proceso de crear un nuevo usuario a través del panel de administración se lleva a cabo mediante un modal dedicado exclusivamente a la introducción de los datos solicitados, asignando un nombre, apellidos, nombre de usuario, rol, contraseña, fecha de nacimiento y género. Este formulario garantiza la introducción correcta de todos los datos mediante la validación nativa de HTML 5. En el *backend* se hace uso de un *endpoint* nuevo muy similar al ya existente para el registro de usuarios en el sistema a través de la aplicación, garantizando la correcta inserción mediante las cláusulas *BEGIN*, *COMMIT* y *ROLLBACK*, con las diferencias de que, en este caso, además del token, también se comprueba que el usuario sea administrador del sistema antes de dar permiso para realizar la creación del usuario. También se permite la creación de usuarios con rol de administrador, cosa que, en el *endpoint* de registro mediante aplicación, obviamente, no se permitía.

- Actualización dinámica

Se ha optado por optimizar la actualización de los datos de la tabla de usuarios y anuncios del panel de administración, de manera que, al crear o eliminar un usuario o un anuncio, no se solicite de nuevo toda la lista al servidor. En su lugar, lo que se hace es un filtrado desde el mismo código del *frontend* para mostrar o no la entidad en cuestión. De esta manera se reduce el tráfico de llamadas al *backend*, mejorando así el rendimiento del sistema.

- Gestión de la memoria

Como ya se comenta en el capítulo 6, concretamente en el apartado 6.5.5, la implementación del sistema de recomendaciones sigue el patrón *Singleton* [114], garantizando que solo exista una instancia de la clase *MotorRecomendacion* en todo el sistema, evitando así la duplicidad del modelo NLP utilizado. De esta forma, el motor de búsqueda hace uso de esta ventaja, reutilizando por composición la instancia del modelo que ya reside en el motor de recomendaciones y calculando así la puntuación de cada entidad del sistema obtenida como resultado de la búsqueda.

6.7.3 Moderación del contenido

Como se comenta a lo largo de este documento, uno de los principales objetivos de esta aplicación es mantener la transparencia y evitar todo tipo de contenido engañoso, fraudulento o inapropiado. Para esto, es fundamental que el administrador tenga permisos que le permitan llevar a cabo un control de los datos de la aplicación. Para esto se han implementado dos tipos de borrado de contenido.

- *Soft Delete*

Por cuestiones de prevención y trazabilidad, se ha implementado este tipo de borrado lógico, donde la entidad no desaparece por completo del sistema, sino que simplemente es ocultada a ojos de los usuarios. Este tipo de borrado sirve para la comprobación de casos donde se cree que un anuncio o valoración pueden incumplir las normas del sistema, o por motivos de prevención ante posibles disputas legales. La implementación de este borrado se ha llevado a cabo mediante la adición de un atributo booleano, *ocultar*, en las tablas Valoraciones y Anuncios. De esta manera, el administrador dispone de un botón que, mediante peticiones *PUT* al servidor, invierte el valor de este campo.

- *Hard Delete*

Este tipo de borrado, en cambio, supone la eliminación total de la entidad (anuncio o valoración) de la base de datos y, por tanto, del sistema. Para este tipo de borrado se lleva a cabo una sentencia SQL de tipo *DELETE FROM [tabla]*. Este tipo de borrados son necesarios para aquellos casos en los que se ha confirmado que el contenido de un anuncio o valoración incumple las normas del sistema, borrando así su rastro completo y liberando espacio en el sistema.

6.7.4 Motor de búsqueda dinámico

Para evitar la sobrecarga del servidor con múltiples consultas *LIKE* a la base de datos, los buscadores integrados en todas las tablas del panel se implementaron íntegramente en el lado del cliente.

Aprovechando que la API devuelve la colección completa de registros en la carga inicial, el componente de React almacena el término de búsqueda en el estado local. Mediante la renderización reactiva, en cada pulsación de tecla se recalcula dinámicamente el *array* visualizado utilizando funciones nativas de JavaScript (*.filter()*, *.includes()*, *.toLowerCase()*). Esto permite realizar búsquedas en tiempo real en múltiples campos de forma simultánea (nombres, correos, IDs y roles) con una latencia imperceptible, ofreciendo una experiencia de navegación fluida.

6.8 Cierre de la implementación

Tras esta explicación del trabajo realizado, se da por concluida la fase de implementación llevada a cabo para este proyecto, cerrando así el proceso de desarrollo inicial planteado de cara a la entrega del Trabajo de Fin de Grado. A continuación, en el siguiente capítulo, se darán las conclusiones y reflexiones obtenidas tras este proceso, además de detallar el trabajo propuesto a futuro para continuar con el avance de la aplicación hasta alcanzar una fase final apta para producción.

7. CONCLUSIONES Y TRABAJO FUTURO

Finalmente, como última sección antes de pasar al anexo y la bibliografía, se va a emplear este capítulo para reflejar las conclusiones obtenidas del desarrollo e implementación de este proyecto, con todas las reflexiones y desviaciones dejadas como trabajo a futuro.

7.1 Conclusiones

Tras completar todas las fases planteadas para la realización de este proyecto, se ha obtenido una primera versión de la aplicación propuesta al inicio del documento que, si bien aún no está en una fase final preparada para su despliegue, ha servido para definir todas las bases y funcionalidades principales propuestas, con una serie de implementaciones planeadas que se han apartado finalmente de la entrega actual y que se detallarán más en profundidad en la sección 0.

7.1.1 Evaluación de objetivos planteados

En general, el resultado obtenido es óptimo y cumple con la mayoría de los objetivos planteados en el capítulo 0 de este documento, más concretamente en la sección 1.4. A continuación, se van a evaluar cada uno de estos objetivos.

- **Objetivo general**

El objetivo general de la aplicación ha sido satisfecho de manera óptima, creando un producto de software en formato MVP (Minimum Viable Product) que, por cuestiones temporales, no ha dado tiempo de implementar en una versión final con todas las funcionalidades requeridas para ello. No obstante, esto es algo que se tenía previsto y el motivo por el que no se añadió en ningún momento la fase de despliegue del proyecto de manera real.

- **OE1**

Este objetivo se ha cumplido con éxito, identificando correctamente todos los requisitos indispensables para el funcionamiento esperado de la plataforma, como se puede observar en la sección 4.3 de este documento.

- **OE2**

Este objetivo ha sido cumplido con éxito, permitiendo el registro y edición de perfiles de usuario mediante la aplicación correctamente, eligiendo un rol concreto y usando la aplicación de la manera adecuada y acorde a él, como puede observarse en la sección 6.1.

- **OE3**

Este objetivo se ha cumplido con éxito, permitiendo la creación, publicación y edición de anuncios por parte de los caseros, además de la solicitud de registro en viviendas por parte de los inquilinos a través de las tarjetas de anuncios y siendo estas aceptadas o rechazadas en función de la decisión del casero del inmueble ofertado. Esta implementación puede observarse en la sección 6.2 y 6.3.

- **OE3**

El tercer objetivo específico se ha cumplido con éxito, llevando a cabo la implementación de un sistema de valoraciones entre usuarios y de usuarios a viviendas. A pesar de no haberse cumplido exactamente como se definió en la idea principal del proyecto y como se había reflejado en la planificación de este, el resultado obtenido es el esperado tras la decisión de cambiar el enfoque por cuestiones de experiencia de usuario y valor real de la información. Este aspecto se puede encontrar explicado en mayor detalle en la sección 7.2, mientras que la implementación del módulo se puede encontrar en la sección 6.4.

- **OE4**

A continuación, el cuarto objetivo específico ha sido implementado con éxito, dando lugar a un motor de recomendaciones complejo, compuesto por varias capas y ofreciendo un valor de

afinidad calculado en función de los datos obtenidos de los diferentes usuarios del sistema. No obstante, el planteamiento inicial de añadir Machine Learning al motor de recomendaciones ha quedado como trabajo a futuro, como se puede comprobar en la siguiente sección y, por tanto, se ha realizado una implementación parcial a falta de integración de Machine Learning, que es totalmente funcional y que resuelve el problema del arranque en frío planteado para el caso de uso concreto. Esta implementación se puede observar con mayor detalle en la sección 6.5.

- OE5

El quinto objetivo planteado se ha cumplido de forma parcial, implementando un panel de administración con las funcionalidades básicas al cual le falta principalmente la aceptación de anuncios, funcionalidad que ahora mismo, al tratarse de un producto mínimo viable, no ha sido aún integrada. La implementación de este sistema puede encontrarse en la sección 6.7.

- OE6

Este es el último objetivo específico planteado y el que menos se ha logrado tras todo el desarrollo, ya que este se ha focalizado en el desarrollo de la aplicación móvil, presentando una versión web a través de la versión generada por Expo para ordenador, pero no presenta la interfaz ni acabado que se quieren presentar. Por este motivo, al ser el objetivo centrado en la implementación de ambas versiones, se califica como logrado de manera parcial, teniendo una buena versión móvil, pero a implementar correctamente para la versión web. la implementación relacionada con este objetivo se puede ver a lo largo de todo el documento, debido a referirse este a la implementación de la aplicación en sí a lo largo de todas sus fases.

- OP1

Este objetivo personal ha sido cumplido con éxito, dando lugar al desarrollo de un proyecto full-stack basado en el framework React Native junto a Expo y a un aprendizaje muy útil y valorado en el sector actual, como se puede ver en el análisis de tecnologías planteado en el capítulo 3.

7.2 Trabajo futuro

Como se ha comentado en el apartado anterior, ciertos aspectos de este desarrollo se han dejado como trabajo a implementar tras la entrega de este documento y de su correspondiente defensa ya que, por diversos motivos, no se han podido llevar a cabo durante este desarrollo.

7.2.1 Integración de Machine Learning en el motor de recomendaciones

En primer lugar, como ya se comenta en la sección 6.5, ha quedado pendiente la implementación al completo del sistema de recomendaciones añadiendo correctamente la adición de interacciones del usuario con la aplicación y la integración de algoritmos de aprendizaje automático una vez obtenidos datos suficientes como para poder llevarlo a cabo.

Esta implementación se ha dejado para futuro debido a la imposibilidad de entrenar un modelo de ML en el plazo establecido y la falta de datos para poder realizarlo correctamente.

7.2.2 Optimización de parámetros arbitrarios en los cálculos del motor de recomendación

En la misma línea que el punto anterior, debido a falta de datos para el sistema de recomendaciones, ciertos parámetros explicados en detalle en el punto 6.5 han sido elegidos a modo de estimación y cumplen perfectamente con una primera versión del motor y con el problema del arranque en frío de este. No obstante, la variación y optimización de estos valores queda propuesta como trabajo futuro para cuando el sistema cuente con un número óptimo de usuarios como para poder realizar esa alteración.

7.2.3 Implementación web dedicada

Como ya se ha mencionado brevemente en la sección anterior, queda pendiente la implementación de una página web con las mismas funcionalidades que la versión móvil, de manera que se facilite a los usuarios el uso de StayIn desde cualquier dispositivo del que dispongan, de manera cómoda y ágil.

7.2.4 Finalización de una primera versión estable y despliegue de la aplicación

Como no puede ser de otra manera al desarrollar una aplicación móvil, el último paso a realizar tras la implementación de una primera versión funcional y operativa, lista para producción, es llevar a cabo el correspondiente despliegue de la misma en la Google Play Store y la Apple App Store, de manera que esté disponible para todos los usuarios independientemente del dispositivo del que posean.

7.3 Valoración final

Después de todo el proceso y trabajo realizado para la realización de este proyecto, se puede decir que se ha obtenido una versión inicial y escalable de un producto que hace meses era tan solo una idea y que ahora mismo es una realidad, contando con todo el proceso de aprendizaje que ha conllevado y de los errores cometidos en cada una de las fases, que han servido para pulir la idea inicial y seguir dándole forma a medida que se ha avanzado en el desarrollo de esta aplicación.

Este proyecto ha servido para expandir la imaginación a la hora tanto de plantear ideas iniciales como soluciones a determinados problemas con los que no se contaba en el punto de partida, dando lugar a un proceso de evolución personal y al último escalón necesario en la formación de un ingeniero informático.

8. ANEXO

Este último capítulo antes de pasar a la bibliografía del proyecto va a servir para mostrar aquellos fragmentos de código más extensos que no tienen lugar en el contenido de la memoria en sí, pero que son fundamentales para el correcto funcionamiento del sistema y que, por tanto, deben ser mencionados.

8.1 Creación de tablas de la base de datos

En este apartado se van a definir las sentencias para la creación de las tablas de la base de datos.

```
1 CREATE TABLE Usuarios (  
2     id_usuario SERIAL PRIMARY KEY,  
3     username VARCHAR(20) NOT NULL UNIQUE,  
4     email VARCHAR(255) NOT NULL UNIQUE,  
5     dni VARCHAR(9) UNIQUE,  
6     nombre VARCHAR(50) NOT NULL,  
7     apellidos VARCHAR(50) NOT NULL,  
8     rol VARCHAR(15) CHECK(rol IN('Inquilino', 'Casero', 'Administrador'))  
9     NOT NULL,  
10    passwd VARCHAR(255) NOT NULL,  
11    f_nac DATE NOT NULL,  
12    sexo VARCHAR(15) NOT NULL,  
13    verificado BOOLEAN NOT NULL DEFAULT FALSE,  
14    numero_tlf VARCHAR(9) UNIQUE,  
15    img_perfil VARCHAR(100),  
16    f_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
17 );
```

Código 13. Usuarios

```
1 CREATE TABLE Inquilino (  
2     id_inquilino INT PRIMARY KEY,  
3     estudiante BOOLEAN NOT NULL DEFAULT FALSE,  
4     fuma BOOLEAN NOT NULL DEFAULT FALSE,  
5     mascota BOOLEAN NOT NULL DEFAULT FALSE,  
6     descripcion varchar(500),  
7     estado VARCHAR(50) NOT NULL CHECK(estado IN('Vivienda', 'Compañero',  
8     'Privado')) DEFAULT 'Privado',  
9     vector_realidad JSONB DEFAULT '{}':JSONB,  
10    vector_test JSONB DEFAULT '{}':JSONB,  
11    FOREIGN KEY (id_inquilino) REFERENCES Usuarios(id_usuario) ON DELETE  
    CASCADE  
12 );
```

Código 14. Inquilinos

```
1 CREATE TABLE Caseros (  
2     id_casero INT PRIMARY KEY,  
3     estado BOOLEAN DEFAULT TRUE,  
4     FOREIGN KEY(id_casero) REFERENCES Usuarios(id_usuario) ON DELETE  
5     CASCADE  
6 );
```

Código 15. Caseros

```

1 CREATE TABLE Viviendas (
2     id_vivienda SERIAL PRIMARY KEY,
3     id_casero INT,
4     area INT NOT NULL,
5     direccion VARCHAR(50) NOT NULL,
6     max_inquilinos INT NOT NULL,
7     cpostal VARCHAR(5) NOT NULL DEFAULT '00000',
8     numero INT NOT NULL DEFAULT 1,
9     puerta VARCHAR(4) NOT NULL DEFAULT 1,
10    ciudad VARCHAR(50),
11    FOREIGN KEY(id_casero) REFERENCES Caseros(id_casero) ON DELETE CASCADE
12 );

```

Código 16. Viviendas

```

1 CREATE TABLE Anuncios (
2     id_anuncio SERIAL PRIMARY KEY,
3     id_vivienda INT,
4     id_casero INT,
5     f_publicacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
6     titulo VARCHAR(50) NOT NULL,
7     img VARCHAR(100) NOT NULL,
8     direccion VARCHAR(50) NOT NULL,
9     precio INT NOT NULL,
10    descripcion TEXT,
11    características JSONB DEFAULT '{} '::JSONB,
12    multimedia JSONB DEFAULT '[] '::JSONB,
13    FOREIGN KEY (id_vivienda) REFERENCES Viviendas(id_vivienda) ON DELETE
14    CASCADE,
15    FOREIGN KEY (id_casero) REFERENCES Caseros(id_casero) ON DELETE
    CASCADE
16 );

```

Código 17. Anuncios

```

1 CREATE TABLE Estancias (
2     id_estancia SERIAL PRIMARY KEY,
3     id_inquilino INT NOT NULL,
4     id_vivienda INT NOT NULL,
5     f_inicio_estancia DATE NOT NULL DEFAULT CURRENT_TIMESTAMP,
6     f_fin_estancia DATE,
7     FOREIGN KEY (id_inquilino) REFERENCES Inquilino(id_inquilino) ON
8     DELETE CASCADE,
9     FOREIGN KEY (id_vivienda) REFERENCES Viviendas(id_vivienda) ON DELETE
10    CASCADE
11 );

```

Código 18. Estancias

```

1 CREATE TABLE Solicitudes (
2     id_solicitud SERIAL PRIMARY KEY,
3     id_anuncio INT,
4     id_inquilino INT,
5     f_solicitud TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
6     estado VARCHAR(50) NOT NULL CHECK(estado IN('Aceptada', 'Rechazada',
7     'Pendiente')) DEFAULT 'Pendiente',
8     FOREIGN KEY (id_anuncio) REFERENCES Anuncios(id_anuncio) ON DELETE
9     CASCADE,
10    FOREIGN KEY (id_inquilino) REFERENCES Inquilino(id_inquilino) ON
11    DELETE CASCADE,
12    UNIQUE (id_anuncio, id_inquilino)

```

```
13 );
```

Código 19. Solicitudes

```
1 CREATE TABLE Valoraciones (  
2     id_valoracion SERIAL PRIMARY KEY,  
3     id_estancia INT NOT NULL,  
4     id_valorador INT NOT NULL,  
5     id_valorado INT NOT NULL,  
6     comentario TEXT,  
7     f_valoracion DATE NOT NULL DEFAULT CURRENT_TIMESTAMP,  
8     oculto BOOLEAN DEFAULT FALSE,  
9     FOREIGN KEY (id_estancia) REFERENCES Estancias(id_estancia) ON DELETE  
10    CASCADE,  
11     FOREIGN KEY (id_valorador) REFERENCES Usuarios(id_usuario) ON DELETE  
12    CASCADE,  
13     FOREIGN KEY (id_valorado) REFERENCES Usuarios(id_usuario) ON DELETE  
14    CASCADE  
);
```

Código 20. Valoraciones

```
1 CREATE TABLE Criterios_Valoracion (  
2     id_criterio SERIAL PRIMARY KEY,  
3     categoria VARCHAR(50) NOT NULL,  
4     pregunta TEXT NOT NULL,  
5     tipo_criterio VARCHAR(20) DEFAULT 'usuario' CHECK(tipo_criterio  
IN('usuario', 'inmueble'),  
6     pregunta_test TEXT,  
7     clave_json VARCHAR(50)  
);
```

Código 21. Criterios_Valoracion

```
1 CREATE TABLE Puntuacion_Criterios (  
2     id_criterio INT NOT NULL,  
3     id_valoracion INT NOT NULL,  
4     puntuacion INT NOT NULL CHECK (puntuacion >= 1 AND puntuacion <= 5),  
5     PRIMARY KEY (id_criterio, id_valoracion),  
6     FOREIGN KEY (id_criterio) REFERENCES Criterios_Valoracion(id_criterio)  
7     ON DELETE CASCADE,  
8     FOREIGN KEY (id_valoracion) REFERENCES Valoraciones(id_valoracion) ON  
9     DELETE CASCADE  
10 );
```

Código 22. Puntuacion_Criterios

```
1 CREATE TABLE Interacciones_Companeros (  
2     id_interaccion SERIAL PRIMARY KEY,  
3     id_valorador INT REFERENCES Usuarios(id_usuario) ON DELETE CASCADE,  
4     id_valorado INT REFERENCES Usuarios(id_usuario) ON DELETE CASCADE,  
5     tipo_interaccion VARCHAR(50),  
6     peso_interaccion INT,  
7     UNIQUE(id_valorador, id_valorado, tipo_interaccion)  
8 );
```

Código 23. Interacciones_Companeros

```

1 CREATE TABLE Interacciones_Anuncios (
2     id_interaccion SERIAL PRIMARY KEY,
3     id_inquilino INT NOT NULL,
4     id_anuncio INT NOT NULL,
5     f_interaccion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
6     peso_interaccion INT NOT NULL,
7     tipo_interaccion VARCHAR(50) NOT NULL DEFAULT 'favorito',
8     FOREIGN KEY (id_inquilino) REFERENCES Inquilino(id_inquilino) ON
9 DELETE CASCADE,
10    FOREIGN KEY (id_anuncio) REFERENCES Anuncios(id_anuncio) ON DELETE
11 CASCADE
12 );

```

Código 24. Interacciones_Anuncios

```

1 CREATE TABLE Preferencias_Inquilinos (
2     id_inquilino INT NOT NULL,
3     presupuesto_max INT NOT NULL,
4     zona VARCHAR(5)[] NOT NULL DEFAULT '{}'::VARCHAR(5)[],
5     mascota BOOLEAN NOT NULL,
6     preferencias JSONB DEFAULT '{}'::JSONB,
7     PRIMARY KEY (id_inquilino),
8     FOREIGN KEY (id_inquilino) REFERENCES Inquilino(id_inquilino) ON
9 DELETE CASCADE
10 );

```

Código 25. Preferencias_Inquilinos

8.2 Creación de instancias

En este punto se reflejan las inserciones más relevantes de instancias en tablas de la base de datos.

```

1 INSERT INTO Criterios_Valoracion (categoria, pregunta) VALUES
2     -- Higiene y orden
3     ('Higiene y orden', 'Frecuencia y calidad de limpieza: ¿Cumple el
4 turno / horario de limpieza?'),
5     ('Higiene y orden', 'Gestión de la vajilla: ¿Friega los platos cuando
6 termina de comer o los deja apilados hasta el día o días después?'),
7     ('Higiene y orden', 'Zonas comunes: ¿Es ordenado en las zonas comunes
8 de la vivienda?'),
9
10    -- Economía y responsabilidad
11    ('Economía y responsabilidad', 'Puntualidad de los pagos: ¿Paga los
12 gastos/alquiler a tiempo?'),
13    ('Economía y responsabilidad', 'Gastos compartidos: ¿Es proactivo a la
14 hora de reponer los elementos en común?'),
15    ('Economía y responsabilidad', 'Consumo responsable: ¿Realiza un
16 consumo responsable de la luz/agua/calefacción de la vivienda?'),
17
18    -- Ruido y horarios
19    ('Ruido y horarios', 'Respeto de las horas de descanso: ¿Respetas las
20 horas de silencio de la vivienda (noche/madrugada/horas impuestas en la
21 vivienda)?'),
22
23    -- Visitas
24    ('Visitas', 'Frecuencia de visitas: ¿Trae visitas a casa a menudo?'),
25    ('Visitas', 'Uso de espacio con invitados: Cuando hay visita, ¿se
26 respeta el uso de zonas comunes, permitiendo al resto de inquilinos
27 usarlas cómodamente?'),

```

```

28
29     -- Comunicación
30     ('Comunicación', 'Reacción ante el conflicto: ¿Es resolutivo a la hora
31 de resolver problemas en la convivencia?'),
32     ('Comunicación', 'Empatía: ¿Es comprensivo con los posibles problemas
33 que hayan podido surgir?'),
34     ('Comunicación', 'Forma de comunicar las cosas: ¿Es expresivo con
35 aquellas cosas que considera problemas?'),
36
37     -- Respeto de la propiedad
38     ('Respeto de la propiedad', 'Cuidado del mobiliario de la vivienda:
39 ¿Trata bien los electrodomésticos y muebles?');

```

Código 26. Inserción de criterios Inquilino-Inquilino

```

1  CREATE OR REPLACE FUNCTION actualizar_vector_realidad()
2  RETURNS TRIGGER AS $$
3  DECLARE
4      v_id_valorado INT;
5      v_vector_test JSONB;
6      v_peso_test   CONSTANT NUMERIC := 3;
7  BEGIN
8      SELECT id_valorado INTO v_id_valorado
9      FROM Valoraciones
10     WHERE id_valoracion = COALESCE(NEW.id_valoracion, OLD.id_valoracion);
11
12     IF v_id_valorado IS NULL THEN
13         RETURN COALESCE(NEW, OLD);
14     END IF;
15     SELECT vector_test INTO v_vector_test
16     FROM Inquilino
17     WHERE id_inquilino = v_id_valorado;
18     WITH recibidas AS (
19         SELECT pc.id_criterio, pc.puntuacion
20         FROM Valoraciones val
21         JOIN Puntuacion_Criterios pc ON pc.id_valoracion =
22 val.id_valoracion
23         WHERE val.id_valorado = v_id_valorado
24         AND val.id_valorador <> v_id_valorado
25     ) por_criterio AS (
26         SELECT cr.categoria, cr.clave,
27             ROUND(
28                 ( v_peso_test * COALESCE((v_vector_test #>>
29 ARRAY[cr.categoria, cr.clave])::numeric, 3)
30                 + COALESCE(SUM(r.puntuacion), 0) )
31                 / ( v_peso_test + COALESCE(COUNT(r.puntuacion), 0) )
32                 , 2) AS valor
33         FROM Criterios_Valoracion cr
34         LEFT JOIN recibidas r ON r.id_criterio = cr.id_criterio
35         GROUP BY cr.categoria, cr.clave
36     ),
37     por_modulo AS (
38         SELECT categoria, jsonb_object_agg(clave, valor) AS preguntas
39         FROM por_criterio
40         GROUP BY categoria
41     )
42     UPDATE Inquilino
43     SET vector_realidad = COALESCE(

```

```

44      (SELECT jsonb_object_agg(categoria, preguntas) FROM
45 por_modulo),
46      '{}'::jsonb)
47 WHERE id_inquilino = v_id_valorado;
48
49 RETURN COALESCE(NEW, OLD);
50 END;
51 $$ LANGUAGE plpgsql;
52
53 CREATE TRIGGER trigger_actualizar_realidad
54 AFTER INSERT OR UPDATE OR DELETE ON Puntuacion_Criterios
55 FOR EACH ROW
56 EXECUTE FUNCTION actualizar_vector_realidad();

```

Código 27. Implementación trigger actualización vector_realidad

8.3 Sistema de recomendaciones

Este punto contiene los fragmentos de código más relevantes del módulo de recomendaciones.

```

1  {
2    "modulos": {
3      "economia_contrato": 40.0,
4      "convivencia_normas": 30.0,
5      "caracteristicas_vivienda": 15.0,
6      "habitacion": 15.0
7    },
8
9    "internos": {
10     "economia_contrato": {
11       "gastos_incluidos": 50.0,
12       "estancia_minima_meses": 30.0,
13       "meses_fianza_max": 20.0
14     },
15     "convivencia_normas": {
16       "ambiente": 40.0,
17       "limpieza": 30.0,
18       "visitas": 15.0,
19       "ocupacion_inquilinos": 9.0,
20       "genero_inquilinos": 4.0,
21       "lgtb-friendly": 2.0
22     },
23     "habitacion": {
24       "bano_privado": 30.0,
25       "cama_doble": 35.0,
26       "zona_estudio": 35.0
27     },
28     "caracteristicas_vivienda": {
29       "tipo_inmueble": 40.0,
30       "banos_minimos": 30.0,
31       "climatizacion": 20.0,
32       "ascensor": 10.0
33     }
34   }
35 }

```

Código 28. Distribución de pesos en puntuaciones de anuncios


```

1  {
2      "economia_contrato": {
3          "gastos_incluidos": ["agua", "internet", "luz", "comunidad"],
4          "meses_fianza_max": 1,
5          "estancia_minima_meses": 6
6      },
7      "convivencia_normas": {
8          "visitas": "rara_vez", "ocasional", "a_menudo", "indiferente"
9          "limpieza": "ocasional", "a_menudo", "turnos_estrechos",
10         "ambiente": "estudio", "tranquilo", "animado",
11         "ocupacion_inquilinos": "solo_estudiantes", "solo_trabajadores",
12         "erasmus", "indiferente",
13         "genero_inquilinos": "femenino", "masculino", "indiferente"
14         "lgbt-friendly": true, false
15     },
16     "habitacion": {
17         "cama_doble": true, false
18         "zona_estudio": true, false
19         "bano_privado": true, false
20     },
21     "caracteristicas_vivienda": {
22         "tipo_inmueble": "casa", "bajo", "piso", "duplex", "atico",
23         "banos_minimos": 2,
24         "ascensor": true, false
25         "climatizacion": "calefaccion_central", "aire_acondicionado",
26     "nada"
27     }
28 }

```

Código 29. Posibles valores de JSON de características y preferencias de anuncios

```

1 self.enrutador = {
2     # Variables de Conjuntos (Arrays)
3     "gastos_incluidos": self.evaluar_array,
4     "tipo_inmueble": self.evaluar_array,
5
6
7     # Variables Numéricas (Tope de Fricción)
8     "meses_fianza_max": self.evaluar_friccion,
9     "estancia_minima_meses": self.evaluar_friccion,
10
11
12     # Variables Numéricas (Umbral Mínimo)
13     "banos_minimos": self.evaluar_minimo,
14
15     # Variables Binarias / Exactas
16     "cama_doble": self.evaluar_exacta,
17     "zona_estudio": self.evaluar_exacta,
18     "bano_privado": self.evaluar_exacta,
19     "ascensor": self.evaluar_exacta,
20     "climatizacion": self.evaluar_exacta,
21     "lgtb-friendly": self.evaluar_exacta,
22
23
24     # Variable con opción de indiferente
25     "genero_inquilinos": self.evaluar_exacta_indiferente,
26     "ocupacion_inquilinos": self.evaluar_exacta_indiferente,
27
28
29     # Variables con proximidad en la escala
30     "visitas": lambda d, r: self.evaluar_proximidad(d, r, ["rara_vez",
31 "ocasional", "a_menudo"]),
32     "limpieza": lambda d, r: self.evaluar_proximidad(d, r,
33 ["ocasional", "a_menudo", "turnos_estrictos"]),
34     "ambiente": lambda d, r: self.evaluar_proximidad(d, r,
35 ["estudio", "tranquilo", "animado"])
36
37 }
38
39

```

Código 30. Declaración de enrutador

8.4 Sistema de búsqueda

```

1 <GooglePlacesAutocomplete
2     placeholder="Escribe el nombre de la calle o barrio..."
3     fetchDetails={true}
4     onFail={(error) => console.error('[Places] Fallo:', error)}
5     onNotFound={() => console.warn('[Places] Sin resultados de
6 detalles')}
7
8
9
10
11     onPress={(data, details = null) => {
12         // Actualizamos la vista visual inmediatamente para el usuario
13         setZonaVisual(data.description);
14         setModalVisible(false);
15
16         // Extraemos el texto base por si los detalles de Google fallan
17         let ciudad = data.structured_formatting?.main_text ||
18 data.description.split(',')[0];
19         let codigoPostal = null;
20         let ubicacion = null;
21
22

```

```

23     if (details && details.address_components) {
24         // Extraer Código Postal
25         const postalCodeObj = details.address_components.find((c: any)
26 => c.types.includes('postal_code'));
27         if (postalCodeObj) codigoPostal = postalCodeObj.long_name;
28
29         // Extraer Ciudad (buscando en todas las etiquetas posibles de
30 Google)
31         const ciudadObj = details.address_components.find((c: any) =>
32             c.types.includes('locality') ||
33             c.types.includes('administrative_area_level_2') ||
34             c.types.includes('postal_town'));
35         if (ciudadObj) ciudad = ciudadObj.long_name;
36
37         ubicacion = details.geometry?.location || null;
38     }
39
40     // Enviamos los datos asegurando que 'ciudad' nunca vaya vacía
41     onZonaSeleccionada(
42         codigoPostal ? [codigoPostal] : [],
43         ciudad,
44         ubicacion
45     );
46 }
47
48 query={{
49     key: process.env.EXPO_PUBLIC_GOOGLE_API_KEY,
50     language: 'es',
51     components: 'country:es',
52     types: 'geocode'
53 }}
54
55 keyboardShouldPersistTaps="handled"
56
57 keepResultsAfterBlur={true}
58
59 styles={{
60     container: {
61         flex: 1,
62         paddingHorizontal: 15,
63         marginTop: 10
64     },
65     textInput: {
66         backgroundColor: currentColors.formTextArea,
67         color: currentColors.formTextColor,
68         borderRadius: 10,
69         borderWidth: 2,
70         borderColor: currentColors.flatCardBorderColor,
71         height: 50,
72         paddingHorizontal: 15,
73         fontSize: 16,
74     },
75     listView: {
76         backgroundColor: currentColors.background,
77         marginTop: 10,
78     },
79     row: {

```

```

89         backgroundColor: currentColors.background,
90         paddingVertical: 15,
91         flexDirection: 'row',
92     },
93     separator: {
94         height: 1,
95         backgroundColor: currentColors.flatCardBorderColor,
96         opacity: 0.3
97     },
98     description: {
99         color: currentColors.formTextColor,
100        fontSize: 15,
101    }
102  }}
103
104  }}
105
106  textInputProps={{
107    placeholderTextColor: currentColors.placeholderColor,
108    clearButtonMode: 'always', // para que funcione al pulsar en el
109    modal con el teclado abierto
110  }}
111  />
112
113

```

Código 31. GooglePlacesAutocomplete

9. USO DE IA EN ESTE TRABAJO

Se ha hecho uso de inteligencia artificial generativa en la realización de este trabajo con el fin de obtención de ideas, aclaración de diferentes aspectos o problemas planteados a lo largo de su desarrollo, apoyándose en ella principalmente con el fin de agilizar tareas de aprendizaje sobre conceptos desconocidos inicialmente, que posteriormente se han revisado y realizado conscientemente.

También se ha hecho uso de esta tecnología para la ayuda en la detección de incoherencias o faltas de cohesión en el texto redactado a lo largo de esta memoria, con el fin de obtener el mejor resultado posible a partir de la idea inicialmente planteada, tomando únicamente las sugerencias que se han considerado que mejoraban el producto de alguna forma.

10. BIBLIOGRAFÍA

- [1] «carloshoyo/Trabajo-Fin-de-Grado: Repositorio para mi proyecto de fin de grado de Ingeniería Informática», GitHub. Accedido: 19 de junio de 2026. [En línea]. Disponible en: <https://github.com/carloshoyo/Trabajo-Fin-de-Grado>
- [2] «La juventud española no puede emanciparse sin empobrecerse: el alquiler ya consume el 98,7% de su salario». Accedido: 21 de junio de 2026. [En línea]. Disponible en: https://www.cje.org/observatorio_2025/
- [3] «El 87% de la juventud emancipada en España comparte vivienda para reducir gastos, según un nuevo informe del Consejo de la Juventud de España». Accedido: 21 de junio de 2026. [En línea]. Disponible en: <https://www.cje.org/nota-de-prensa-un-problema-como-una-casa/>
- [4] «What is personalization? | McKinsey». Accedido: 23 de febrero de 2026. [En línea]. Disponible en: <https://www.mckinsey.com/featured-insights/mckinsey-explainers/what-is-personalization>
- [5] C. Anderson, *The long tail: why the future of business is selling less of more*, Rev. and Updated ed. New York: Hyperion, 2008.
- [6] E. GraphEverywhere, «Sistemas de recomendación | Qué son, tipos y ejemplos», GraphEverywhere. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.grapheverywhere.com/sistemas-de-recomendacion-que-son-tipos-y-ejemplos/>
- [7] «¿Qué es un motor de recomendación? | IBM». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/recommendation-engine>
- [8] admin, «Sistemas de recomendación: ¿qué son y cómo funcionan?», MFAIA. Accedido: 23 de febrero de 2026. [En línea]. Disponible en: <https://mfaia.dia.fi.upm.es/sistemas-de-recomendacion-personalizando-la-experiencia-del-usuario-en-la-era-digital/>
- [9] «Reproductor web de Spotify: música para todo el mundo», Spotify. Accedido: 22 de junio de 2026. [En línea]. Disponible en: <https://open.spotify.com/>
- [10] «Amazon.es: compra online de electrónica, libros, deporte, hogar, moda y mucho más.» Accedido: 22 de junio de 2026. [En línea]. Disponible en: <https://www.amazon.es/>
- [11] «Home | Meta Open Source». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://opensource.fb.com/>
- [12] «K-Nearest Neighbors (k-NN) Algorithm - Amazon SageMaker AI». Accedido: 23 de febrero de 2026. [En línea]. Disponible en: <https://docs.aws.amazon.com/sagemaker/latest/dg/k-nearest-neighbors.html>
- [13] «Filtrado basado en el contenido | Machine Learning», Google for Developers. Accedido: 23 de febrero de 2026. [En línea]. Disponible en: <https://developers.google.com/machine-learning/recommendation/content-based/basics?hl=es-419>
- [14] «¿Qué es el filtrado basado en contenido? | IBM». Accedido: 23 de febrero de 2026. [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/content-based-filtering>
- [15] «Netflix España - Ver series en línea, ver películas en línea». Accedido: 22 de junio de 2026. [En línea]. Disponible en: <https://www.netflix.com/es/>
- [16] D. Jurafsky y J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*, 3rd ed. 2026. [En línea]. Disponible en: <https://web.stanford.edu/~jurafsky/slp3/>
- [17] «¿Qué es el test de Turing? Explicación de su definición y función», IONOS Digital Guide. Accedido: 19 de marzo de 2026. [En línea]. Disponible en: <https://www.ionos.es/digitalguide/online-marketing/analisis-web/test-de-turing/>
- [18] «Alan Turing: Creator of modern computing - BBC Teach». Accedido: 19 de marzo de 2026. [En línea]. Disponible en: <https://www.bbc.co.uk/teach/articles/zhwp7nb>
- [19] A. M. Turing, «Computing Machinery and Intelligence», en *Parsing the Turing Test: Philosophical and Methodological Issues in the Quest for the Thinking Computer*, R. Epstein, G. Roberts, y G. Beber, Eds., Dordrecht: Springer Netherlands, 2009, pp. 23-65. doi: 10.1007/978-1-4020-6710-5_3.
- [20] «¿Qué son los modelos de lenguaje de grandes (LLM)? | Microsoft Azure». Accedido: 19 de marzo de 2026. [En línea]. Disponible en: <https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-are-large-language-models-llms>
- [21] T. Mikolov, K. Chen, G. Corrado, y J. Dean, «Efficient Estimation of Word Representations in Vector Space». 2013. [En línea]. Disponible en: <https://arxiv.org/abs/1301.3781>
- [22] A. Vaswani *et al.*, «Attention Is All You Need». 2023. [En línea]. Disponible en: <https://arxiv.org/abs/1706.03762>

- [23] N. Reimers y I. Gurevych, «Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks». 2019. [En línea]. Disponible en: <https://arxiv.org/abs/1908.10084>
- [24] C. D. Manning, P. Raghavan, y H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. [En línea]. Disponible en: <https://books.google.es/books?id=t1PoSh4uwVcC>
- [25] «Alternativas a Idealista, Fotocasa y Pisos.com: estas son las mejores apps para encontrar piso», *elEconomista.es*. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.eleconomista.es/tecnologia/noticias/13078890/11/24/adios-a-idealista-fotocasa-y-pisoscom-estas-son-las-mejores-apps-para-encontrar-piso.html>
- [26] «idealista — Casas y pisos, alquiler y venta. Anuncios gratis». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.idealista.com/>
- [27] M. Bastero, «Las mejores apps y webs para buscar piso online en España», *Marketing4eCommerce*. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://marketing4ecommerce.net/mejores-apps-webs-para-buscar-piso-online/>
- [28] «Fotocasa.es: Alquiler de pisos, compra y venta». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.fotocasa.es/es/>
- [29] «habitaclia - Pisos y casas en venta y alquiler». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.habitaclia.com/>
- [30] «Casas y pisos en venta, alquila o vende tu piso - pisos.com». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.pisos.com>
- [31] «yaencontre: Buscador de casas y pisos en venta y en alquiler», *yaencontre*. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.yaencontre.com>
- [32] «Habitaciones y pisos de alquiler | Badi». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://badi.com>
- [33] «Pisos de alquiler por temporada y habitaciones por meses», *Spotahome*. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.spotahome.com/es>
- [34] «Housfy: Inmobiliaria, administración de fincas e hipotecas». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://housfy.com>
- [35] «Alquiler de habitaciones, pisos, apartamentos y alojamiento en Uniplaces», *Uniplaces*. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: https://www.uniplaces.com/es/?esl-k=google-ads|ng|c641090525401|me|kuniplaces|p|t|dc|a142856175025|g18898098373&gad_source=1&gad_campaignid=18898098373&gbraid=0AAAAADpU3Fk5P82bH4ykYqYqWPuNUToVQ&gclid=Cj0KCQiAhaHMBhD2ARIsAPAU_D5H4W--1JjHJLWtjt0dnC3vZUd8PNYbSKWFvQnogB6aK_3c0JkR5IaAj0UEALw_wcB
- [36] «Inmobiliaria Tecnocasa. Vende tu piso o encuentra inmuebles en venta o alquiler». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.tecnocasa.es/>
- [37] insfera, «REDPISO | Casas y pisos en venta o alquiler», *Redpiso*. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.redpiso.es>
- [38] «Las 10 agencias inmobiliarias más grandes de España». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://eximiarealestate.com/golden-mile/agencias-inmobiliarias-espana/>
- [39] «Dónde Publicar Gratis tu Piso: Top 10 Portales Inmobiliarios en España (2025)». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.sooprema.com/mejores-portales-inmobiliarios-gratuitos-espana>
- [40] F. Ramirez, «Mejores portales inmobiliarios: Top 10 en España 2025 - Inmoweb», *Inmoweb Blog*. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.inmoweb.es/blog/portales-inmobiliarios-espana/>
- [41] «Technology | 2025 Stack Overflow Developer Survey». Accedido: 20 de junio de 2026. [En línea]. Disponible en: <https://survey.stackoverflow.co/2025/technology>
- [42] «React Native · Learn once, write anywhere». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://reactnative.dev/>
- [43] «React Native for Web». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://necolas.github.io/react-native-web/>
- [44] «Flutter - Build apps for any screen». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://flutter.dev/>
- [45] «Dart programming language». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://dart.dev/index.html>

- [46] «Capacitor by Ionic - Cross-platform apps with web technology», Capacitor. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://capacitorjs.com/>
- [47] J. M. A. A. Maluenda Raquel, «Qué es Ionic: ventajas y desventajas de usarlo en apps móviles híbridas», Profile Software Services. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://profile.es/blog/que-es-ionic/>
- [48] «¿Qué es Ionic? Todo lo que el desarrollador debe conocer», Talently Blog. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://talently.tech/blog/que-es-ionic/>
- [49] T. Bridge, «Lenguajes de Programación Más Usados», The Bridge | Digital Talent Accelerator. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://thebridge.tech/blog/lenguajes-backend/>
- [50] «Top lenguajes backend de libre uso», Rootstack. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://rootstack.com/es/blog/top-lenguajes-backend-de-libre-uso>
- [51] «The top programming languages», The State of the Octoverse. Accedido: 9 de febrero de 2026. [En línea]. Disponible en: <https://octoverse.github.com/2022/top-programming-languages>
- [52] A. M. Team, «Tipos de bases de datos: Todo lo que debes saber en 2026», Astera. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.astera.com/es/type/blog/a-quick-overview-of-different-types-of-databases/>
- [53] «MySQL». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.mysql.com/>
- [54] P. G. D. Group, «PostgreSQL», PostgreSQL. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.postgresql.org/>
- [55] «MariaDB Foundation - MariaDB.org». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://mariadb.org/>
- [56] «PostgreSQL vs MariaDB y MySQL: guía práctica para elegir tu base de datos – Puntocomunica: Comunicación 2.0». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.puntocomunica.com/postgresql-vs-mariadb-y-mysql-guia-practica-para-elegir-tu-base-de-datos/>
- [57] «MariaDB o PostgreSQL: diferencia entre bases de datos relacionales de código abierto - AWS», Amazon Web Services, Inc. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://aws.amazon.com/es/compare/the-difference-between-mariadb-and-postgresql/>
- [58] «PostgreSQL vs. MySQL: ¿Cuál es la diferencia? | IBM». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/postgresql-vs-mysql>
- [59] «¿Qué es una base de datos NoSQL? | IBM». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/nosql-databases>
- [60] «¿Qué es NoSQL?», Amazon Web Services, Inc. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://aws.amazon.com/es/nosql/>
- [61] «Base de datos NoSQL: ¿qué es NoSQL? | Microsoft Azure». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-is-nosql-database>
- [62] «Base de Datos con Android - Blog de Código Facilito», CódigoFacilito. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://codigofacilito.com/articulos/base-de-datos-con-android>
- [63] «SQLite Home Page». Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://sqlite.org/>
- [64] «Realm», GitHub. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://github.com/realm>
- [65] campusMVP, «Los 4 mejores lenguajes de programación para Inteligencia Artificial / Machine Learning», campusMVP.es. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.campusmvp.es/recursos/post/los-4-mejores-lenguajes-de-programacion-para-inteligencia-artificial-machine-learning.aspx>
- [66] «Guía docente de Sistemas de Información Basados en Web (Especialidad Ingeniería del Software) (296113)», Universidad de Granada. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://www.ugr.es/estudiantes/grados/grado-ingenieria-informatica/sistemas-informacion-basados-webeingsoft/guia-docente>
- [67] «Expo», Expo. Accedido: 8 de febrero de 2026. [En línea]. Disponible en: <https://expo.dev/>
- [68] «¿Qué es scrum? Una guía para el marco de trabajo ágil | Atlassian». Accedido: 14 de marzo de 2026. [En línea]. Disponible en: <https://www.atlassian.com/es/agile/scrum>
- [69] Admin_Donetonic, «Product Owner, definición y funciones. Una pieza clave para el éxito», DoneTonic. Accedido: 14 de marzo de 2026. [En línea]. Disponible en: <https://donetonic.com/es/funciones-de-un-product-owner/>

- [70] Admin_Donetonic, «El equipo Scrum: quiénes lo componen y cuáles son sus funciones», DoneTonic. Accedido: 14 de marzo de 2026. [En línea]. Disponible en: <https://donetonic.com/es/equipo-scrum/>
- [71] Admin_Donetonic, «¿Qué es el Scrum Master y cuáles son sus funciones?», DoneTonic. Accedido: 14 de marzo de 2026. [En línea]. Disponible en: <https://donetonic.com/es/que-es-scrum-master/>
- [72] Admin_Donetonic, «Los 5 eventos en Scrum claves para el desarrollo de producto», DoneTonic. Accedido: 14 de marzo de 2026. [En línea]. Disponible en: <https://donetonic.com/es/que-son-los-sprints-en-scrum/>
- [73] Admin_Donetonic, «¿Sabes qué es el Sprint Planning en Scrum? Planifica el trabajo», DoneTonic. Accedido: 14 de marzo de 2026. [En línea]. Disponible en: <https://donetonic.com/es/que-es-el-sprint-planning/>
- [74] Admin_Donetonic, «Qué es un sprint en la metodología Scrum y qué beneficios aporta», DoneTonic. Accedido: 14 de marzo de 2026. [En línea]. Disponible en: <https://donetonic.com/es/que-es-un-sprint-en-la-metodologia-scrum/>
- [75] Admin_Donetonic, «¿Qué es el Daily Scrum Meeting? Sincroniza el trabajo de tu equipo», DoneTonic. Accedido: 14 de marzo de 2026. [En línea]. Disponible en: <https://donetonic.com/es/que-es-el-daily-scrum-meeting/>
- [76] Admin_Donetonic, «Qué es el Product Backlog en el marco Scrum • DoneTonic», DoneTonic. Accedido: 14 de marzo de 2026. [En línea]. Disponible en: <https://donetonic.com/es/que-es-el-product-backlog/>
- [77] «El Incremento de producto». Accedido: 14 de marzo de 2026. [En línea]. Disponible en: <https://www.scrumio.com/scrum/incremento-de-producto/>
- [78] «Jira | Gestión de proyectos para la generación de la IA | Atlassian». Accedido: 21 de junio de 2026. [En línea]. Disponible en: https://www.atlassian.com/es/software/jira?campaign=19324540046&adgroup=143040486445&targetid=kwd-855725830&matchtype=e&network=g&device=c&device_model=&creative=642122300833&keyword=jira&placement=&target=&ds_eid=700000001558501&ds_e1=GOOGLE&gad_source=1&gad_campaignid=19324540046&gbraid=0AAAAAD_uzhBH6w54HpK0cgHZgAn8sPT2F&gclid=CjwKCAjwI97RBhBWEiwAa9rbXRRft9aFjxE5r-ZejNuDxeLZpC9tD.8QuSeV3rV3qUlQTm6nnte0nxoCs2QQAvD.BwE
- [79] «Fibonacci's numbers, complejidad y programación». Accedido: 16 de marzo de 2026. [En línea]. Disponible en: <https://www.ugr.es/~eaznar/fibo.htm>
- [80] «Cómo estimar los puntos de historia para mejorar la planificación con la metodología Agile», Lucidchart. Accedido: 16 de marzo de 2026. [En línea]. Disponible en: <https://www.lucidchart.com/blog/es/como-estimar-puntos-de-historia-metodologia-agile>
- [81] «El método MoSCoW (A5C53C2D01)». Accedido: 16 de marzo de 2026. [En línea]. Disponible en: <https://opencontent.upct.es/f0fc64e3b2d34f3a872a841a9c37b249/3f9bb933ba6440e59b0db2ecf65fc8a8/>
- [82] «¿Qué es un diagrama de Gantt? Características y ventajas de la plantilla gratuita | Atlassian». Accedido: 17 de marzo de 2026. [En línea]. Disponible en: <https://www.atlassian.com/es/agile/project-management/gantt-chart>
- [83] «Calendario académico», Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación. Accedido: 17 de marzo de 2026. [En línea]. Disponible en: <https://etsiit.ugr.es/docencia/grados/calendario>
- [84] Universidad de Granada, «Propuesta de Normativa de reconocimiento de créditos ECTS)». Granada, 21 de enero de 2016. Accedido: 19 de junio de 2026. [En línea]. Disponible en: <https://etsag.ugr.es/sites/centros/etsag/public/inline-files/Normativa%20de%20reconocimiento%20de%20cr%C3%A9ditos%20ECTS-5.pdf>
- [85] Hays, «guia-del-mercado-laboral-2026-digital». Accedido: 19 de junio de 2026. [En línea]. Disponible en: <https://www.hays.es/documents/d/hays-spain/guia-del-mercado-laboral-2026-digital>
- [86] «Guía Salarial España 2025: salarios y tendencias del mercado laboral», Michael Page. Accedido: 19 de junio de 2026. [En línea]. Disponible en: <https://www.michaelpage.es/prensa-estudios/estudios/estudios-de-remuneracion>
- [87] «Seguridad Social: Cotización / Recaudación de Trabajadores». Accedido: 19 de junio de 2026. [En línea]. Disponible en: <https://www.seg->

- social.es/wps/portal/wss/internet/Trabajadores/CotizacionRecaudacionTrabajadores/36537#SEA%20Inactividad
- [88] «Agencia Tributaria: 1. Por coeficientes de amortización lineal». Accedido: 19 de junio de 2026. [En línea]. Disponible en: <https://sede.agenciatributaria.gob.es/Sede/ayuda/manuales-videos-folletos/manuales-practicos/irpf-2025/c07-rendimientos-actividades-economicas-estimacion-directa/fase-1-determinacion-rendimiento-neto/amortizaciones-dotaciones-ejercicio-fiscalmente-deducibles/requisitos-generales/coeficientes-amortizacion-lineal.html>
 - [89] «BOE-A-2014-12328 Ley 27/2014, de 27 de noviembre, del Impuesto sobre Sociedades.» Accedido: 19 de junio de 2026. [En línea]. Disponible en: <https://www.boe.es/buscar/act.php?id=BOE-A-2014-12328>
 - [90] «Comprar DataGrip: Precios y licencias, Descuentos - Suscripción a JetBrains Toolbox», JetBrains. Accedido: 19 de junio de 2026. [En línea]. Disponible en: <https://www.jetbrains.com/es-es/datagrip/buy/>
 - [91] «Lista de precios de los servicios principales de Google Maps Platform | Google Maps Platform», Google for Developers. Accedido: 19 de junio de 2026. [En línea]. Disponible en: <https://developers.google.com/maps/billing-and-pricing/pricing?hl=es-419>
 - [92] «Google AI Pro y Ultra: accede a Gemini 3.1 Pro y más funciones», Gemini. Accedido: 19 de junio de 2026. [En línea]. Disponible en: <https://gemini.google/es/subscriptions/?hl=es>
 - [93] «Planes y precios Figma», Figma. Accedido: 19 de junio de 2026. [En línea]. Disponible en: <https://www.figma.com/es-es/pricing/>
 - [94] «Añadir servicio - Calculadora de precios de AWS». Accedido: 19 de junio de 2026. [En línea]. Disponible en: <https://calculator.aws/#/addService>
 - [95] «cuanto cuesta la publicación de una app - Comunidad de Google Play». Accedido: 19 de junio de 2026. [En línea]. Disponible en: <https://support.google.com/googleplay/thread/17857232/cuanto-cuesta-la-publicaci%C3%B3n-de-una-app?hl=es>
 - [96] A. Inc, «Become a member - Apple Developer Program», Apple Developer. Accedido: 19 de junio de 2026. [En línea]. Disponible en: <https://developer.apple.com/programs/enroll/>
 - [97] S. C. I. SL, «Buscar dominios libres», DonDominio. Accedido: 22 de junio de 2026. [En línea]. Disponible en: <https://www.dondominio.com/es/buscar/>
 - [98] WakaTime, «WakaTime - Dashboards for developers», WakaTime. Accedido: 19 de junio de 2026. [En línea]. Disponible en: <https://wakatime.com/dashboard>
 - [99] «BOE.es - DOUE-L-2016-80807 Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos y por el que se deroga la Directiva 95/46/CE (Reglamento general de protección de datos).» Accedido: 21 de junio de 2026. [En línea]. Disponible en: <https://www.boe.es/buscar/doc.php?id=DOUE-L-2016-80807>
 - [100] «BOE-A-2018-16673 Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales.» Accedido: 21 de junio de 2026. [En línea]. Disponible en: <https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673>
 - [101] «BOE-A-1982-11196 Ley Orgánica 1/1982, de 5 de mayo, de protección civil del derecho al honor, a la intimidad personal y familiar y a la propia imagen.» Accedido: 21 de junio de 2026. [En línea]. Disponible en: <https://www.boe.es/buscar/act.php?id=BOE-A-1982-11196>
 - [102] «BOE-A-2002-13758 Ley 34/2002, de 11 de julio, de servicios de la sociedad de la información y de comercio electrónico.» Accedido: 21 de junio de 2026. [En línea]. Disponible en: <https://www.boe.es/buscar/act.php?id=BOE-A-2002-13758>
 - [103] «BOE.es - DOUE-L-2022-81573 Reglamento (UE) 2022/2065 del Parlamento Europeo y del Consejo de 19 de octubre de 2022 relativo a un mercado único de servicios digitales y por el que se modifica la Directiva 2000/31/CE (Reglamento de Servicios Digitales).» Accedido: 21 de junio de 2026. [En línea]. Disponible en: <https://www.boe.es/buscar/doc.php?id=DOUE-L-2022-81573>
 - [104] «BOE-A-1994-26003 Ley 29/1994, de 24 de noviembre, de Arrendamientos Urbanos.» Accedido: 21 de junio de 2026. [En línea]. Disponible en: <https://www.boe.es/buscar/act.php?id=BOE-A-1994-26003>
 - [105] «BOE-A-2023-12203 Ley 12/2023, de 24 de mayo, por el derecho a la vivienda.» Accedido: 21 de junio de 2026. [En línea]. Disponible en: <https://www.boe.es/buscar/act.php?id=BOE-A-2023-12203>

- [106] «DataGrip | JetBrains for Data», JetBrains. Accedido: 20 de junio de 2026. [En línea]. Disponible en: <https://www.jetbrains.com/es-es/datagrip/>
- [107] «sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 · Hugging Face». Accedido: 21 de junio de 2026. [En línea]. Disponible en: <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2>
- [108] Y. Hu, Y. Koren, y C. Volinsky, «Collaborative Filtering for Implicit Feedback Datasets», *2008 Eighth IEEE Int. Conf. Data Min.*, pp. 263-272, 2008.
- [109] Y. Ding y X. Li, «Time weight collaborative filtering», en *Proceedings of the 14th ACM International Conference on Information and Knowledge Management*, en CIKM '05. New York, NY, USA: Association for Computing Machinery, 2005, pp. 485-492. doi: 10.1145/1099554.1099689.
- [110] P. Ardagelou y A. Arampatzis, «A Half-Life Decaying Model for Recommender Systems with Matrix Factorization.», en *TDDL/MDQal/Futurity@ TPD*, 2017.
- [111] H. Li, P. Liang, y J. Hu, «A Network Resource Allocation Recommendation Method with An Improved Similarity Measure». 2023. [En línea]. Disponible en: <https://arxiv.org/abs/2307.03399>
- [112] P. S. Bullen, *Handbook of means and their inequalities*, vol. 560. Springer Science & Business Media, 2013.
- [113] C. Van Rijsbergen, «Information retrieval: theory and practice», en *Proceedings of the joint IBM/University of Newcastle upon tyne seminar on data base systems*, Butterworth-Heinemann Oxford, UK, 1979, pp. 1-14.
- [114] C. Chung, «Singleton», en *Pro Objective-C Design Patterns for iOS*, Springer, 2011, pp. 97-105.
- [115] «Google Maps Platform», GitHub. Accedido: 15 de junio de 2026. [En línea]. Disponible en: <https://github.com/googlemaps>
- [116] «Mapbox | Maps, Navigation, Search, and Data». Accedido: 15 de junio de 2026. [En línea]. Disponible en: <https://www.mapbox.com>
- [117] «HERE Developer Portal | DevDocs | Location Data | HERE». Accedido: 15 de junio de 2026. [En línea]. Disponible en: <https://www.here.com/developer>
- [118] «Leaflet — an open-source JavaScript library for interactive maps». Accedido: 15 de junio de 2026. [En línea]. Disponible en: <https://leafletjs.com/>
- [119] «API - OpenStreetMap Wiki». Accedido: 15 de junio de 2026. [En línea]. Disponible en: <https://wiki.openstreetmap.org/wiki/API>
- [120] «Mapbox Web Technology Usage Trends», BuiltWith. Accedido: 15 de junio de 2026. [En línea]. Disponible en: <https://trends.builtwith.com/mapping/Mapbox>
- [121] «Google Maps API Web Technology Usage Trends», BuiltWith. Accedido: 15 de junio de 2026. [En línea]. Disponible en: <https://trends.builtwith.com/mapping/Google-Maps-API>
- [122] «Detalles de la cobertura de la API de Places Aggregate | Places Aggregate API», Google for Developers. Accedido: 15 de junio de 2026. [En línea]. Disponible en: <https://developers.google.com/maps/documentation/places-aggregate/coverage?hl=es-419>
- [123] «axios | Promise based HTTP client». Accedido: 17 de junio de 2026. [En línea]. Disponible en: <https://axios.rest/>